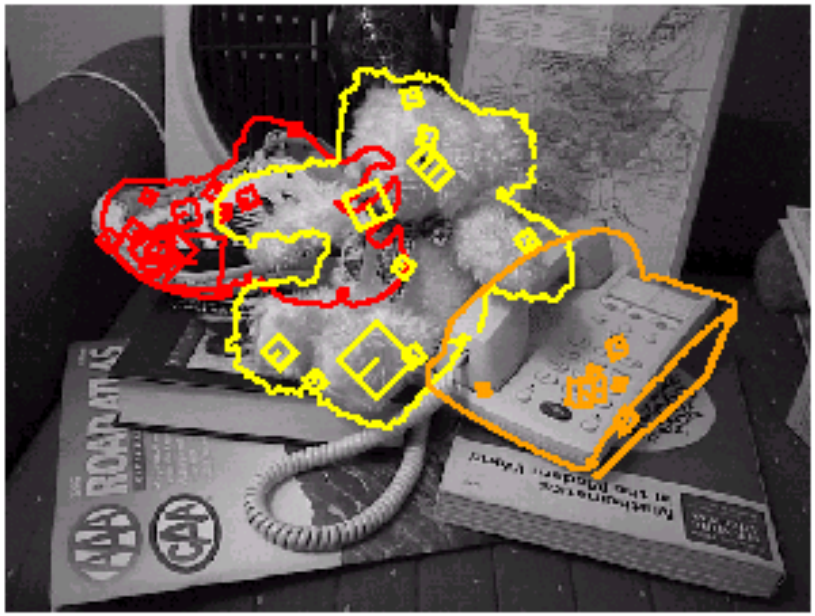
Computer Vision
Lecture 3
Alignment

桑农 王岳环

Slides from CV course with **Svetlana Lazebnik**

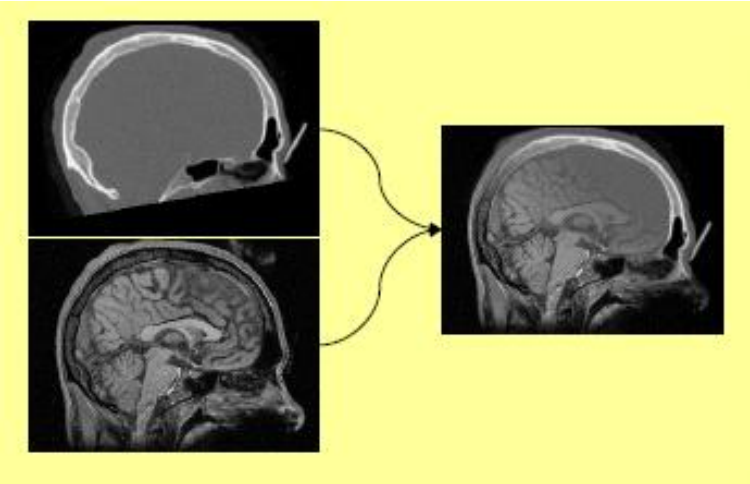
Motivation: Recognition

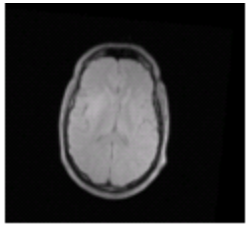


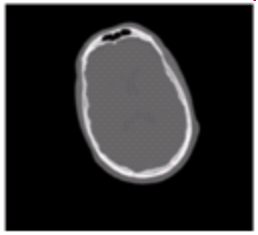


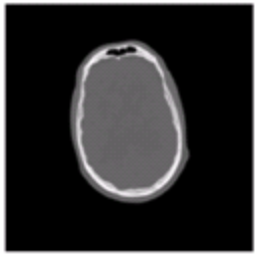
Figures from David Lowe

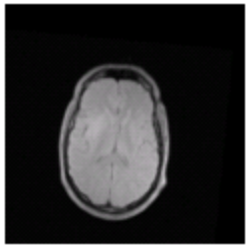
Motivation: medical image registration

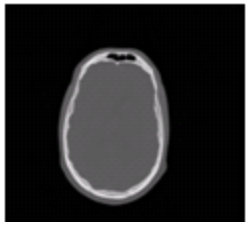




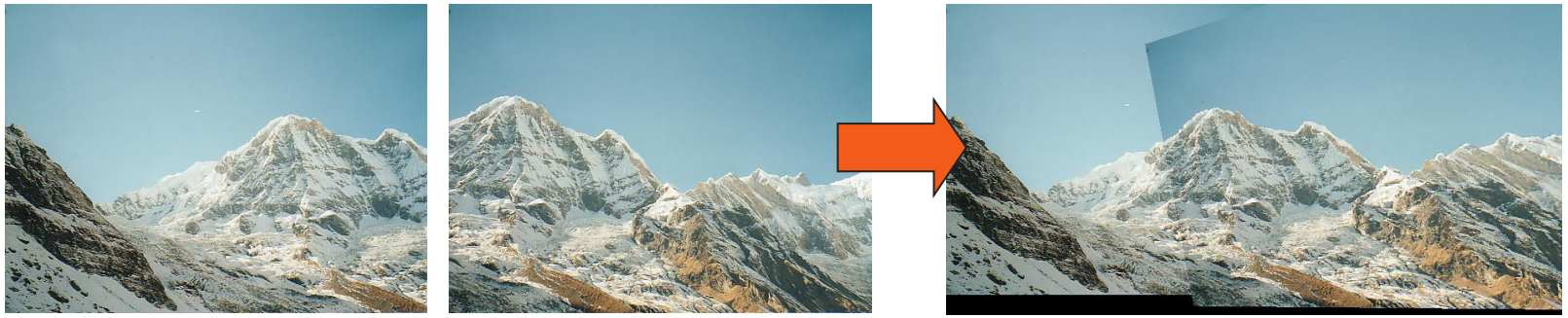




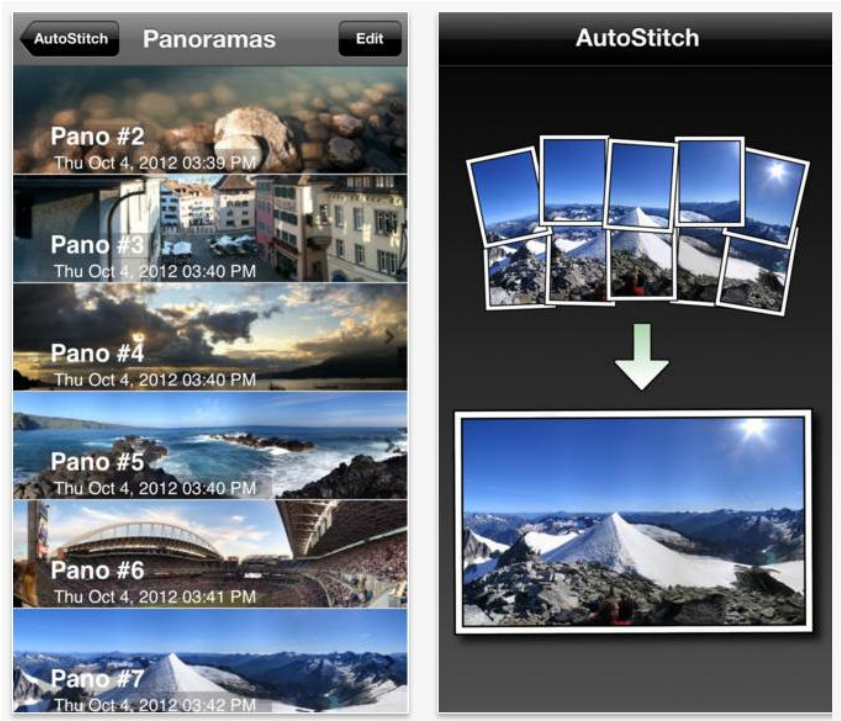




Motivation:Mosaics



Panorama stitching



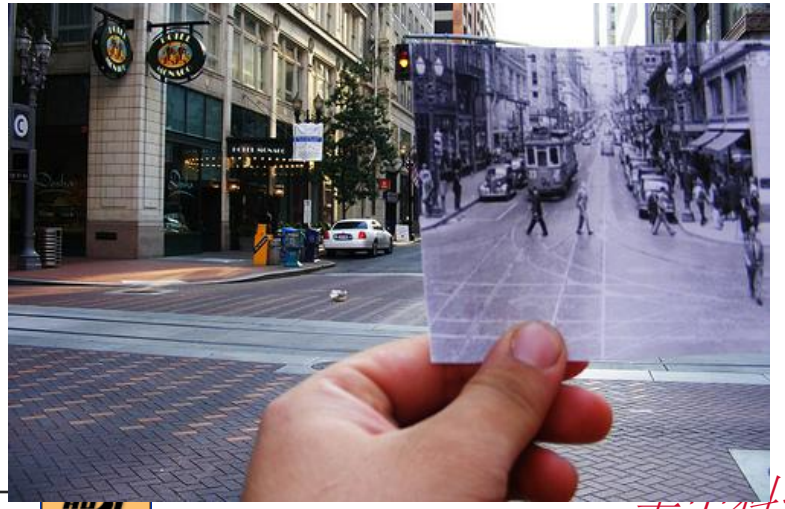
http://matthewalunbrown.com/autostitch/autostitch.html

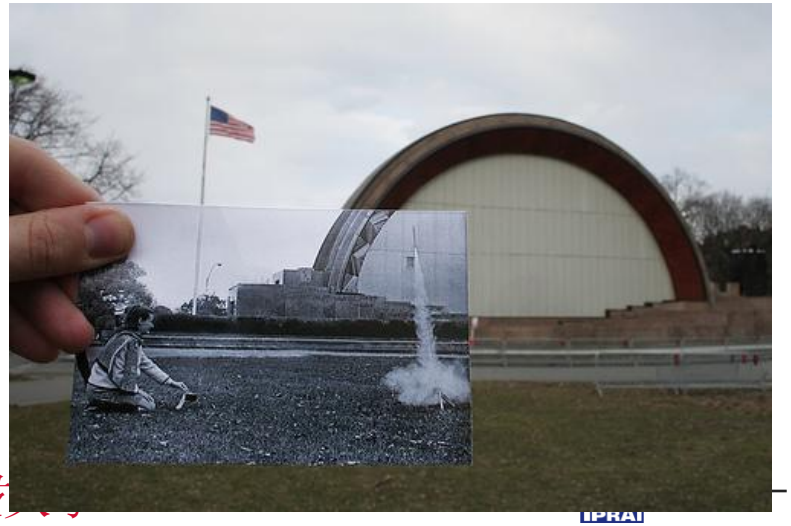
Motivation: Image Blending

- A look into the past

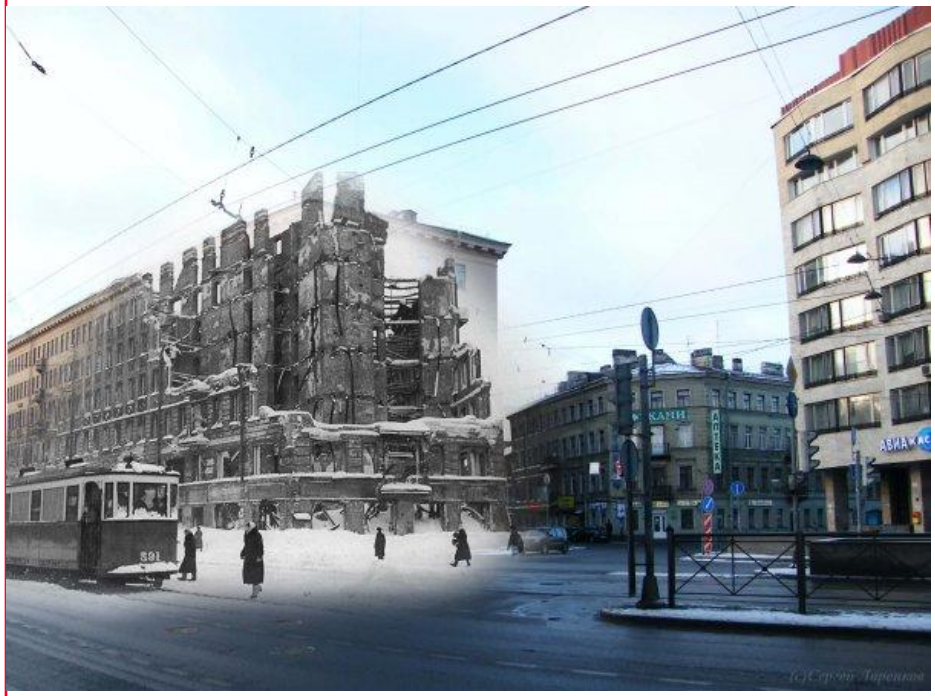






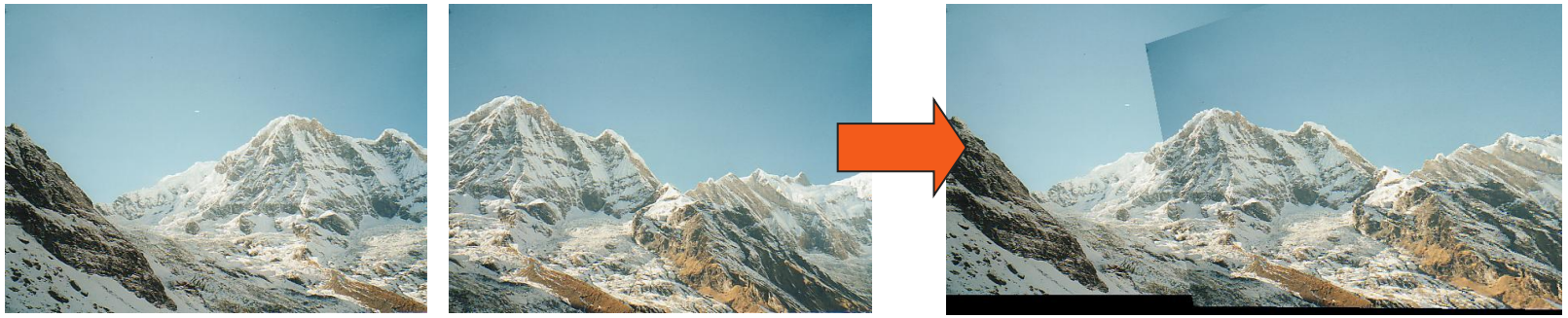


- A look into the past





Alignment challenges



Small degree of overlap
Intensity changes





Occlusion, clutter, viewpoint change ……

Motion models

Motion models

What happens when we take two images with a camera and try to align them?

- translation?
- rotation?
- scale?
- affine?
- perspective?





Szelis

Image Warping

Image Warping

image filtering: change *range* of image

$$g(x) = h(f(x))$$

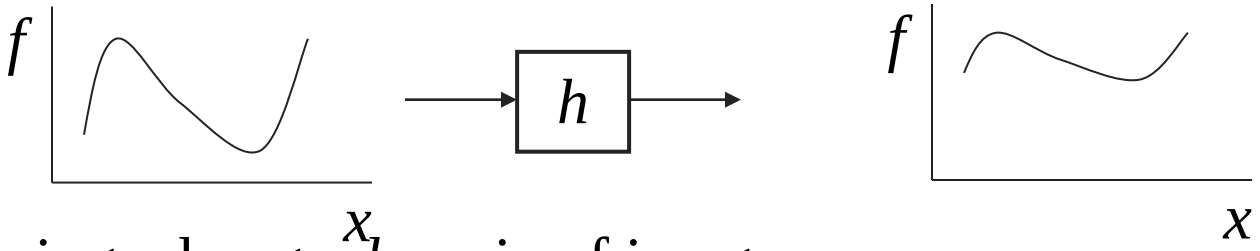
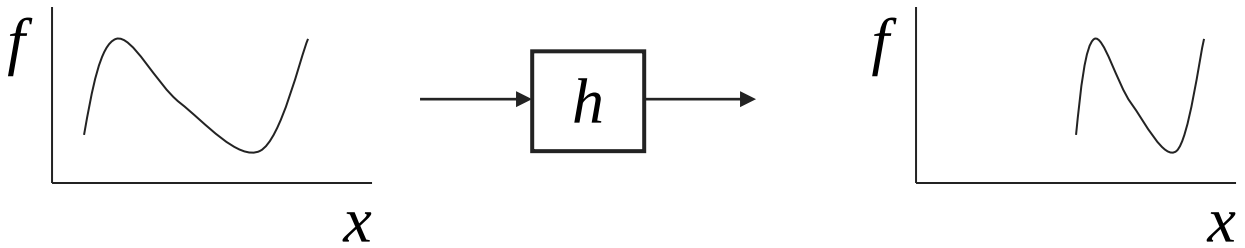


image warping: change *domain* of image

$$g(x) = f(h(x))$$



Szelis

Parametric (global) warping

Examples of parametric warps:



translation



rotation



aspect



affine



perspective

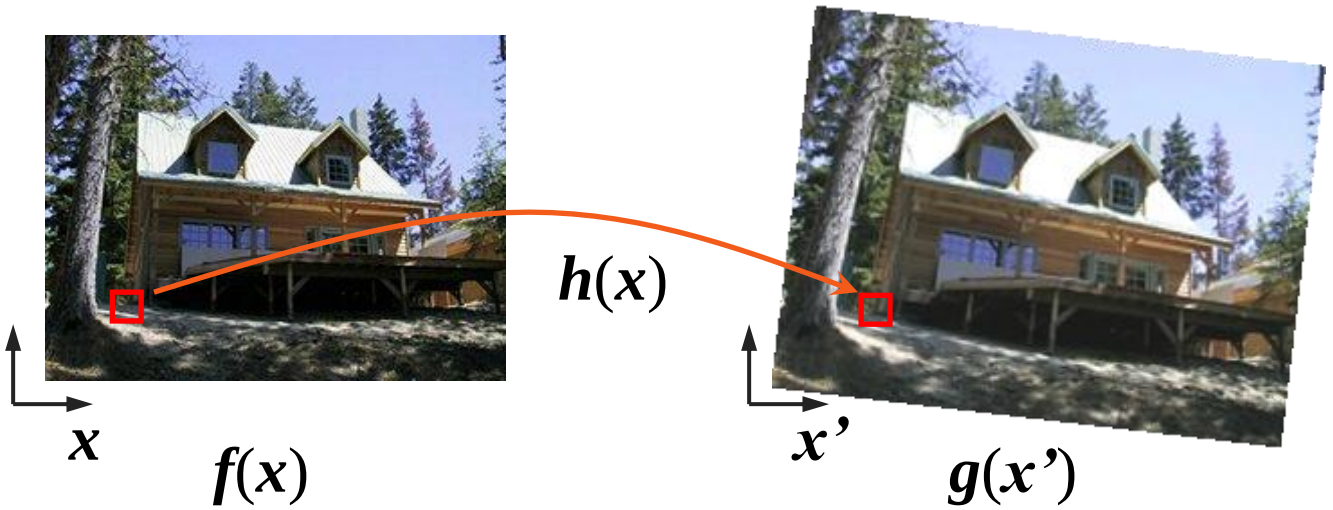


cylindrical

Szelis

Image Warping

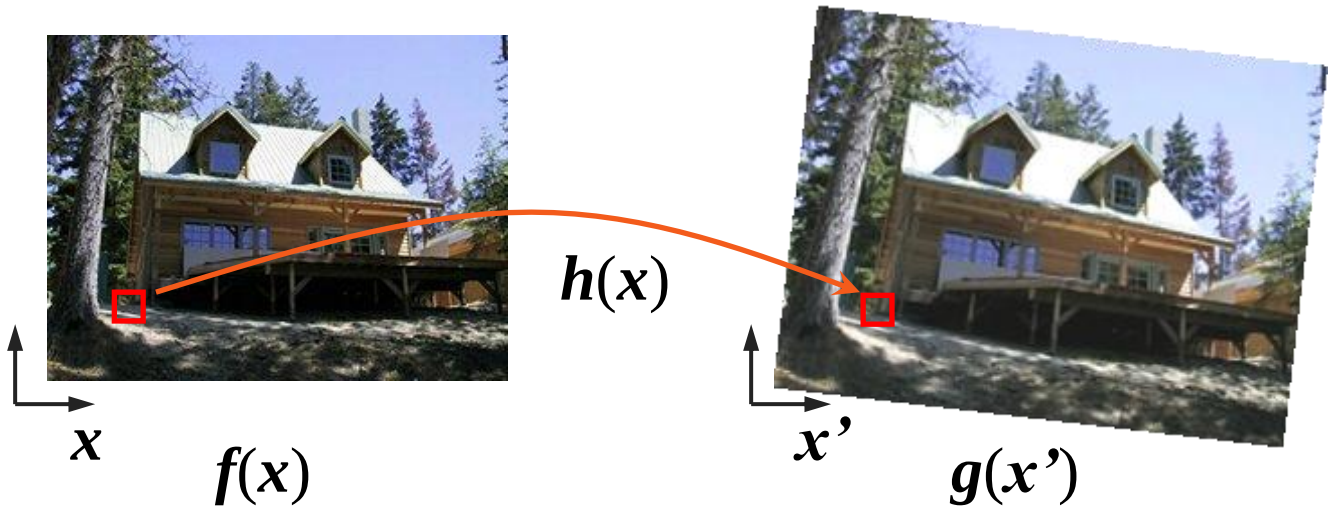
Given a coordinate transform $x' = h(x)$ and a source image $f(x)$, how do we compute a transformed image $g(x') = f(h(x))$?



Forward Warping

Send each pixel $f(x)$ to its corresponding location $x' = h(x)$ in $g(x')$

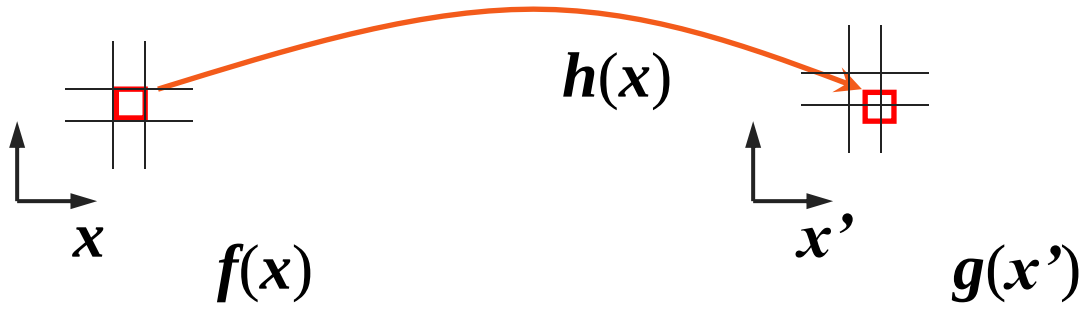
- What if pixel lands "between" two pixels?



Forward Warping

Send each pixel $f(x)$ to its corresponding location $x' = h(x)$ in $g(x')$

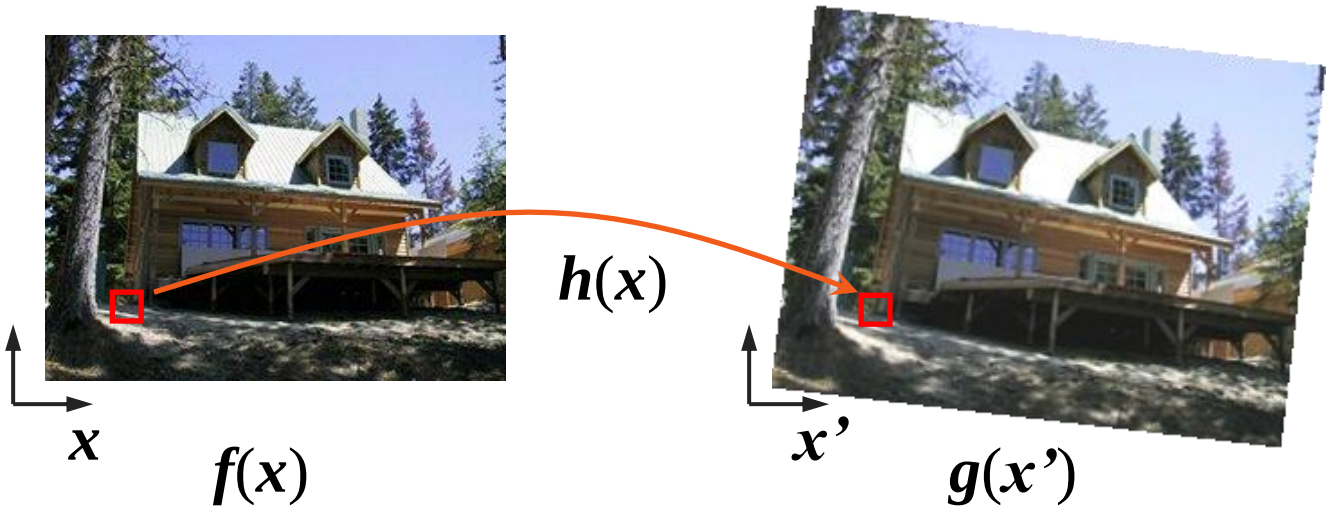
- What if pixel lands "between" two pixels?
- Answer: add "contribution" to several pixels, normalize later (*splatting*)



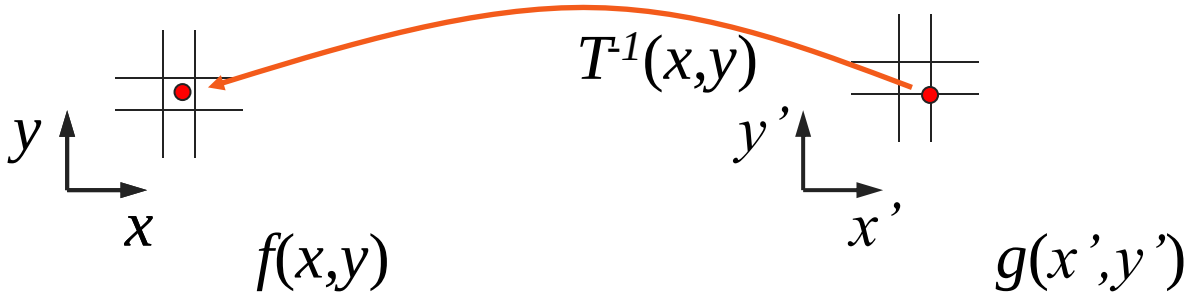
Inverse Warping

Get each pixel $g(x')$ from its corresponding location $x' = h(x)$ in $f(x)$

- What if pixel comes from "between" two pixels?



Inverse warping



Get each pixel $g(x',y')$ from its corresponding location

$(x,y) = T^{-1}(x',y')$ in the first image

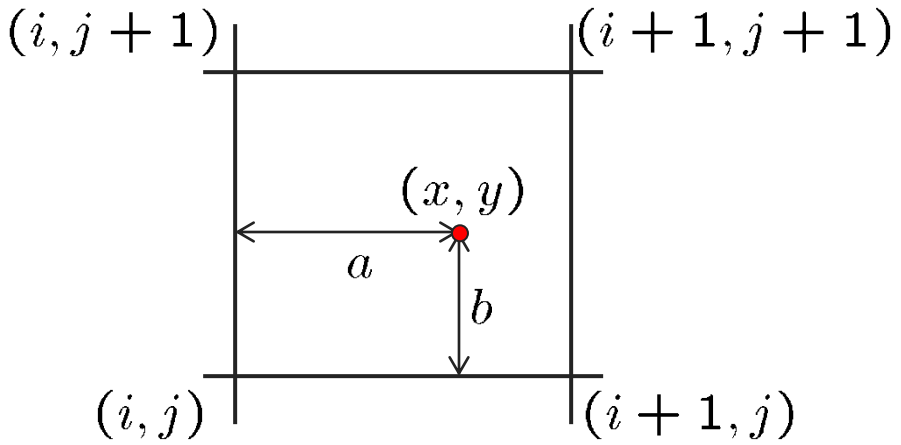
Q: what if pixel comes from "between" two pixels?

A: *Interpolate* color value from neighbors

nearest neighbor, bilinear…

Bilinear interpolation

Sampling at *f(x,y)*:



$$
\begin{aligned}
f(x,y) = \quad & (1-a)(1-b) \quad f[i,j] \\
& +a(1-b) \quad\;\; f[i+1,j] \\
& \quad +ab \quad\quad\;\; f[i+1,j+1] \\
& +(1-a)b \quad\;\; f[i,j+1]
\end{aligned}
$$

Interpolation

Possible interpolation filters:

- nearest neighbor
- bilinear
- bicubic (interpolating)
- sinc / FIR

Needed to prevent “jaggies” and “texture crawl”



Szelis

2D coordinate transformations

translation: $\boldsymbol{x}' = \boldsymbol{x} + \boldsymbol{t}$ $\boldsymbol{x} = (x,y)$

rotation: $\boldsymbol{x}' = \boldsymbol{R}\,\boldsymbol{x} + \boldsymbol{t}$

similarity: $\boldsymbol{x}' = s\,\boldsymbol{R}\,\boldsymbol{x} + \boldsymbol{t}$

affine: $\boldsymbol{x}' = \boldsymbol{A}\,\boldsymbol{x} + \boldsymbol{t}$

perspective: $\underline{\boldsymbol{x}}' \cong \boldsymbol{H}\,\underline{\boldsymbol{x}}$ $\underline{\boldsymbol{x}} = (x,y,1)$

($\underline{\boldsymbol{x}}$ is a *homogeneous* coordinate)

These all form a nested *group*

Szelis

Basic 2D Transformations

Basic 2D transformations as 3x3 matrices

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} \boldsymbol{x}' \\ \boldsymbol{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{s}_x & 0 & 0 \\ 0 & \boldsymbol{s}_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{y} \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\Theta & -\sin\Theta & 0 \\ \sin\Theta & \cos\Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Rotate

$$\begin{bmatrix} \boldsymbol{x}' \\ \boldsymbol{y}' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & \boldsymbol{sh}_x & 0 \\ \boldsymbol{sh}_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{y} \\ 1 \end{bmatrix}$$

Shear

Source: Alyosha Efros

2D Affine Transformations

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\Theta & -\sin\Theta & 0 \\ \sin\Theta & \cos\Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Affine transformations are combinations of …

Linear transformations, and

Translations

Parallel lines remain parallel

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Graumar

Projective Transformations

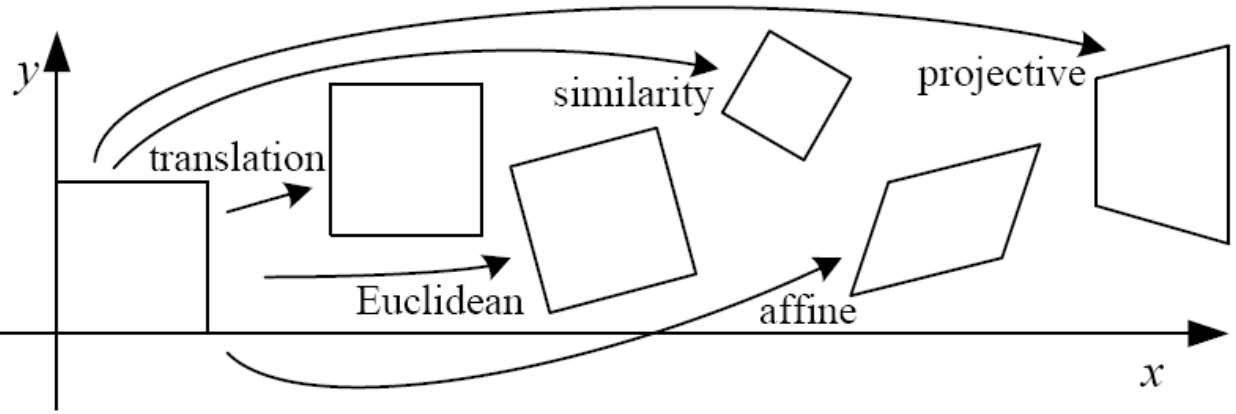
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

Projective transformations:

Affine transformations, and

Projective warps

Parallel lines do not necessarily remain parallel



Grauman

Fitting an affine transformation



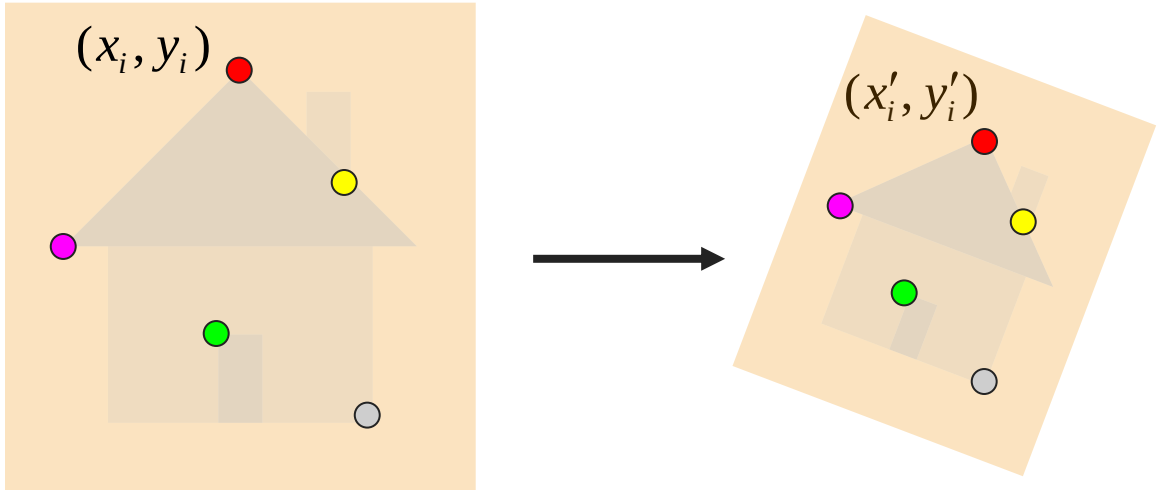


Affine model approximates perspective projection of planar objects.

Figures from David Lowe, ICCV 1999

Fitting an affine transformation

- Assuming we know the correspondences, how do we get the transformation?

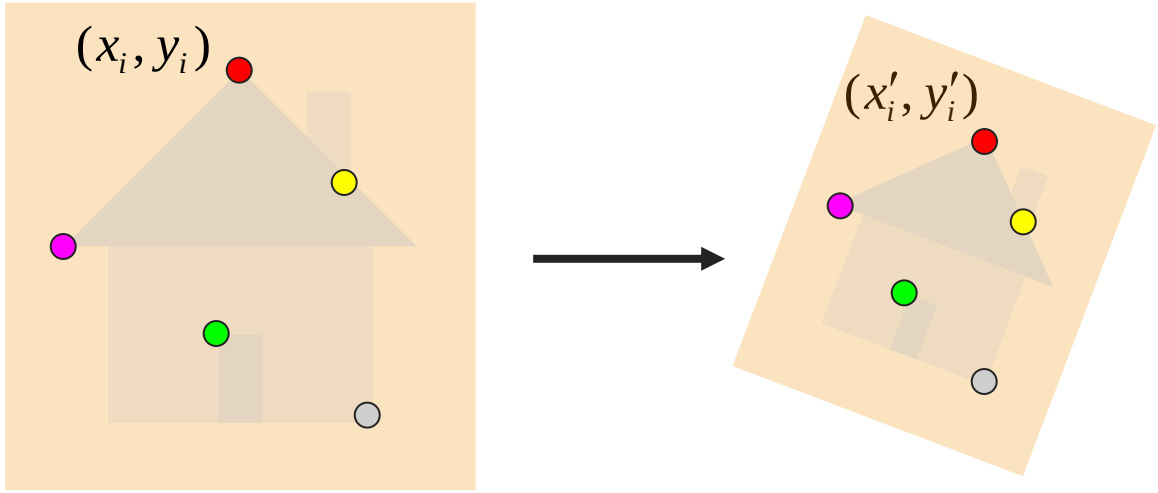


$$\begin{bmatrix} x_i' \\ y_i' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

Grauman

Fitting an affine transformation

- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x_i' \\ y_i' \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

$$\begin{bmatrix} & \\ & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} & \\ & \end{bmatrix}$$

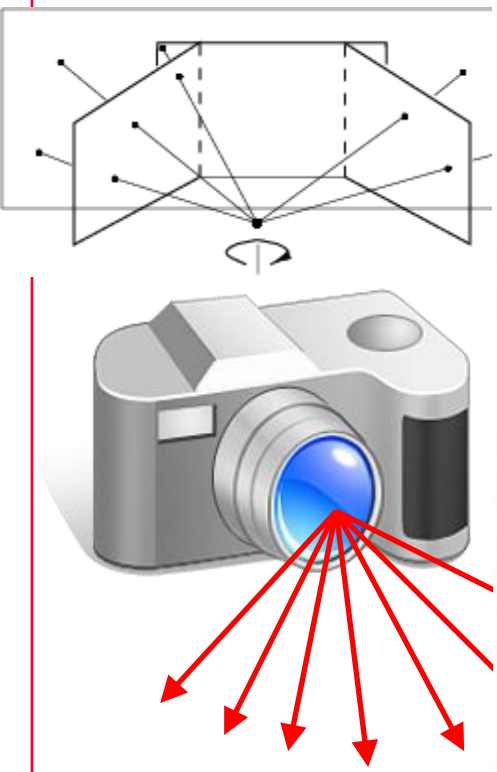
Fitting an affine transformation

$$\begin{bmatrix} & & \cdots & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ & & \cdots & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \cdots \\ x_i' \\ y_i' \\ \cdots \end{bmatrix}$$

- How many matches (correspondence pairs) do we need to solve for the transformation parameters?
- Once we have solved for the parameters, how do we compute the coordinates of the corresponding point for ? (x_{new}, y_{new})

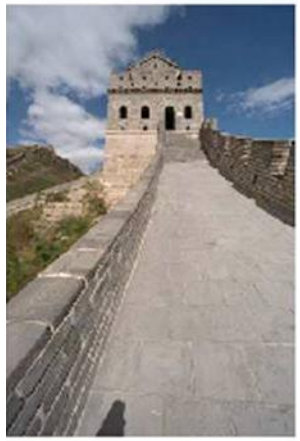
Grauman

Panoramas











. . .



image from S. Seitz

Obtain a wider angle view by combining multiple images.

Grauman

How to stitch together a panorama?

Basic Procedure

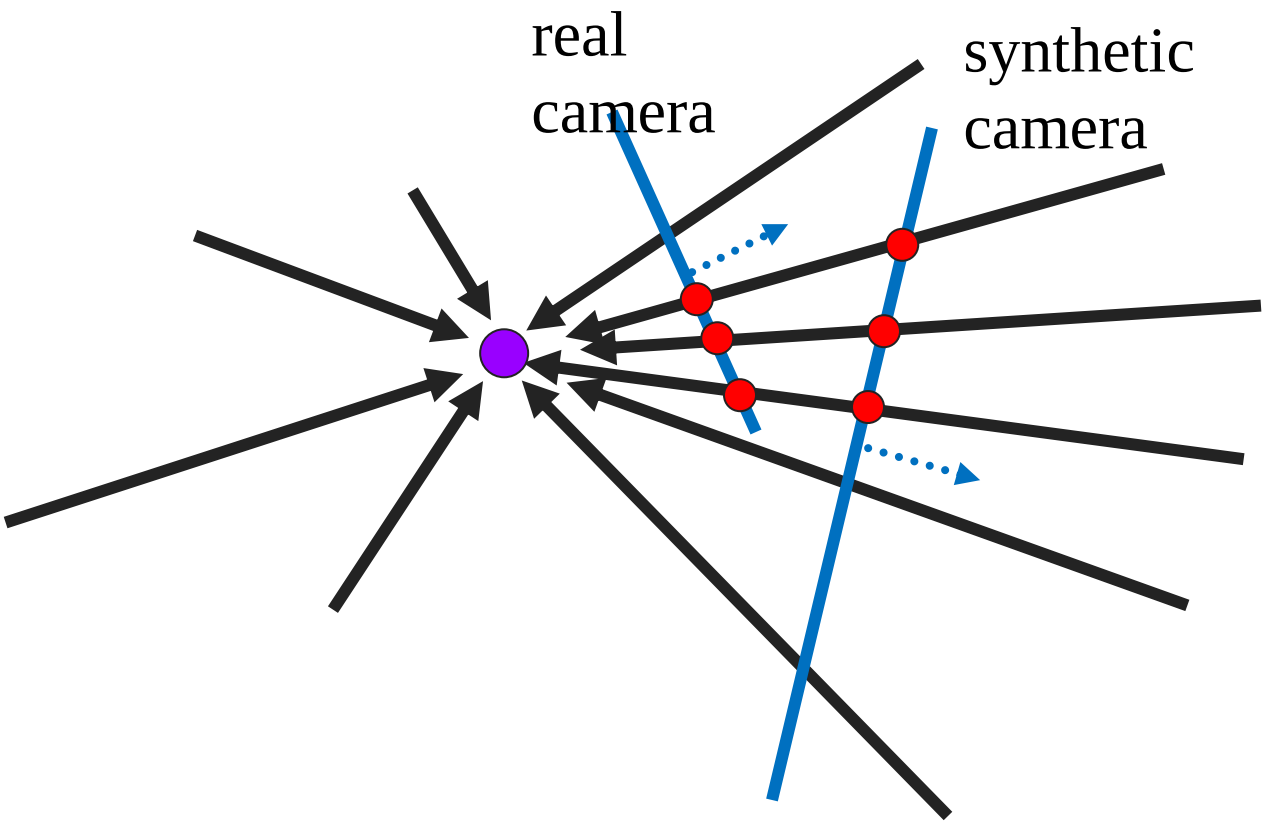
- Take a sequence of images from the same position
  - Rotate the camera about its optical center
- Compute transformation between second image and first
- Transform the second image to overlap with the first
- Blend the two together to create a mosaic
- (If there are more images, repeat)

…but **wait**, why should this work at all?

- What about the 3D geometry of the scene?
- Why aren't we using it?

Source: Steve Seitz

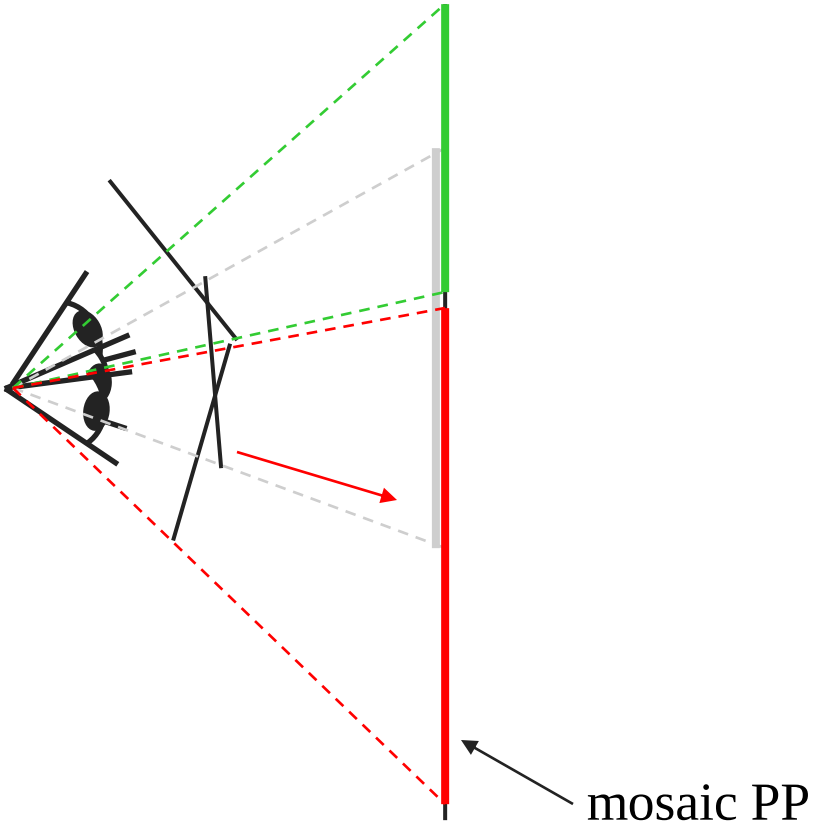
Panoramas: generating synthetic views



Can generate any synthetic camera view
as long as it has **the same center of projection**!

Source: Alyosha Efros

Image reprojection



The mosaic has a natural interpretation in 3D

- The images are reprojected onto a common plane
- The mosaic is formed on this plane
- Mosaic is a *synthetic wide-angle camera*

Source: Steve Seitz

Homography

How to relate two images from the same camera center?

how to map a pixel from PP1 to PP2?

Think of it as a 2D **image warp** from one image to another.

A projective transform is a mapping between any two PPs with the same center of projection

rectangle should map to arbitrary quadrilateral

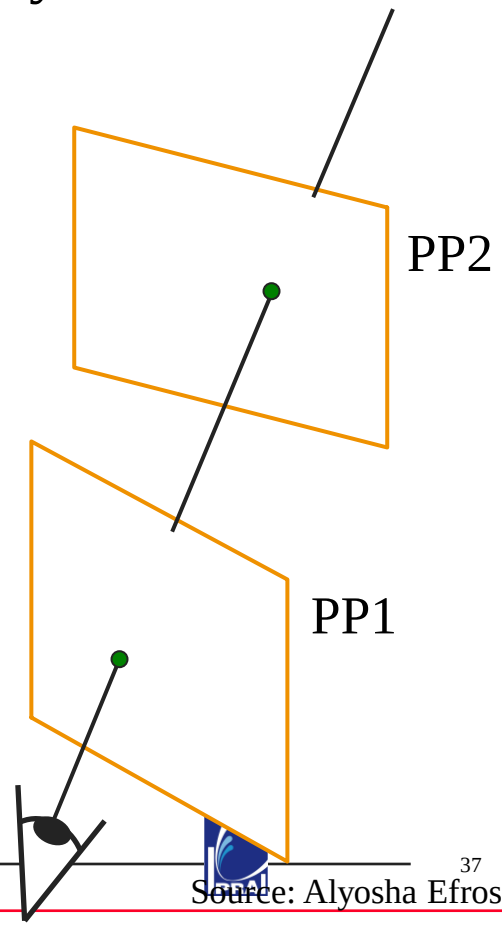
parallel lines aren't

but must preserve straight lines

called **Homography**

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\mathbf{p'} \qquad \mathbf{H} \qquad \mathbf{p}$$

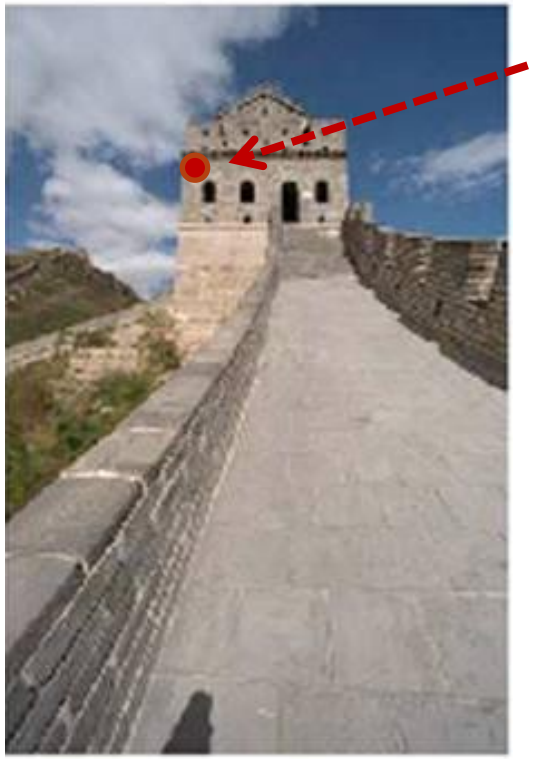


Source: Alyosha Efros

Homography

(x, y)





$\left(wx'/w, wy'/w\right) = (x', y')$

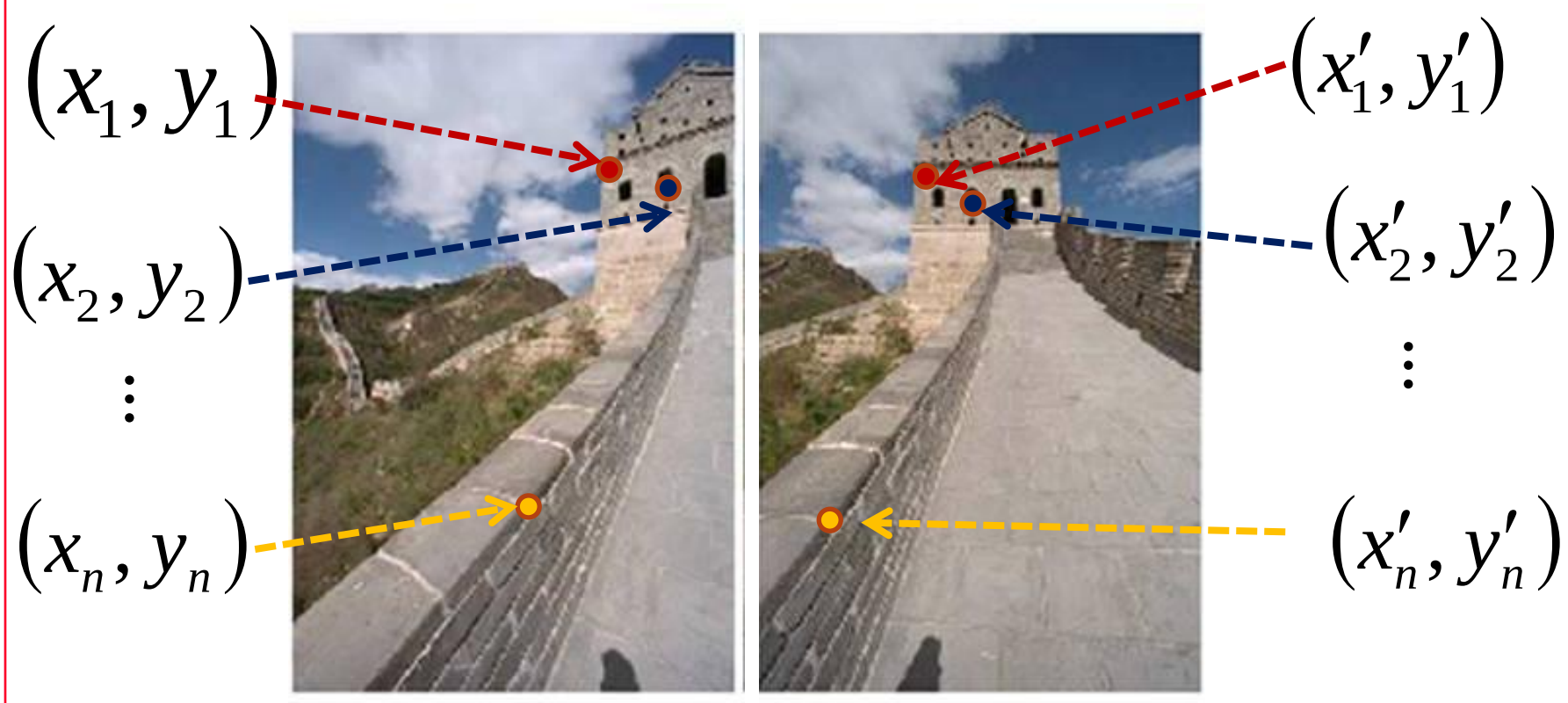
To **apply** a given homography **H**

- Compute **p'** = **Hp** (regular matrix multiply)
- Convert **p'** from homogeneous to image coordinates

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

p' **H** **p**

Homography



To **compute** the homography given pairs of corresponding points in the images, we need to set up an equation where the parameters of **H** are the unknowns…

Grauman

Multi-Scale Oriented Patches

Interest points

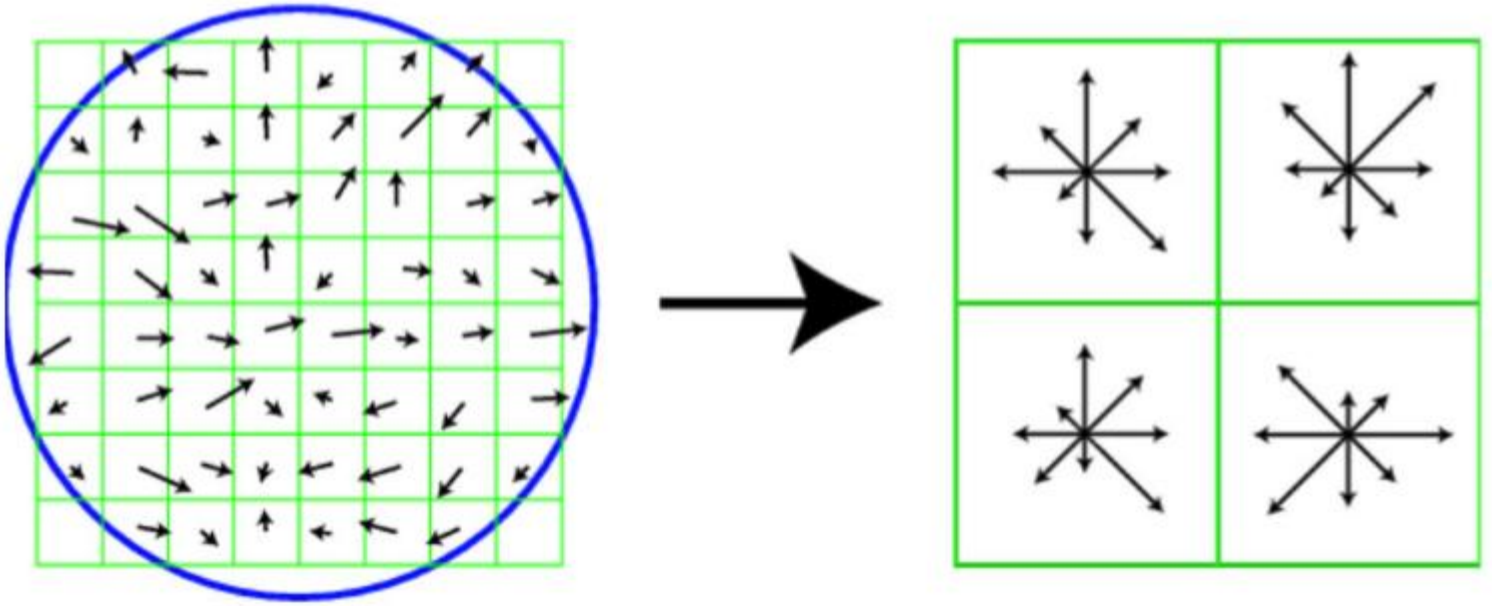
- Multi-scale Harris corners
- Orientation from blurred gradient
- Geometrically invariant to similarity transforms

Descriptor vector

- Bias/gain normalized sampling of local patch (8x8)
- Photometrically invariant to affine changes in intensity

Szelis

Multi-Scale Oriented Patches



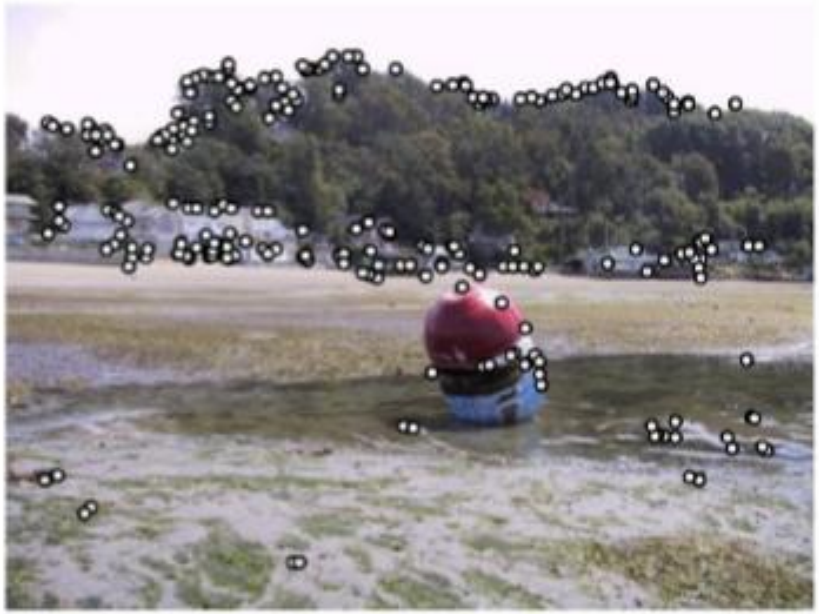
(a) image gradients

(b) keypoint descriptor

Szelis

Features

Distribute points evenly over the image



(a) Strongest 250



(b) Strongest 500

Szelis

Adaptive non-maximal suppression (ANMS)



(a) Strongest 250



(b) Strongest 500



(c) ANMS 250, $r = 24$

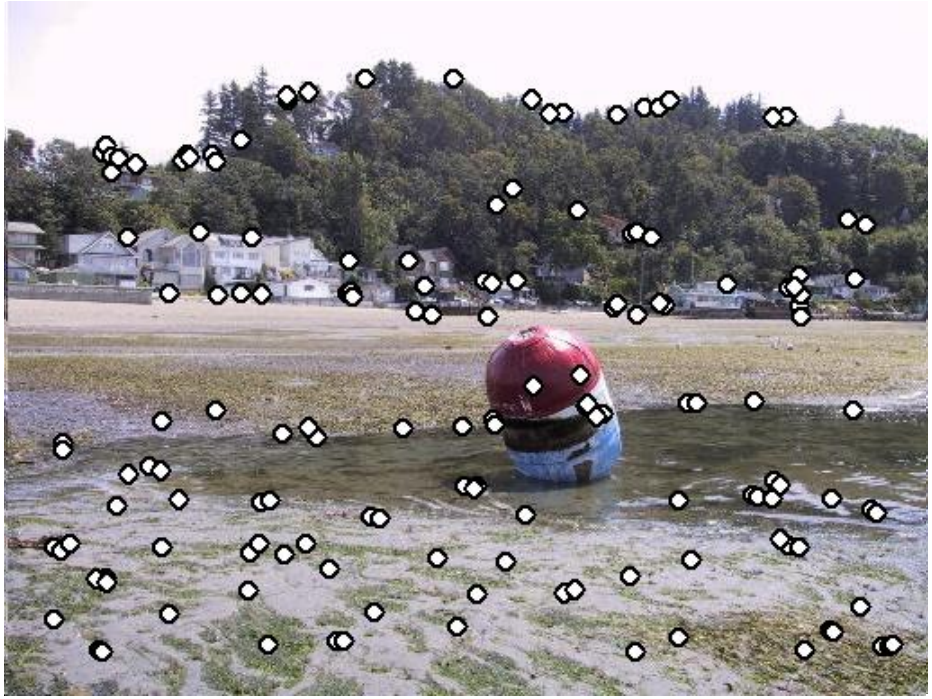


(d) ANMS 500, $r = 16$

$$r_i = \min_j |\mathbf{x}_i - \mathbf{x}_j|, \text{ s.t. } f(\mathbf{x}_i) < c_{\text{robust}} f(\mathbf{x}_j), \ \mathbf{x}_j \in \mathcal{I}$$

Features

Distribute points evenly over the image



Szelis

Feature descriptors

- Simplest descriptor: vector of raw intensity values
- How to compare two such vectors?

Sum of squared differences (SSD)

$$\mathrm{SSD}(\mathbf{u},\mathbf{v}) = \sum_i (u_i - v_i)^2$$

Not invariant to intensity change

Normalized correlation

$$\rho(\mathbf{u},\mathbf{v}) = \frac{(\mathbf{u}-\bar{\mathbf{u}})}{\|\mathbf{u}-\bar{\mathbf{u}}\|} \cdot \frac{(\mathbf{v}-\bar{\mathbf{v}})}{\|\mathbf{v}-\bar{\mathbf{v}}\|} = \frac{\sum_i (u_i - \bar{\mathbf{u}})(v_i - \bar{\mathbf{v}})}{\sqrt{\left(\sum_j (u_j - \bar{\mathbf{u}})^2\right)\left(\sum_j (v_j - \bar{\mathbf{v}})^2\right)}}$$

Invariant to affine intensity change

Solving for homographies

$$\mathbf{p'} = \mathbf{Hp}$$

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Can set scale factor i=1. So, there are 8 unknowns.

Set up a system of linear equations:

$$\mathbf{Ah} = \mathbf{b}$$

where vector of unknowns h = [a,b,c,d,e,f,g,h]^T

Need at least 8 eqs, but the more the better…

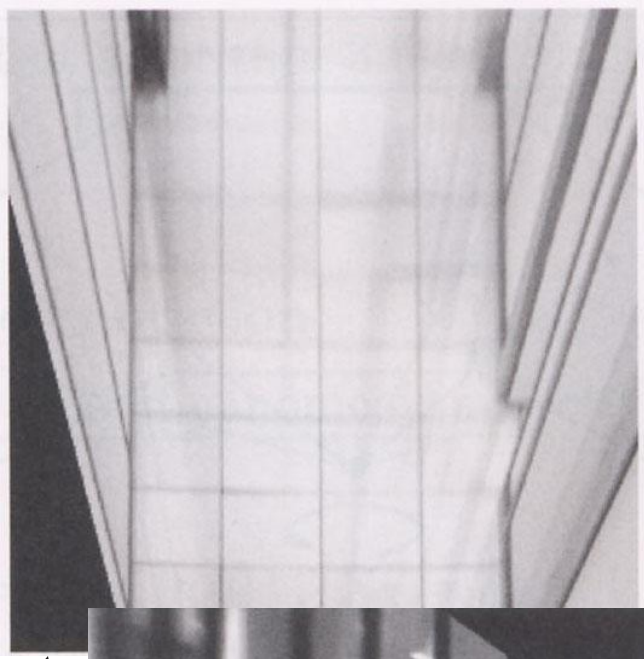
Solve for h. If overconstrained, solve using least-squares:

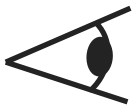
$$\min \|Ah - b\|^2$$

Grauman

Image warping with homographies







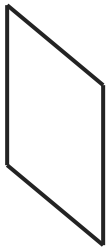




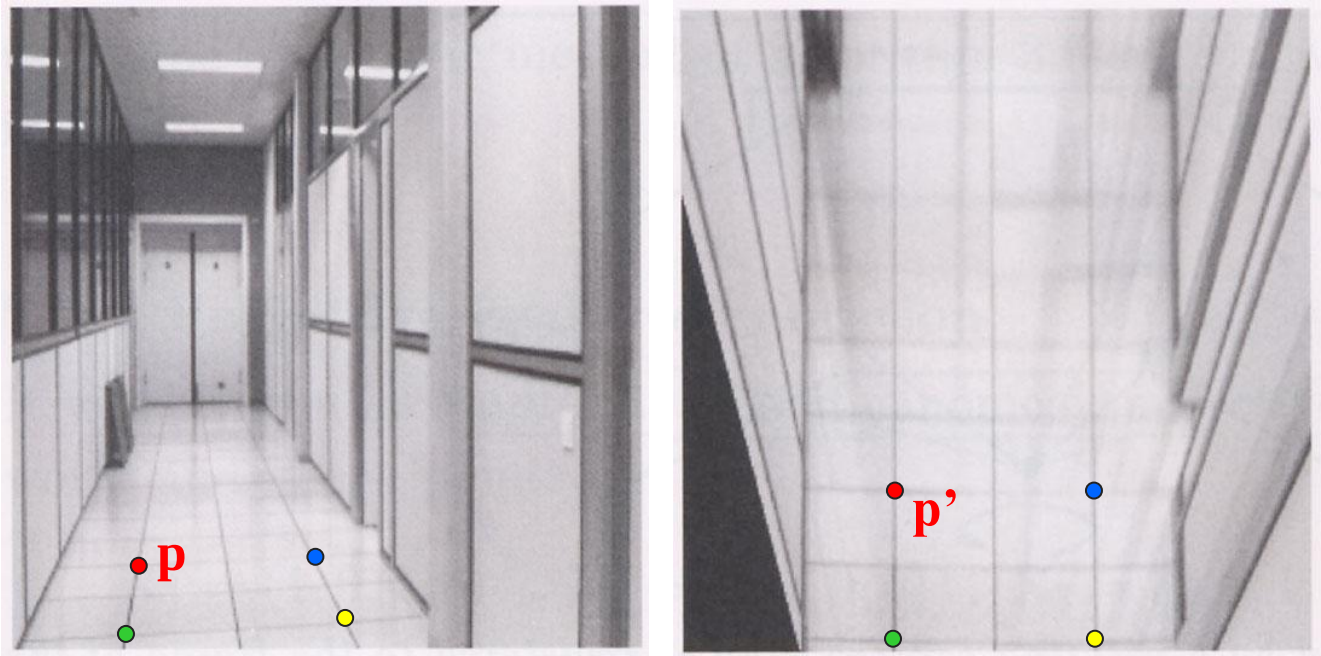
image plane in front

black area
where no pixel
maps to



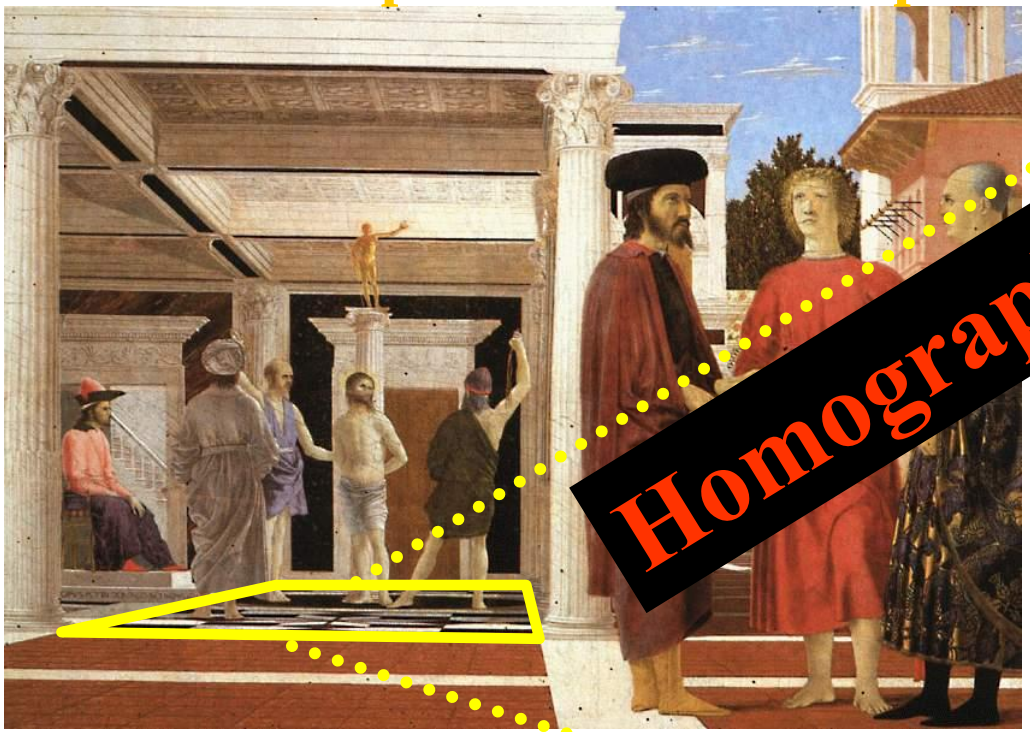
Source: Steve Seitz

Image rectification



Analysing patterns and shapes

What is the shape of the b/w floor pattern?



Homography



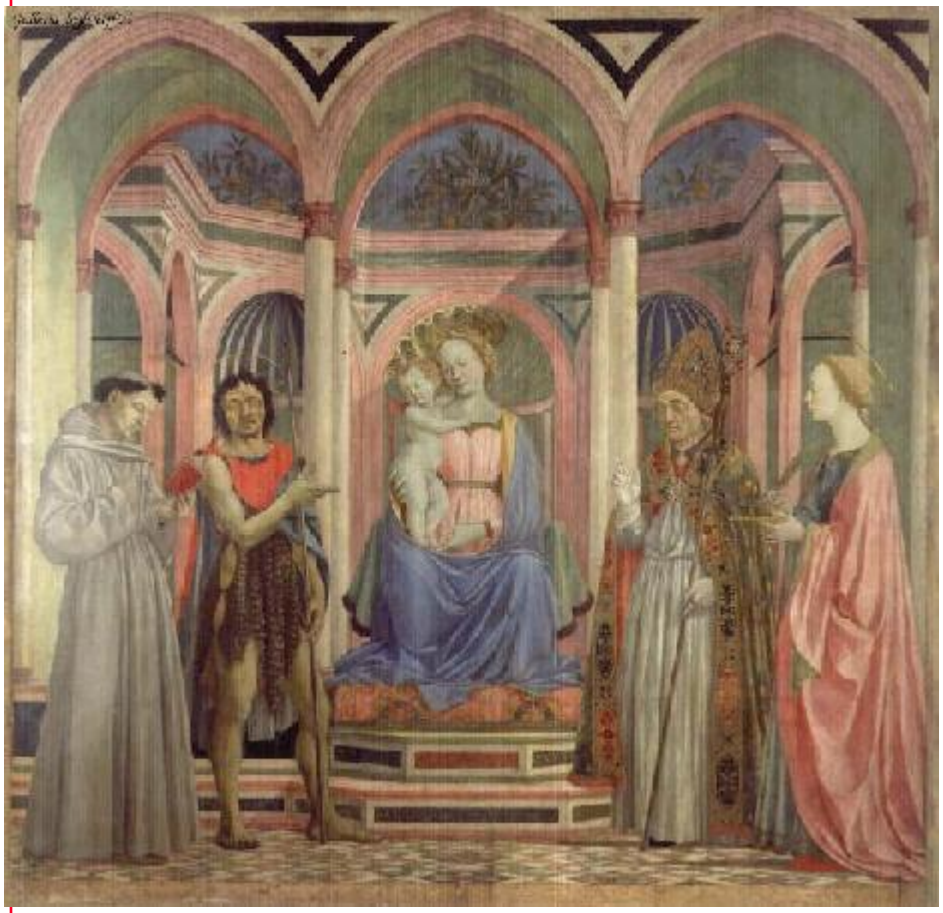
The floor (enlarged)



Automatically rectified floor

Slide from Criminisi

Analysing patterns and shapes



What is the (complicated) shape of the floor pattern?



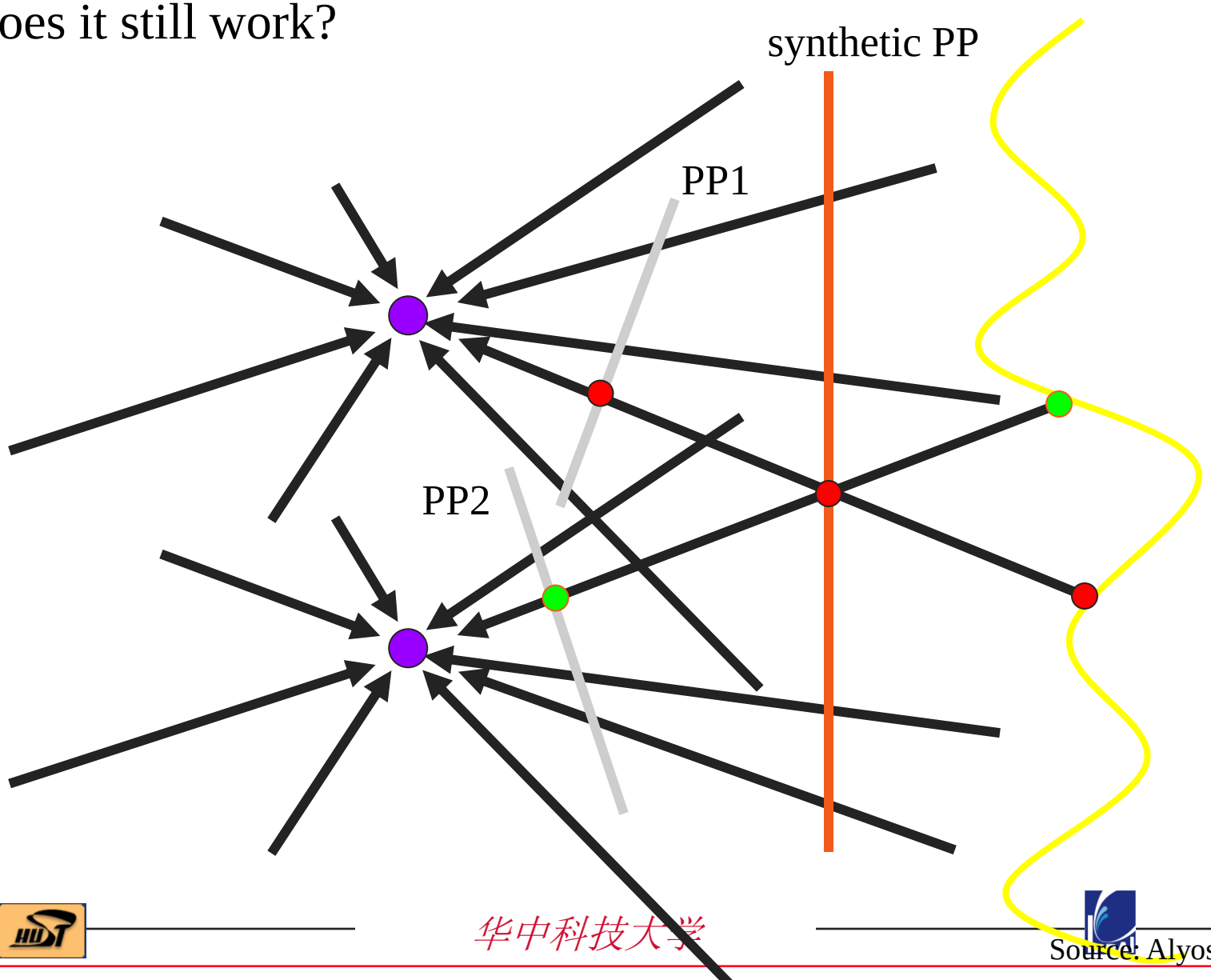
Automatically rectified floor

***St. Lucy Altarpiece,* D. Veneziano**

Slide from Criminisi

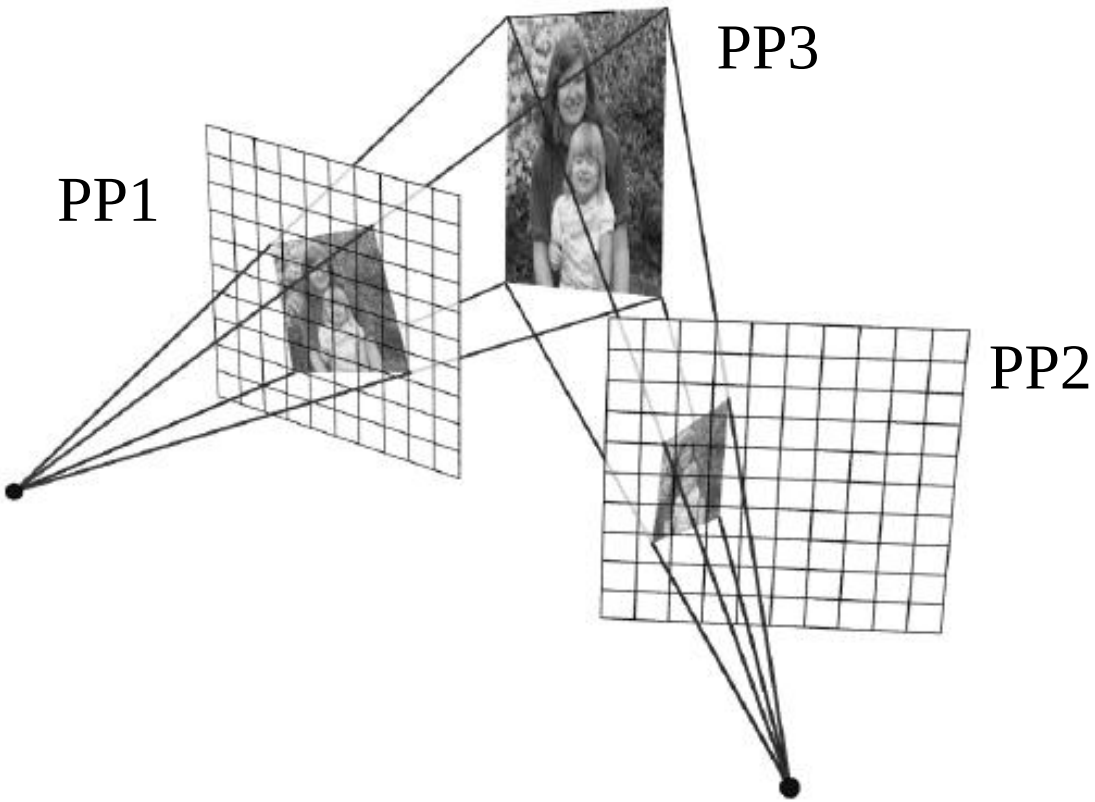
changing camera center

Does it still work?



Source: Alyosha Efros

Planar scene (or far away)



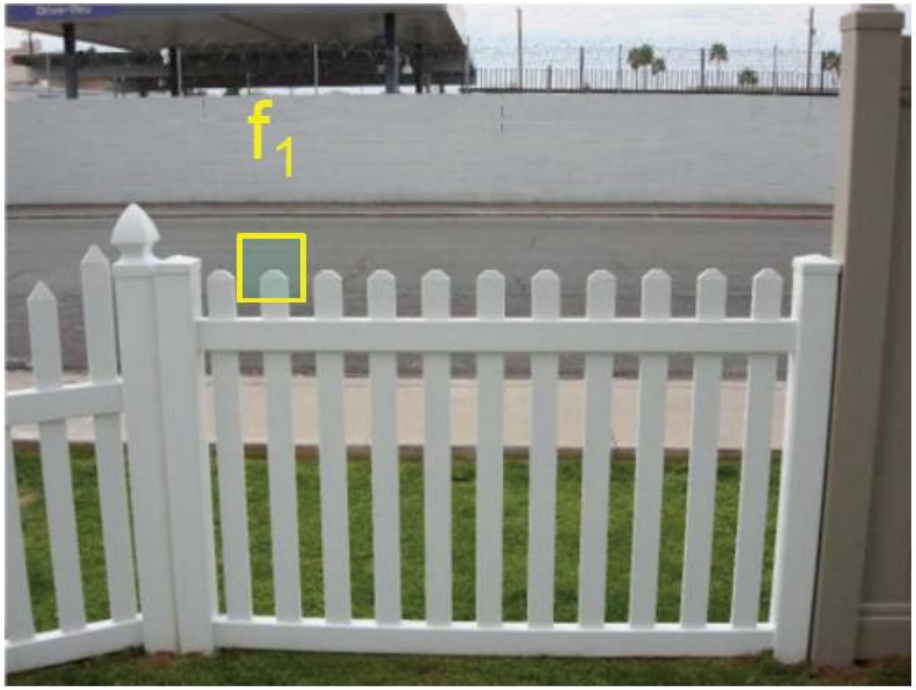
PP3 is a projection plane of both centers of projection, so we are OK!

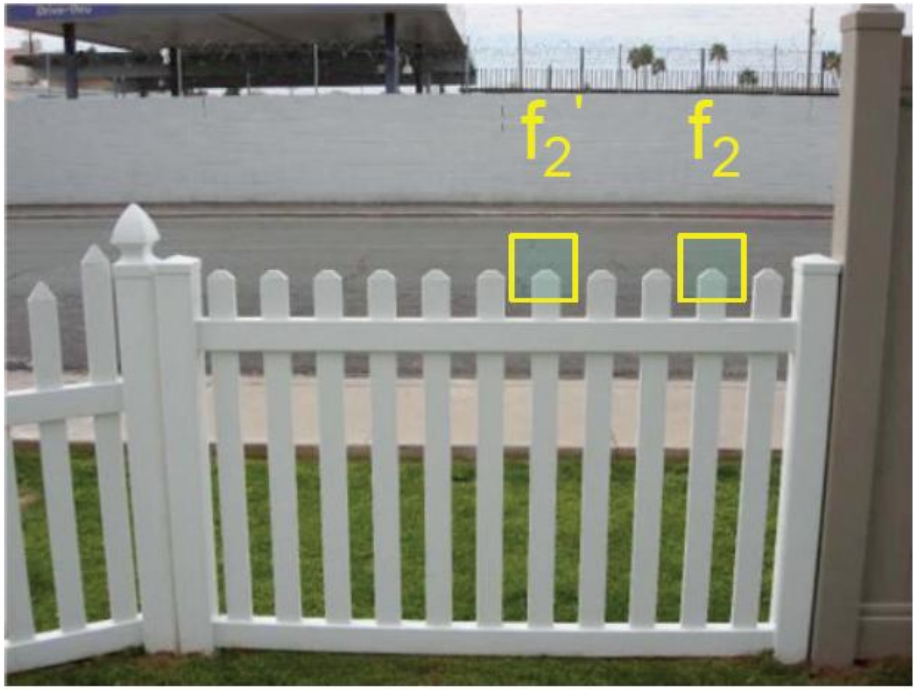
This is how big aerial photographs are made

Source: Alyosha Efros


Map
Satellite
Hybrid
200 ft
100 m
©2007 Google - Terms of Use

Problem: Ambiguous putative matches



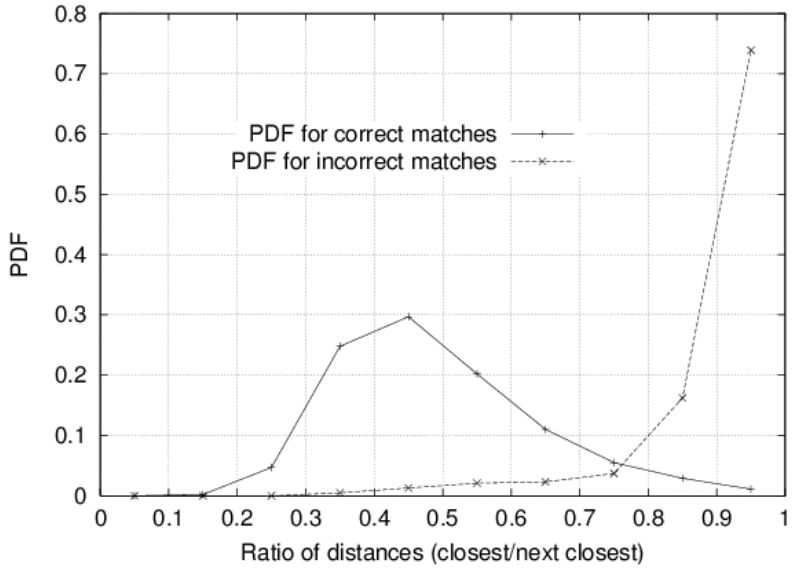


Source: Y. Furukawa

Rejection of unreliable matches

- How can we tell which putative matches are more reliable?
- Heuristic: compare distance of **nearest** neighbor to that of **second** nearest neighbor

  Ratio of closest distance to second-closest distance will be *high* for features that are *not* distinctive



Threshold of 0.8 provides good separation

David G. Lowe. "Distinctive image features from scale-invariant keypoints," *IJCV* 60 (2), pp. 91-110, 2004.

Fitting and Alignment: Methods

Global optimization / Search for parameters

- Least squares fit
- Robust least squares
- Other parameter search methods

Hypothesize and test

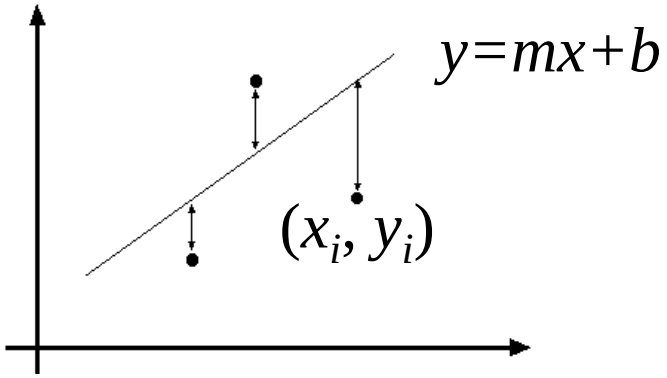
- Generalized Hough transform
- RANSAC

Least squares line fitting

Data: $(x_1, y_1), \ldots, (x_n, y_n)$

Line equation: $y_i = m x_i + b$

Find (m, b) to minimize



$$E = \sum_{i=1}^{n} (y_i - mx_i - b)^2$$

$$E = \sum_{i=1}^{n} \left(\begin{bmatrix} x_i & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - y_i \right)^2 = \left\| \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \right\|^2 = \|\mathbf{Ap} - \mathbf{y}\|^2$$

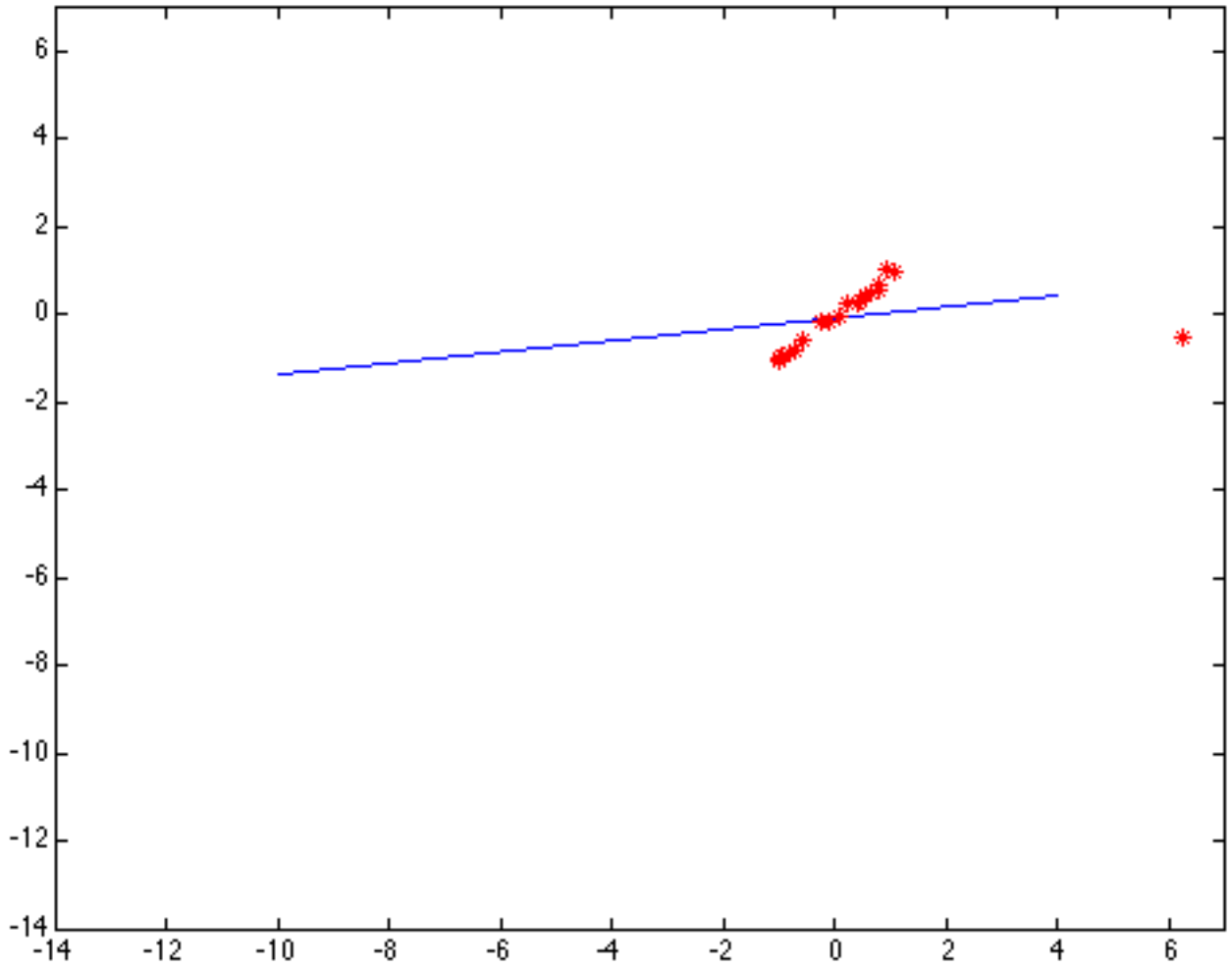
$$= \mathbf{y}^T\mathbf{y} - 2(\mathbf{Ap})^T\mathbf{y} + (\mathbf{Ap})^T(\mathbf{Ap})$$

$$\frac{dE}{dp} = 2\mathbf{A}^T\mathbf{Ap} - 2\mathbf{A}^T\mathbf{y} = 0$$

$$\mathbf{A}^T\mathbf{Ap} = \mathbf{A}^T\mathbf{y} \Rightarrow \mathbf{p} = \left(\mathbf{A}^T\mathbf{A}\right)^{-1}\mathbf{A}^T\mathbf{y}$$

Least squares: Robustness to noise

Least squares fit with an outlier:



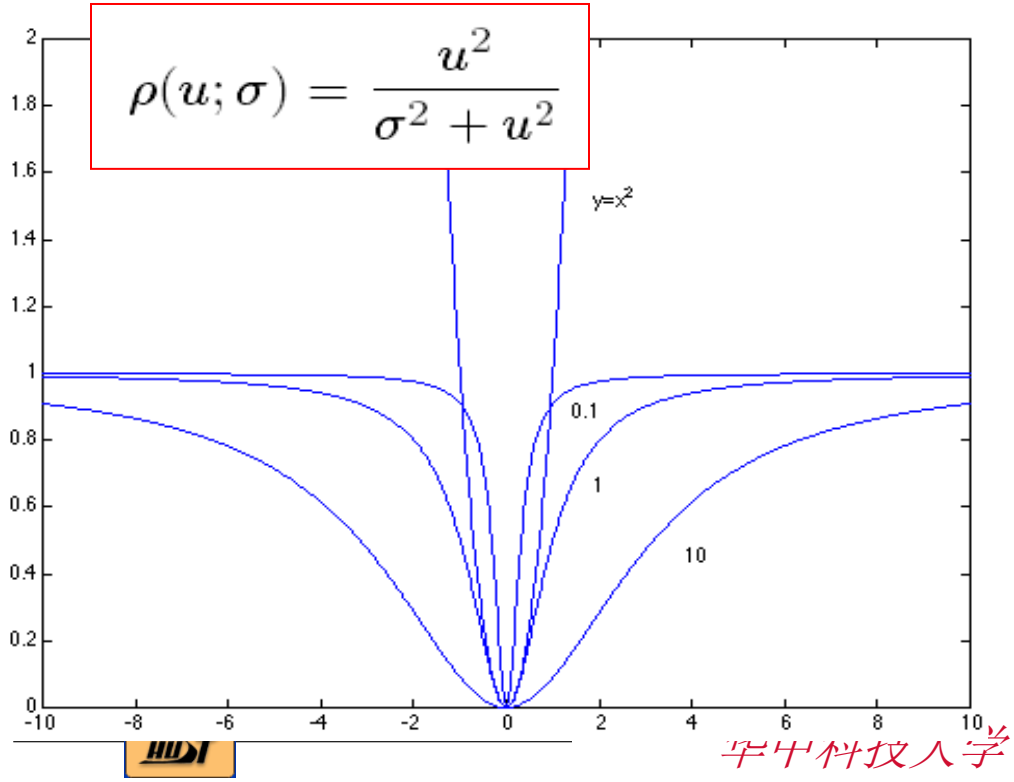
Problem: squared error heavily penalizes outliers

Robust least squares (to deal with outliers)

General approach:

minimize $\sum_i \rho\left(u_i\left(x_i, \theta\right); \sigma\right)$ $\qquad u^2 = \sum_{i=1}^{n} (y_i - mx_i - b)^2$

$u_i(x_i, \theta)$ – residual of ith point w.r.t. model parameters θ
ρ – robust function with scale parameter σ

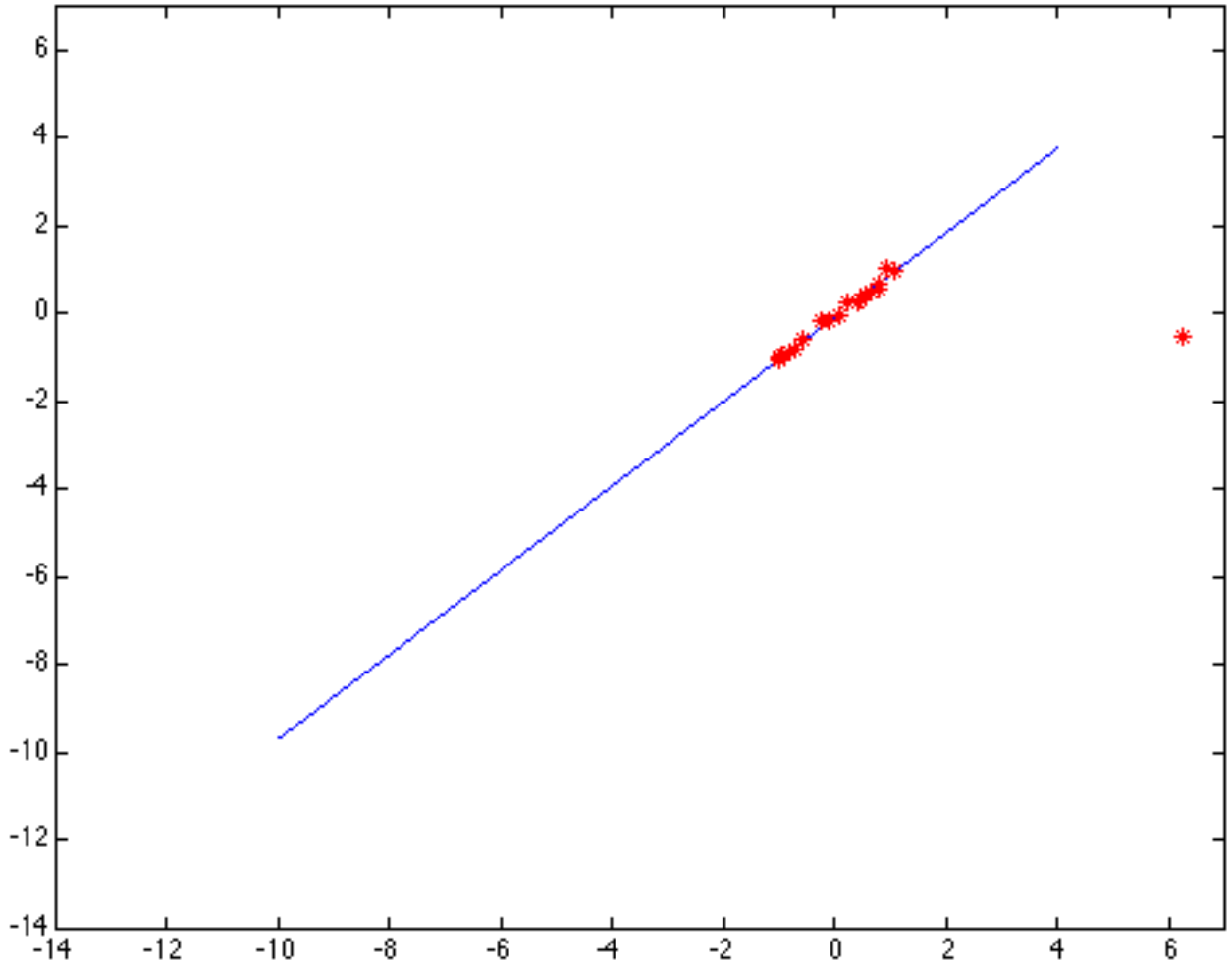


The robust function ρ

- Favors a configuration with small residuals
- Constant penalty for large residuals

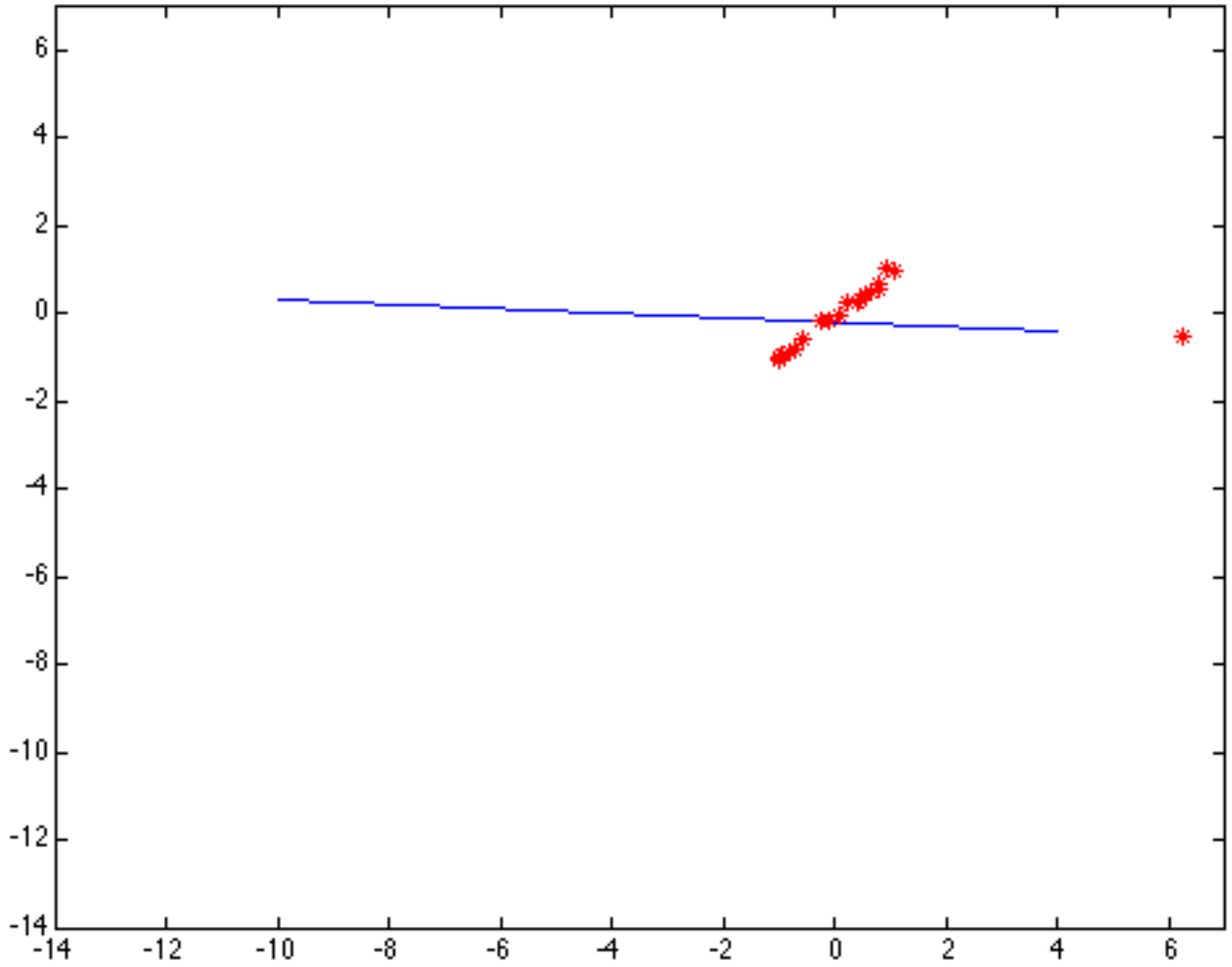
Slide from S. Savarese

Choosing the scale: Just right



The effect of the outlier is minimized

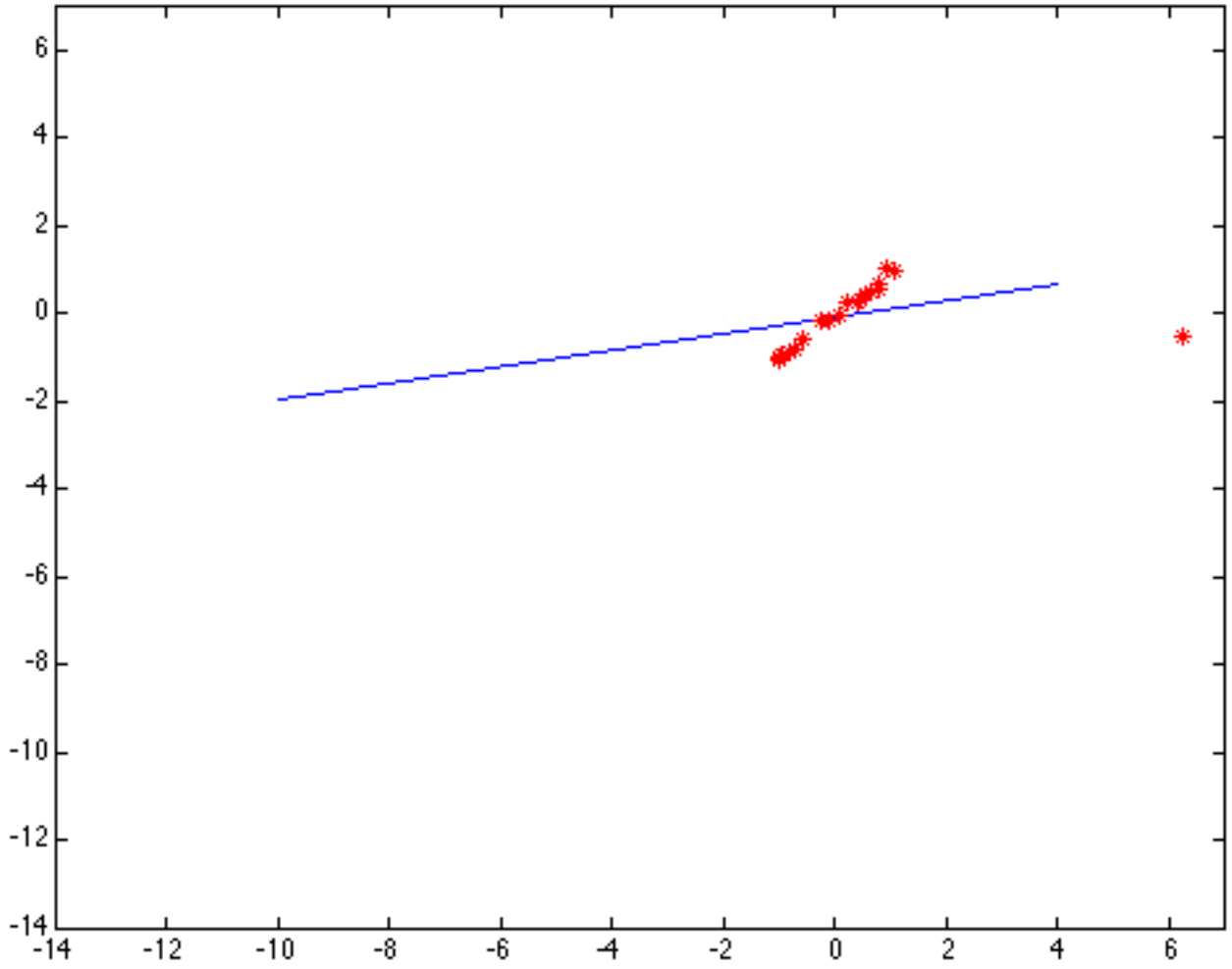
Choosing the scale: Too small



The error value is almost the same for every point and the fit is very poor

华中科技大学

Choosing the scale: Too large



Behaves much the same as least squares

Robust estimation: Details

- Robust fitting is a nonlinear optimization problem that must be solved iteratively
- Least squares solution can be used for initialization
- Scale of robust function should be chosen adaptively based on median residual

Other ways to search for parameters (for when no closed form solution exists)

Line search

1. For each parameter, step through values and choose value that gives best fit
2. Repeat (1) until no parameter changes

Grid search

1. Propose several sets of parameters, evenly sampled in the joint set
2. Choose best (or top few) and sample joint parameters around the current best; repeat

Gradient descent

1. Provide initial position (e.g., random)
2. Locally search for better parameters by following gradient

RANSAC

RANdom Sample Consensus

Approach: we want to avoid the impact of outliers, so let's look for "inliers", and use those only.

Intuition: if an outlier is chosen to compute the current fit, then the resulting line won't have much support from rest of the points.

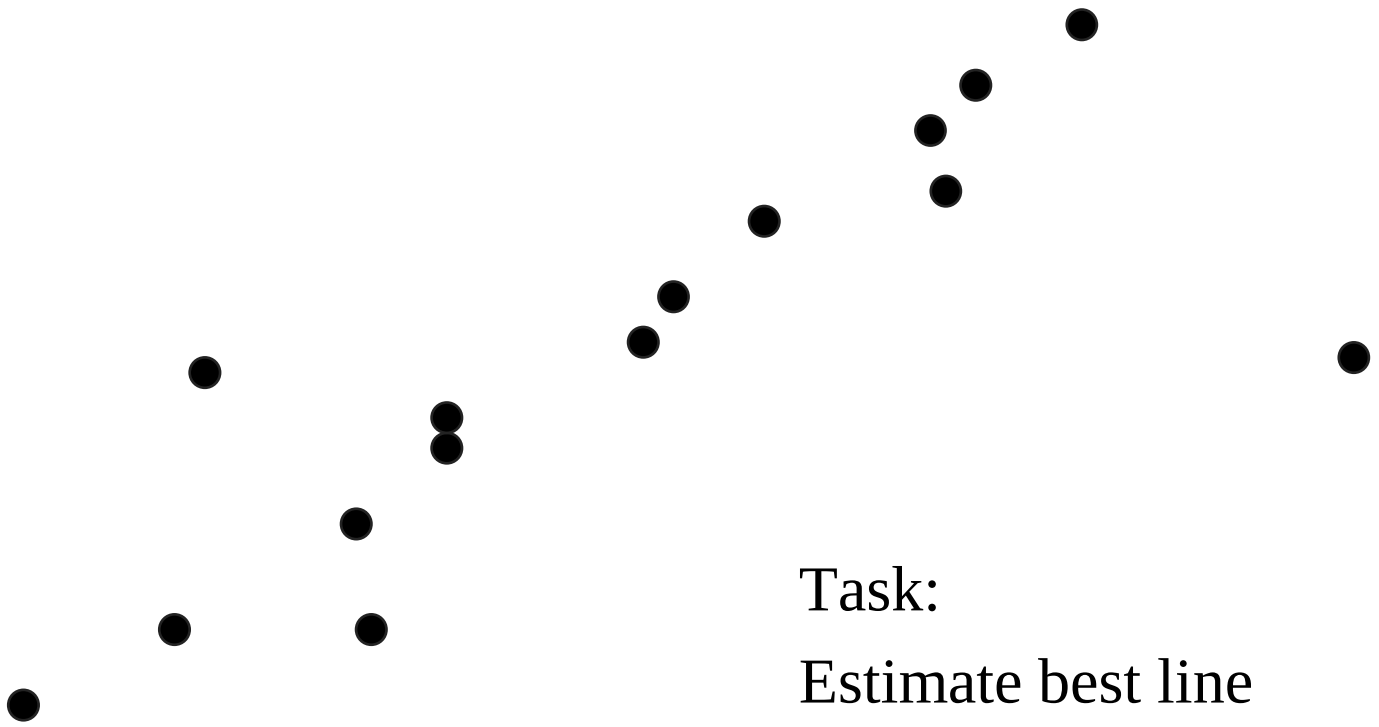
RANSAC

RANSAC loop:

1. Randomly select a *seed group* of points on which to base transformation estimate (e.g., a group of matches)
2. Compute transformation from seed group
3. Find *inliers* to this transformation
4. If the number of inliers is sufficiently large, re-compute least-squares estimate of transformation on all of the inliers

Keep the transformation with the largest number of inliers

RANSAC Line Fitting Example

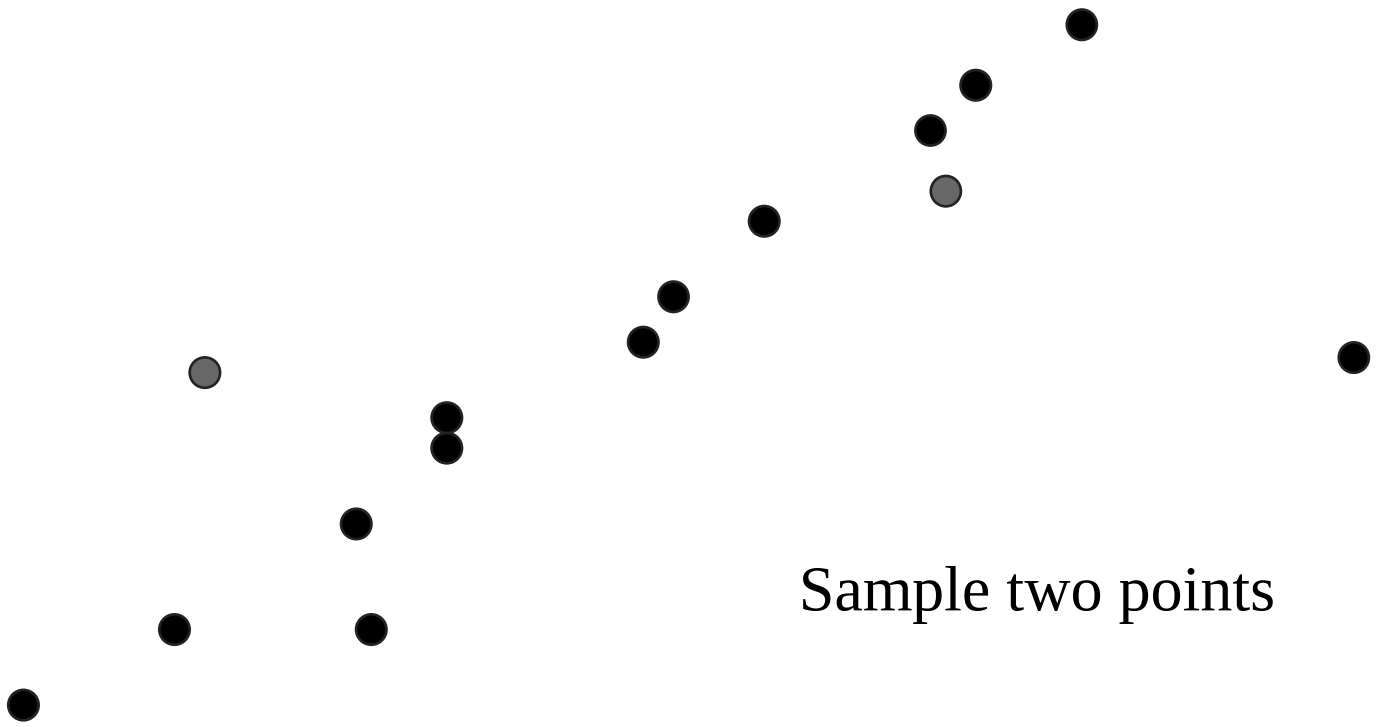


Task:

Estimate best line

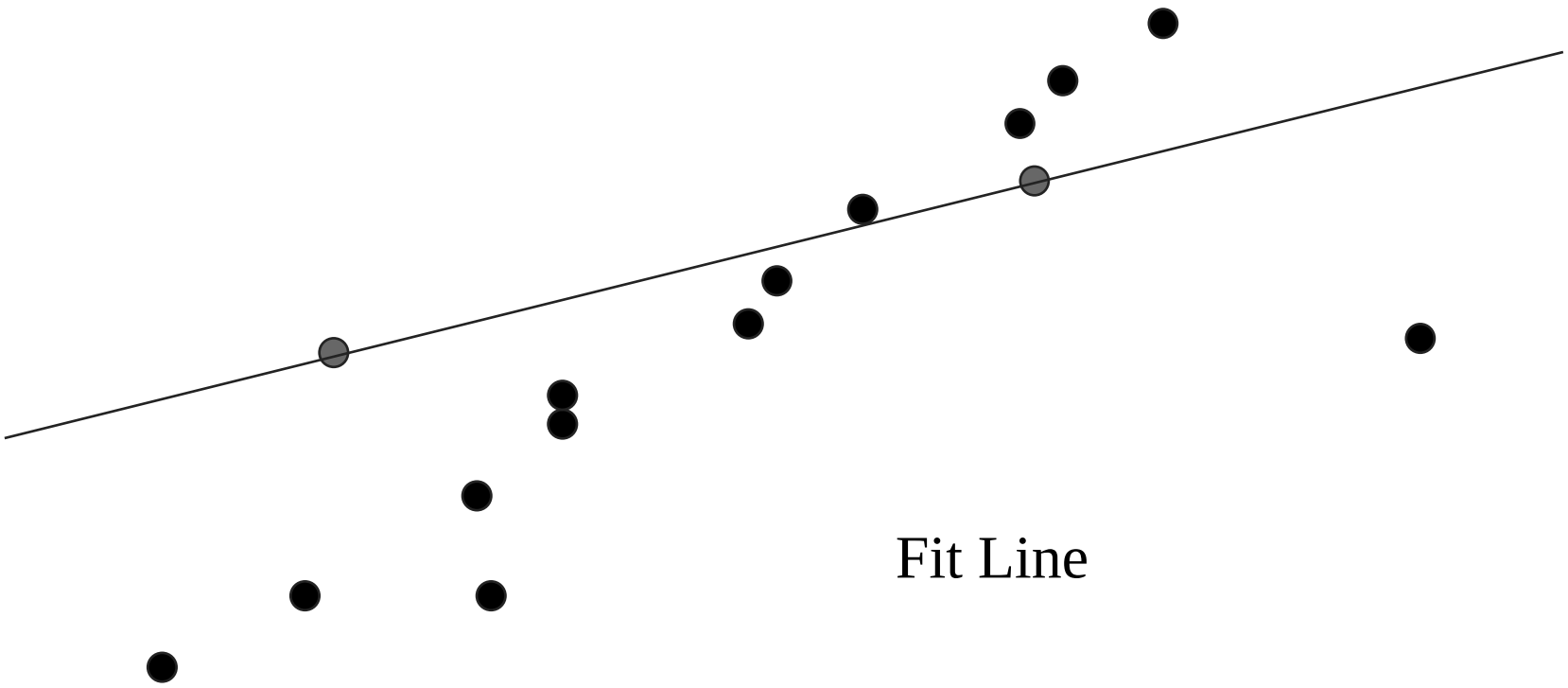
Slide credit: Jinxiang Chai, CMU

RANSAC Line Fitting Example

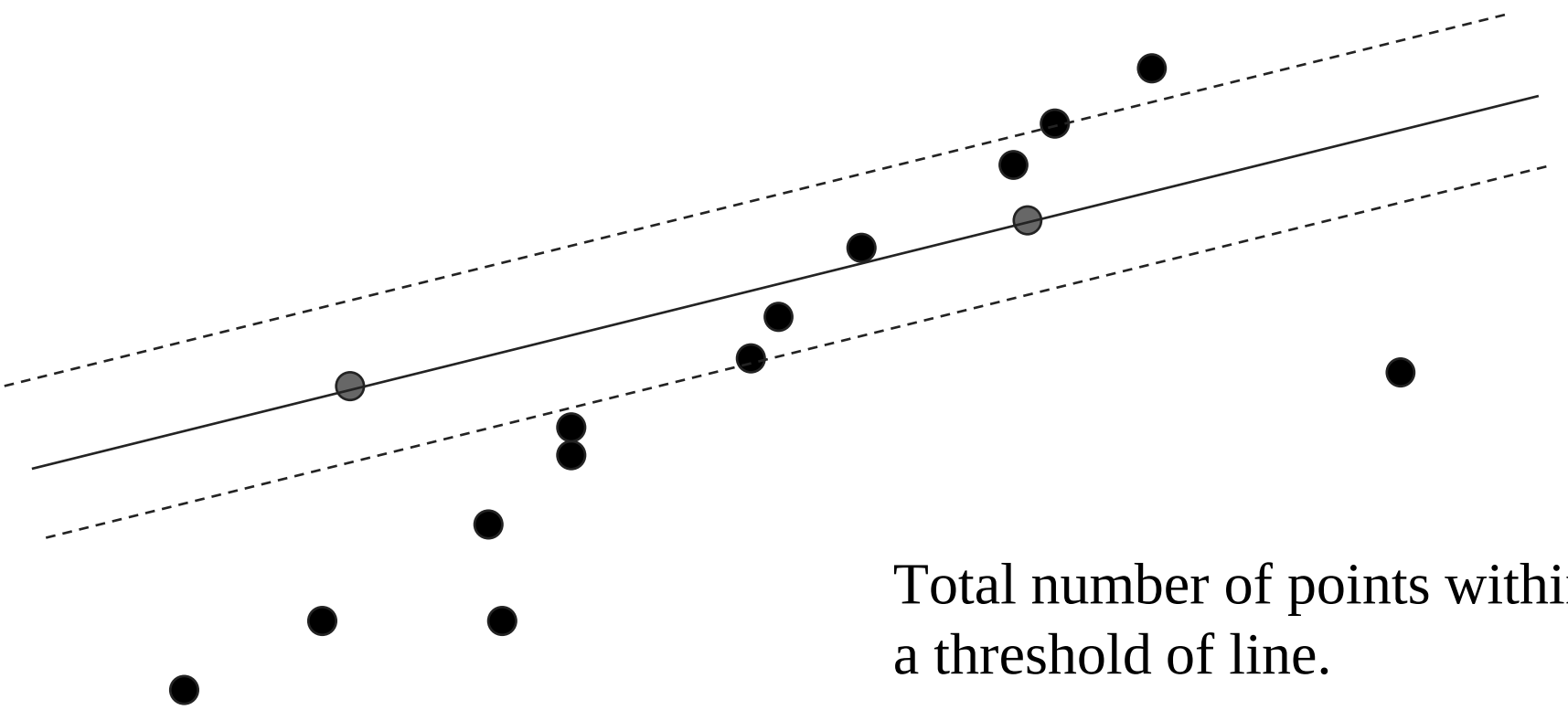


Sample two points

RANSAC Line Fitting Example

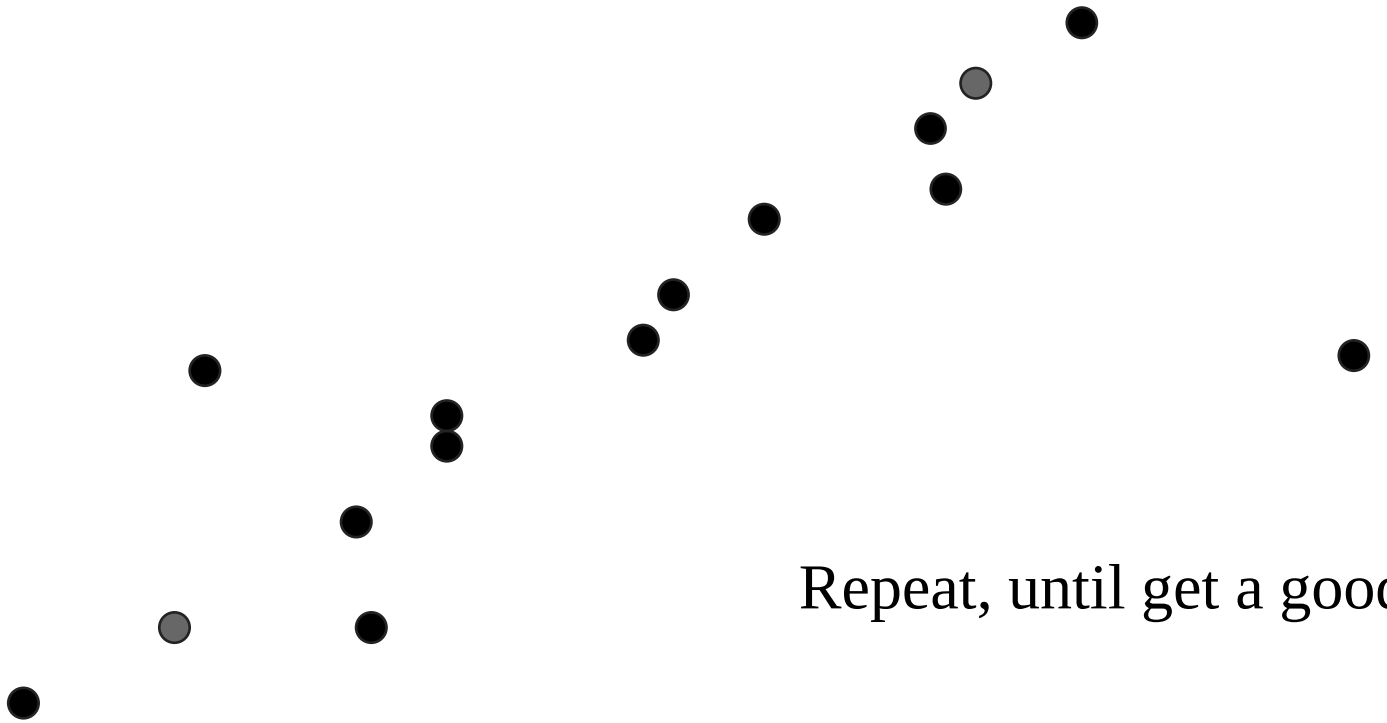


RANSAC Line Fitting Example



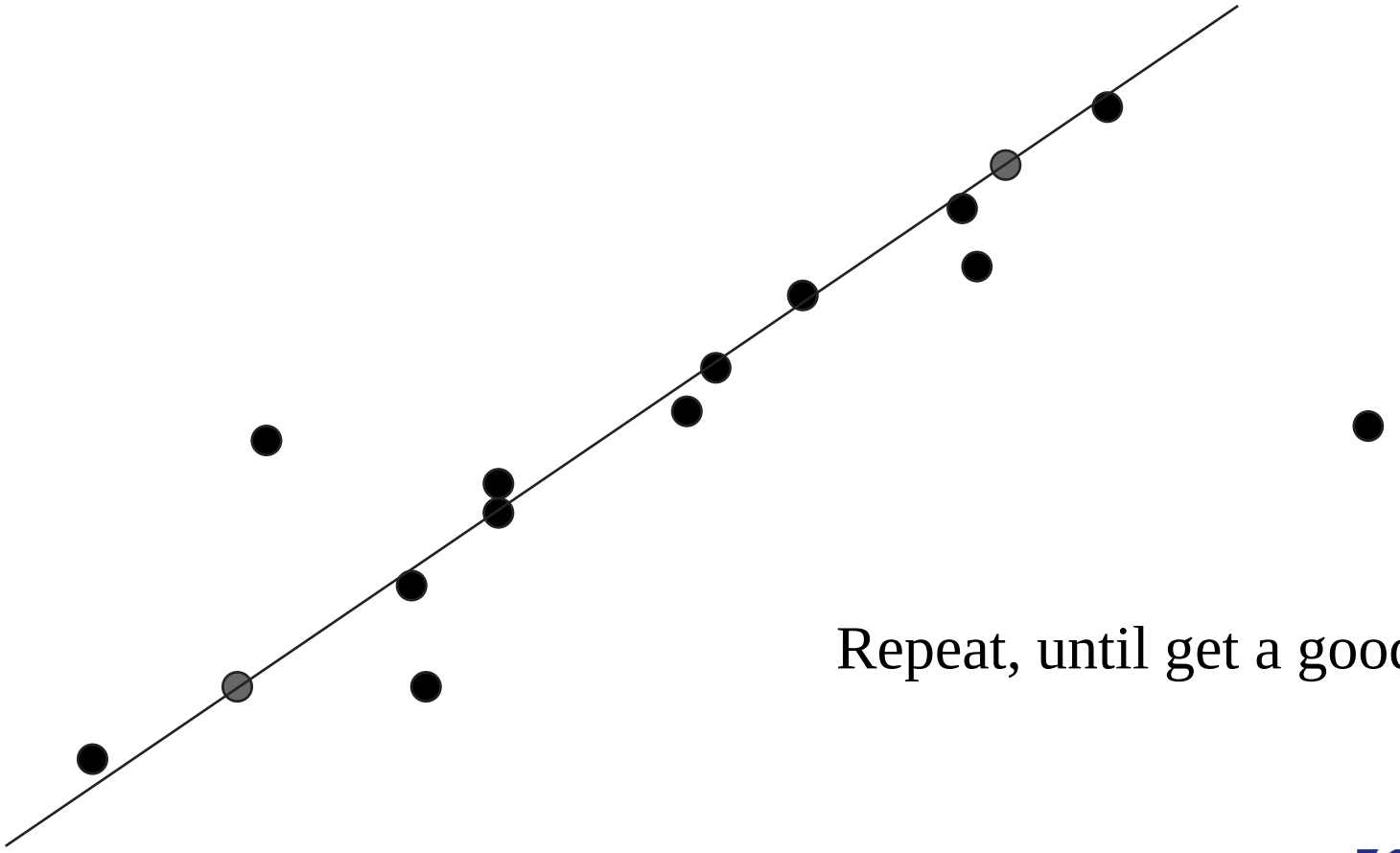
Total number of points within a threshold of line.

RANSAC Line Fitting Example



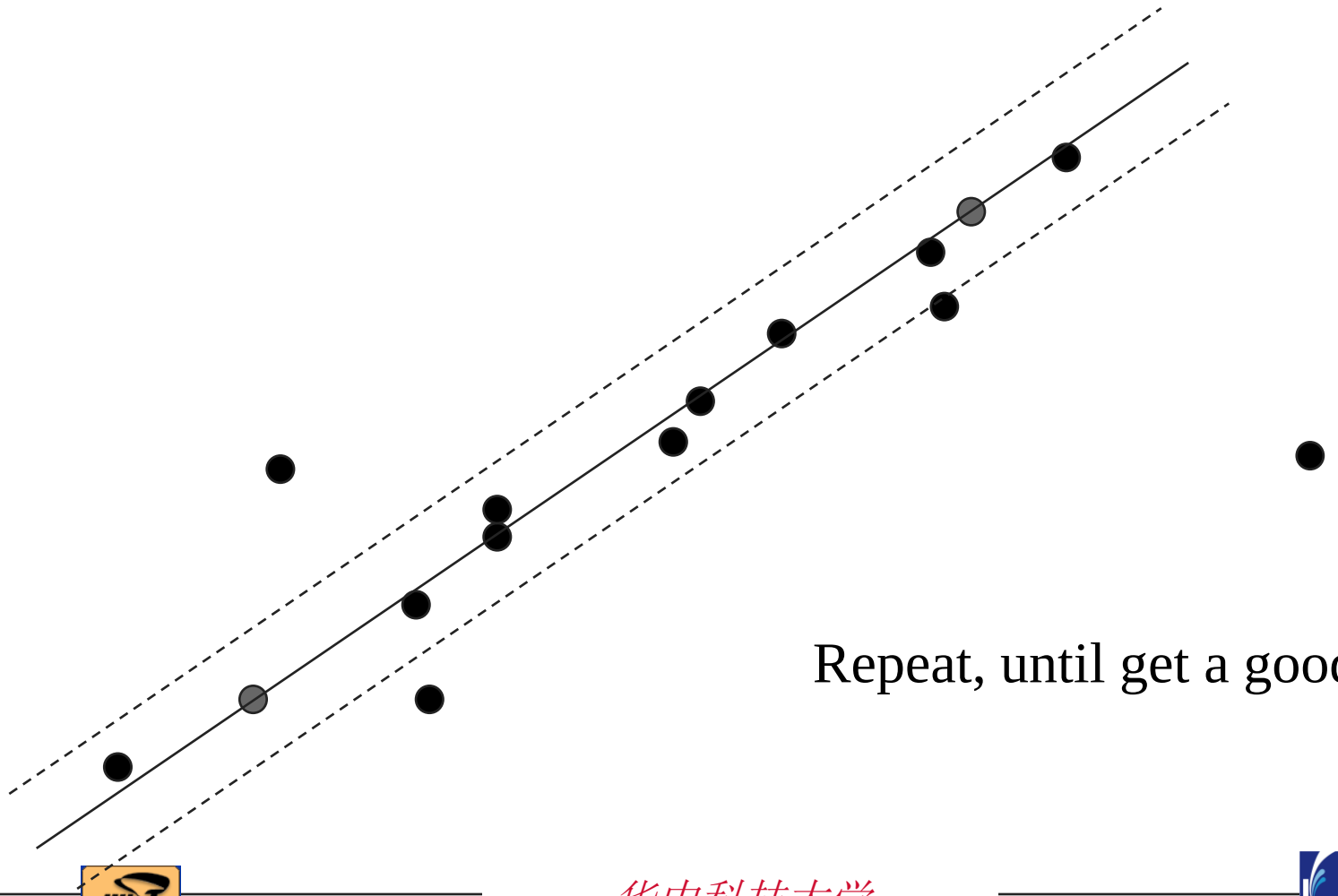
Repeat, until get a good result

RANSAC Line Fitting Example



Repeat, until get a good result

RANSAC Line Fitting Example



Repeat, until get a good result

How to choose parameters?

Number of samples N

Choose N so that, with probability p, at least one random sample is free from outliers (e.g. p=0.99) (outlier ratio: e)

Number of sampled points s

Minimum number needed to fit the model

Distance threshold δ

Choose δ so that a good point with noise is likely (e.g., prob=0.95) within threshold

Zero-mean Gaussian noise with std. dev. σ: $t^2=3.84\sigma^2$

$$N = \log(1-p)/\log\left(1-(1-e)^s\right)$$

$$\left(1-(1-e)^s\right)^N = 1-p$$

	proportion of outliers e						
s	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

For p = 0.99

华中科技大学

modified from M. Pollefeys

Adaptively determining the number of samples

- Outlier ratio *e* is often unknown a priori, so pick worst case, e.g. 50%, and adapt if more inliers are found, e.g. 80% would yield *e*=0.2
- Adaptive procedure:

  $N=\infty$, *sample_count* =0

  While *N* >*sample_count*

  Choose a sample and count the number of inliers

  If inlier ratio is highest of any found so far, set
  e = 1 – (number of inliers)/(total number of points)

  Recompute *N* from *e:*

$$N = \log(1-p)/\log(1-(1-e)^s)$$

  Increment the *sample_count* by 1

Source: M. Pollefeys

RANSAC conclusions

Good

Robust to outliers

Applicable for larger number of model parameters than Hough transform

Optimization parameters are easier to choose than Hough transform

Bad

Computational time grows quickly with fraction of outliers and number of parameters

Not good for getting multiple fits

Common applications

Computing a homography (e.g., image stitching)

Estimating fundamental matrix (relating two views)

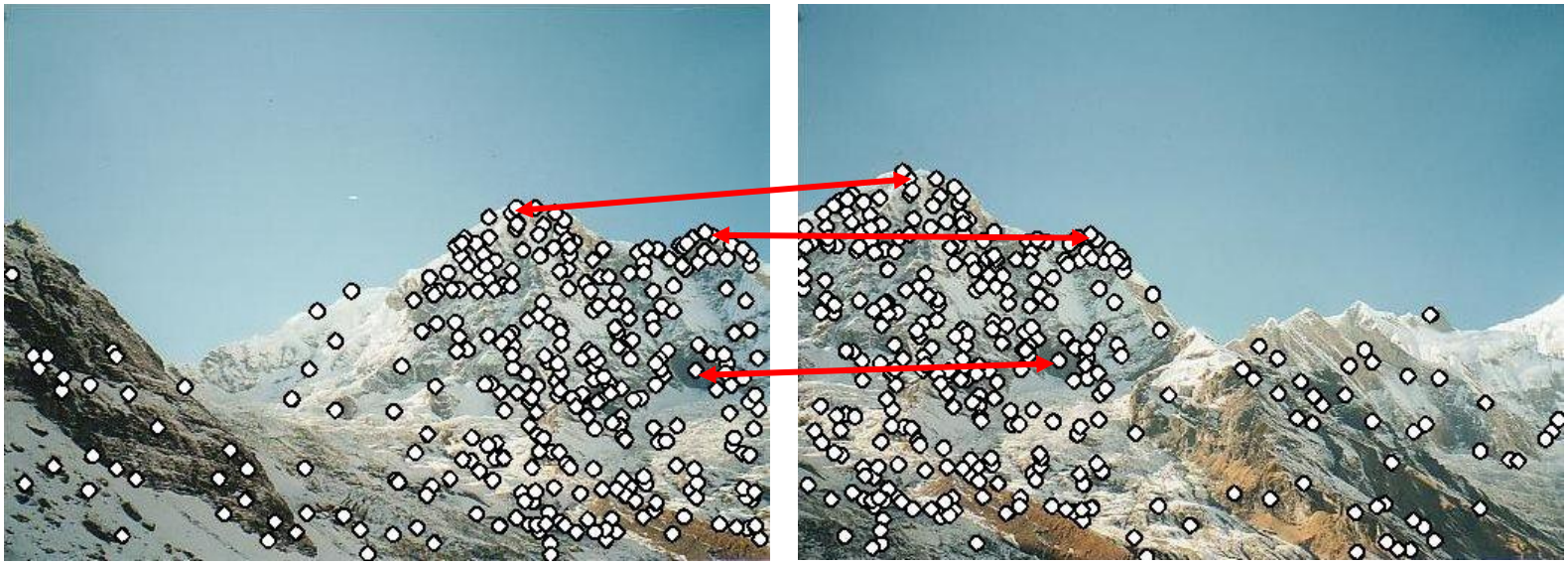
Feature-based alignment outline



- Extract features
- Compute *putative matches*

Source: L. Lazebnik

Feature-based alignment outline

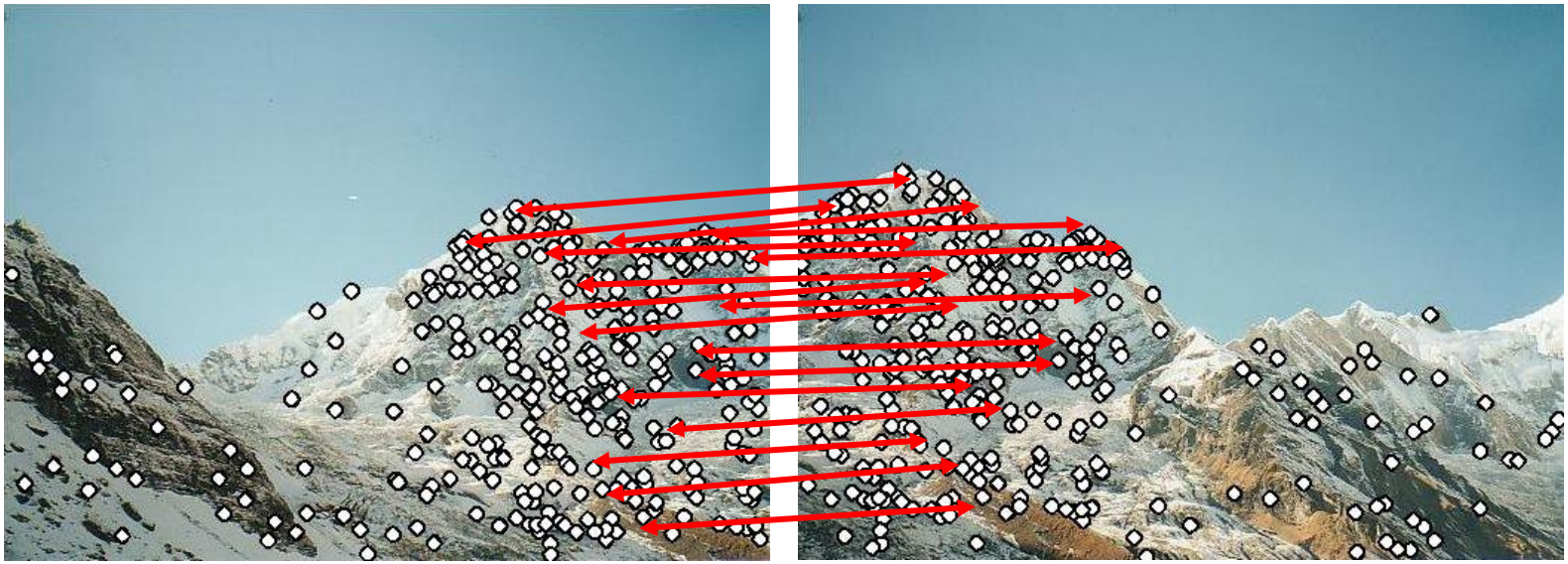


- Extract features
- Compute *putative matches*
- Loop:

  Hypothesize transformation *T* (small group of putative matches that are related by *T*)

Source: L. Lazebnik

Feature-based alignment outline



- Extract features
- Compute *putative matches*
- Loop:

  Hypothesize transformation *T* (small group of putative matches that are related by *T*)

  Verify transformation (search for other matches consistent with *T*)

Source: L. Lazebnik

Feature-based alignment outline



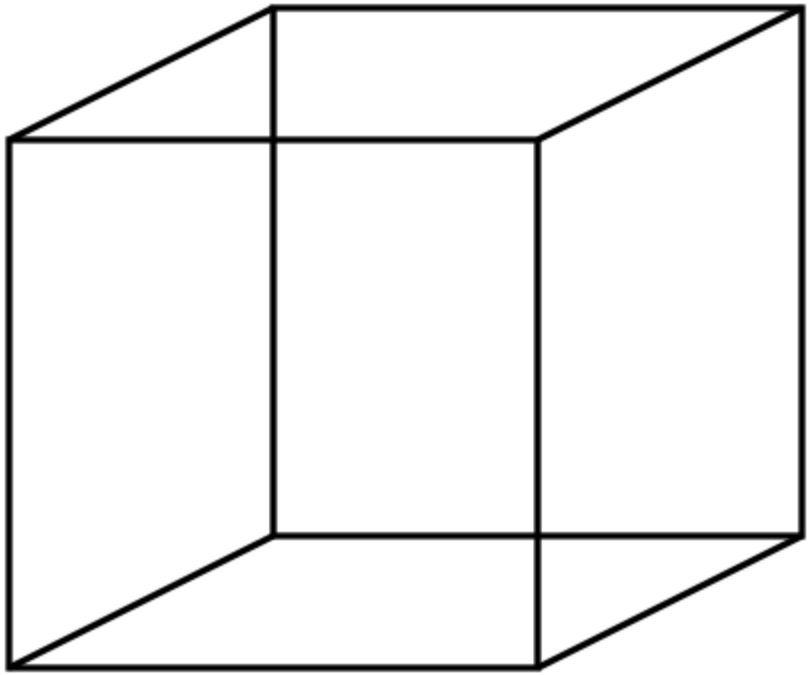
- Extract features
- Compute *putative matches*
- Loop:

  Hypothesize transformation *T* (small group of putative matches that are related by *T*)

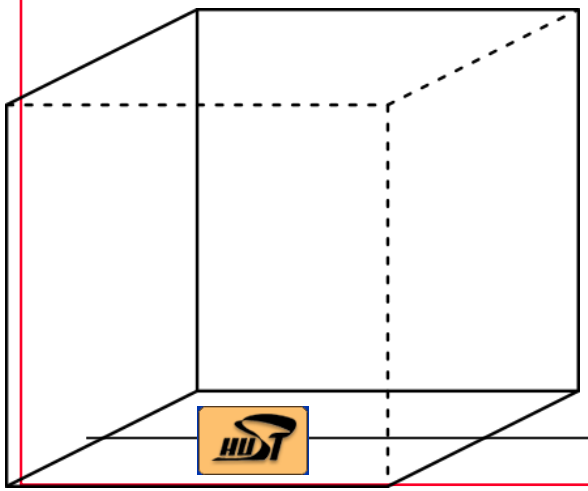
  Verify transformation (search for other matches consistent with *T*)

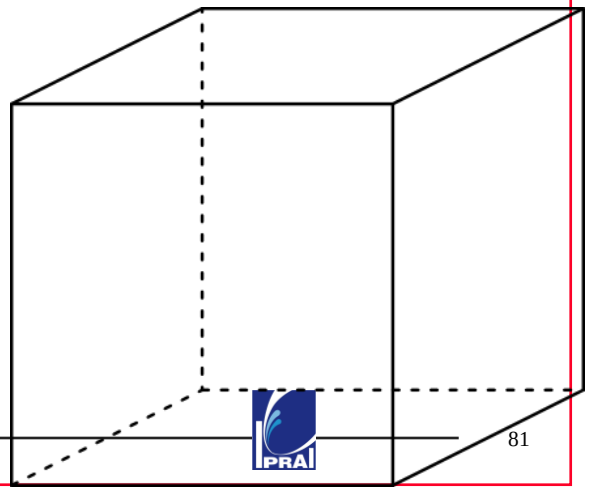
Source: L. Lazebnik

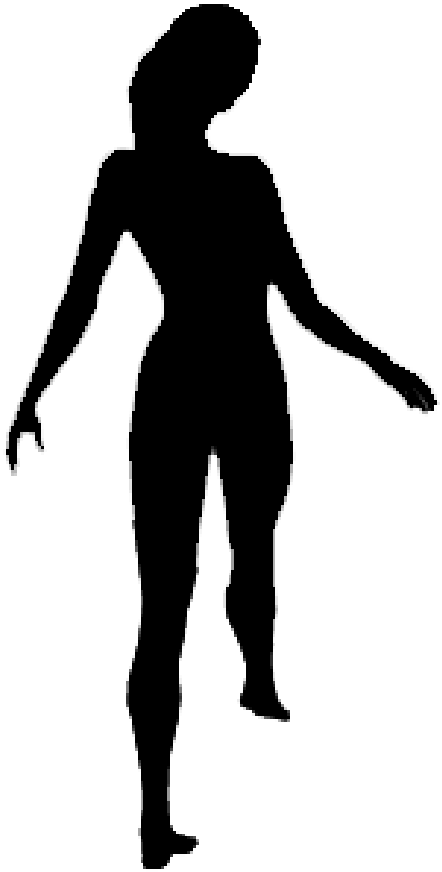
Multi-stable Perception



Necker Cube

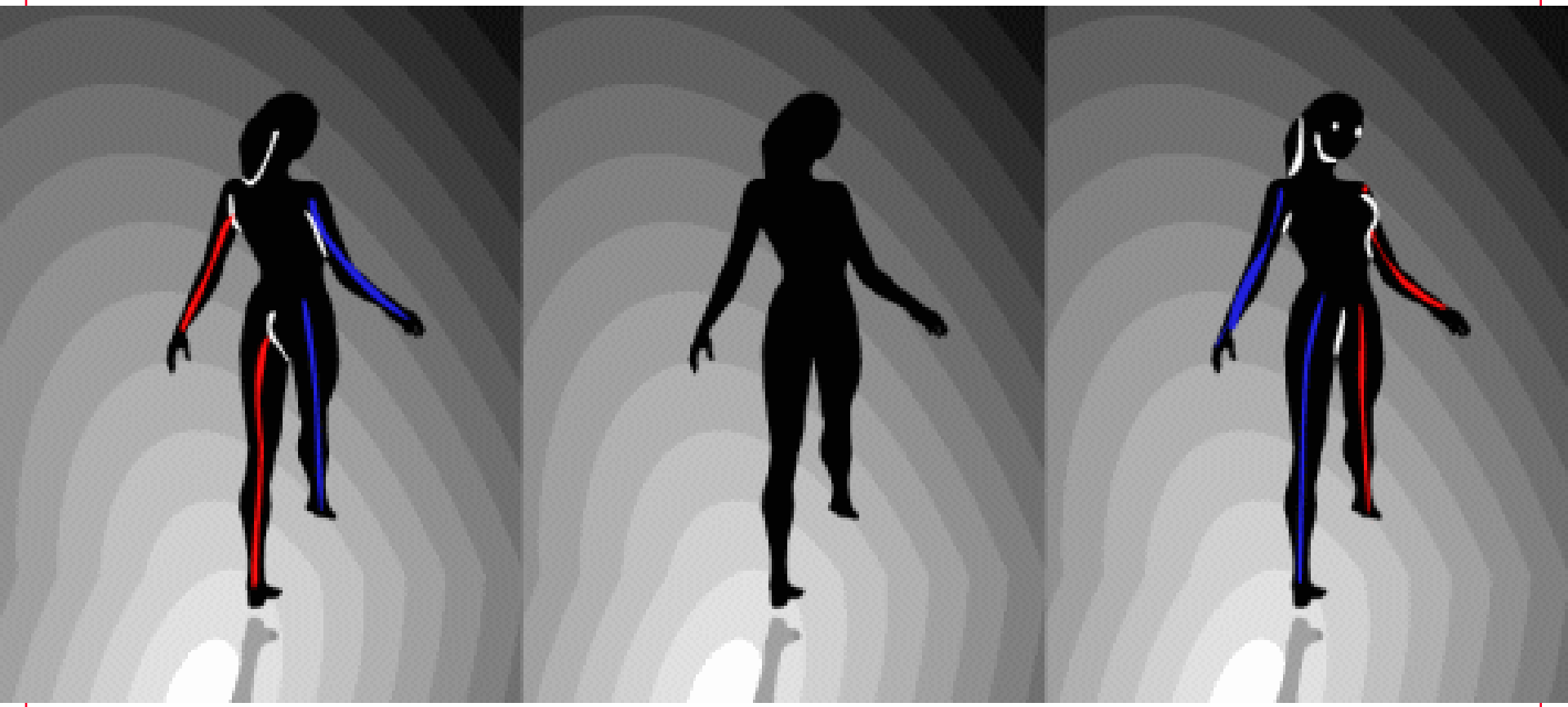






Spinning dancer illusion, Nobuyuki

华中科技大学



Hough Transform: Outline

1. Create a grid of parameter values

2. Each point votes for a set of parameters, incrementing those values in grid

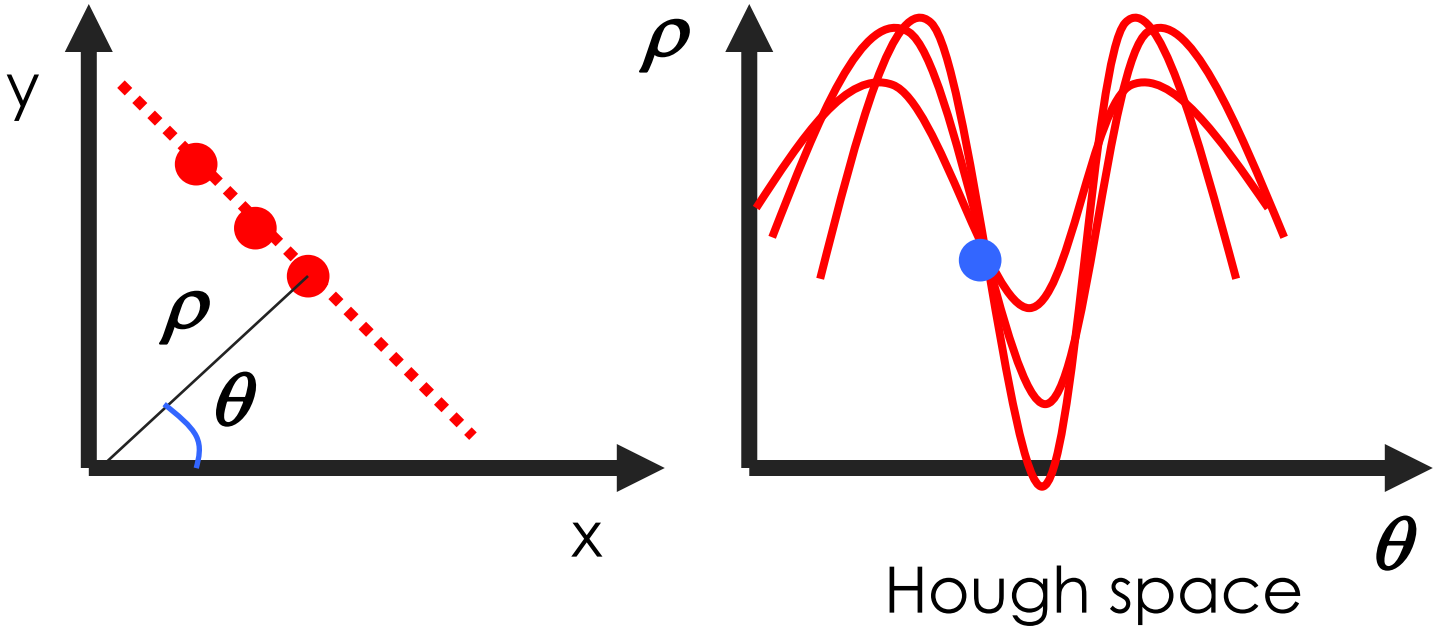
3. Find maximum or local maxima in grid

Hough transform

P.V.C. Hough, *Machine Analysis of Bubble Chamber Pictures,* Proc. Int. Conf. High Energy Accelerators and Instrumentation, 1959

Issue : parameter space [m,b] is unbounded…

Use a polar representation for the parameter space



$$x\cos\theta + y\sin\theta = \rho$$

Slide from S. Savarese

Finding lines using Hough transform

Using m,b parameterization

Using r, theta parameterization

- Using oriented gradients

Practical considerations

- Bin size
- Smoothing
- Finding multiple lines
- Finding line segments

Hough transform conclusions

Good

Robust to outliers: each point votes separately

Fairly efficient (much faster than trying all sets of parameters)

Provides multiple good fits

Bad

Some sensitivity to noise

Bin size trades off between noise tolerance, precision, and speed/memory

- Can be hard to find sweet spot

Not suitable for more than a few parameters

- grid size grows exponentially

Common applications

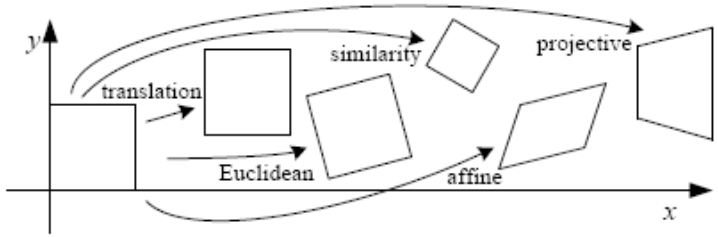
Line fitting (also circles, ellipses, etc.)

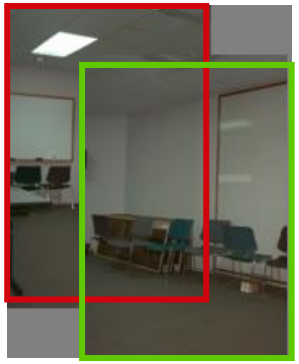
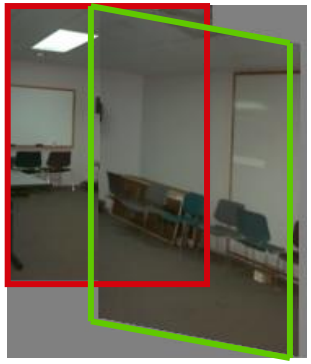
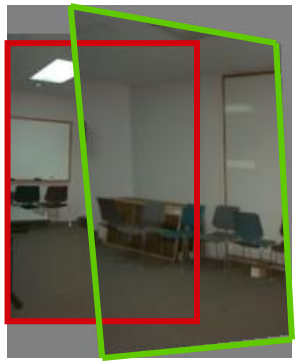
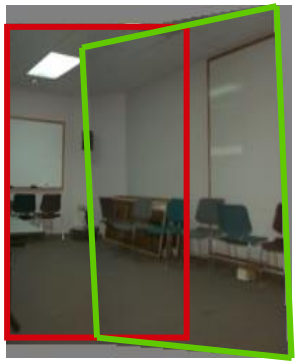
Object instance recognition (parameters are affine transform)

Object category recognition (parameters are position/scale)

Towards large-scale mosaics…

Motion models



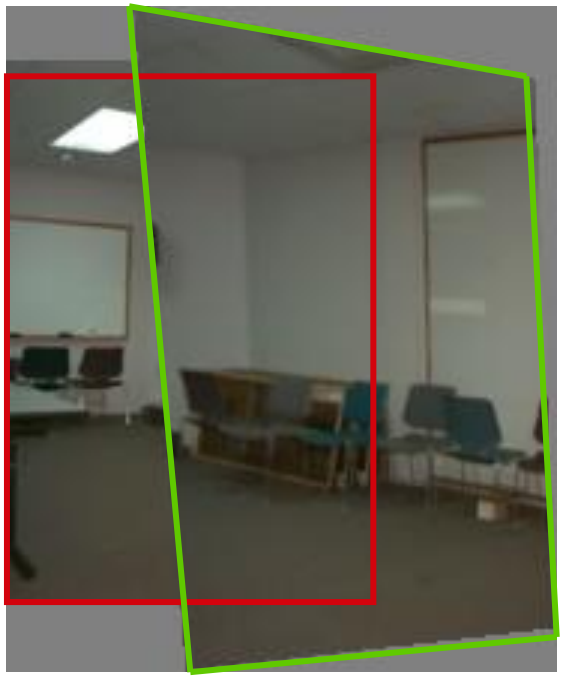
Translation	Affine	Perspective	3D rotation
			
2 unknowns	6 unknowns	8 unknowns	3 unknowns

Plane perspective mosaics

8-parameter homographies

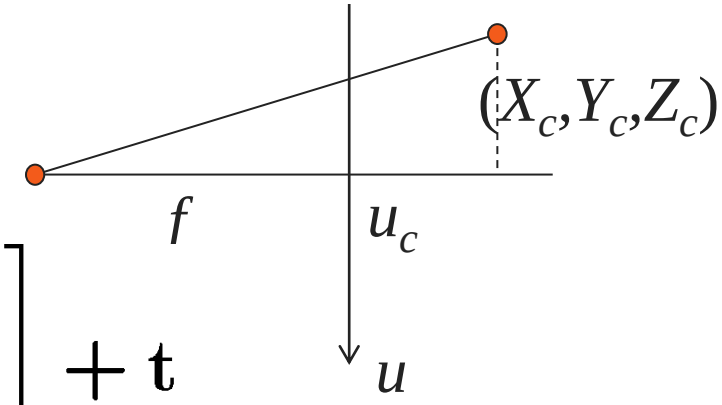
Limitations:

- local minima
- slow convergence
- difficult to control interactively



Szelis

3D → 2D Perspective Projection



$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} \mathbf{R} \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + \mathbf{t}$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \begin{bmatrix} U \\ V \\ W \end{bmatrix} = \begin{bmatrix} f & 0 & u_c \\ 0 & f & v_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix}$$

Szelis

3D Rotation Model

Projection equations

1. Project from image to 3D ray

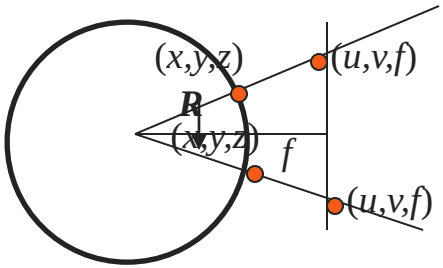
   $(x_0,y_0,z_0) = (u_0-u_c,v_0-v_c,f)$

2. Rotate the ray by camera motion

   $(x_1,y_1,z_1) = \boldsymbol{R}_{01}\,(x_0,y_0,z_0)$

3. Project back into new (source) image

   $(u_1,v_1) = (fx_1/z_1+u_c,fy_1/z_1+v_c)$



Therefore:

$$H = K_1 R_{01} K_0^{-1}$$

Such homography has only 3,4 or 5 DOF, depending if focal length is known, same, or different.

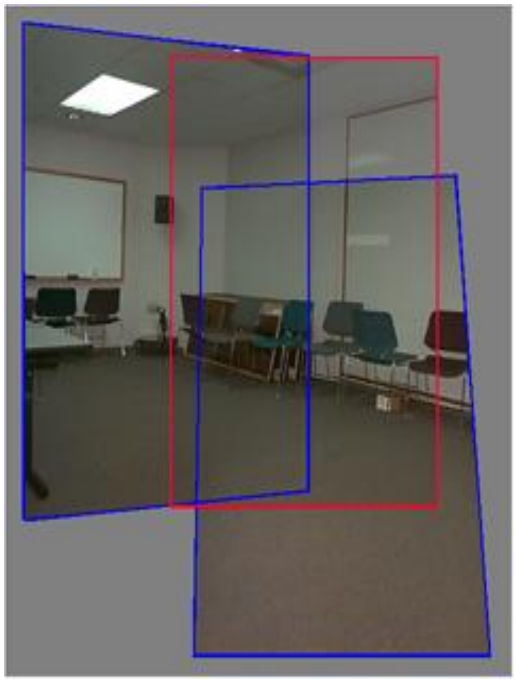
This makes image registration much better behaved

Rotational mosaics

Directly optimize rotation and focal length

Advantages:

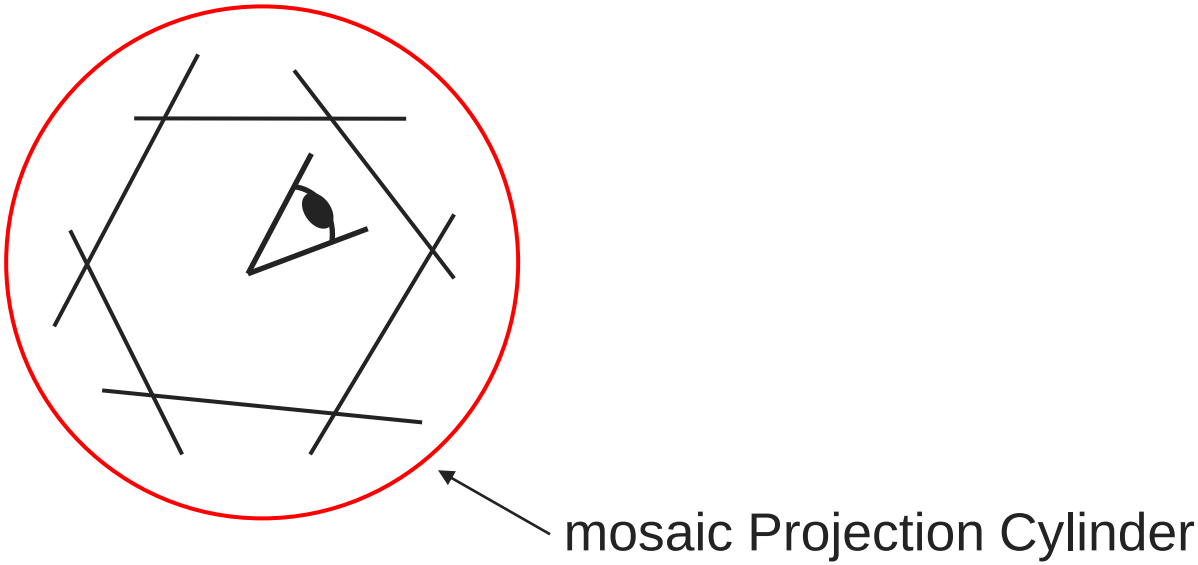
- ability to build full-view panoramas
- easier to control interactively
- more stable and accurate estimates



Szelis

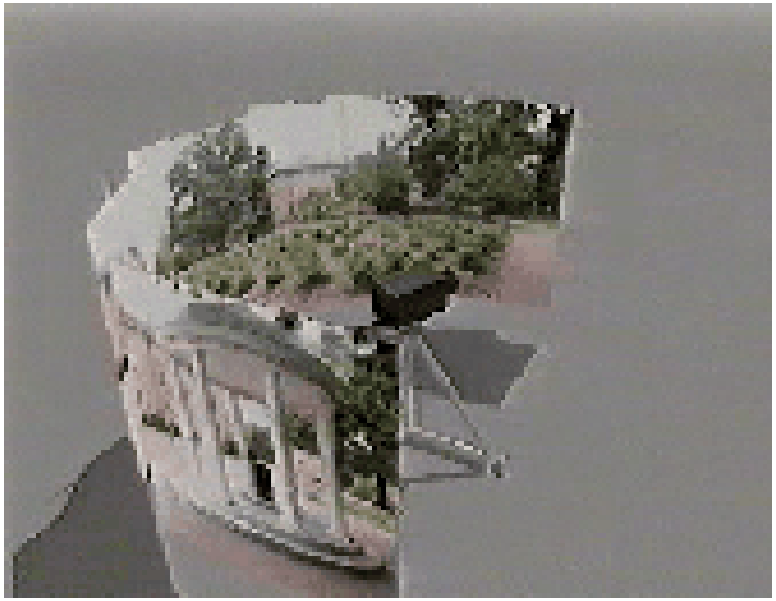
Panoramas

What if you want a 360° field of view?



Szelis

Cylindrical panoramas



Steps

Reproject each image onto a cylinder

Blend

Output the resulting mosaic

Szelis

Cylindrical Panoramas

Map image to cylindrical or spherical coordinates

need *known* focal length



Image 384x300



f = 180 (pixels)



f = 280

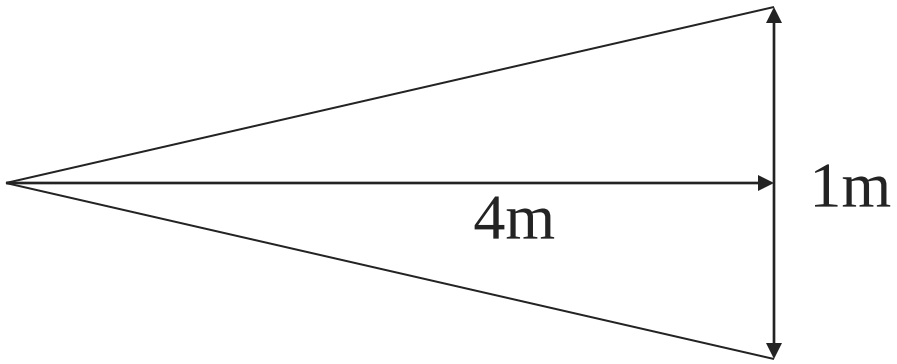


f = 380

Szelis

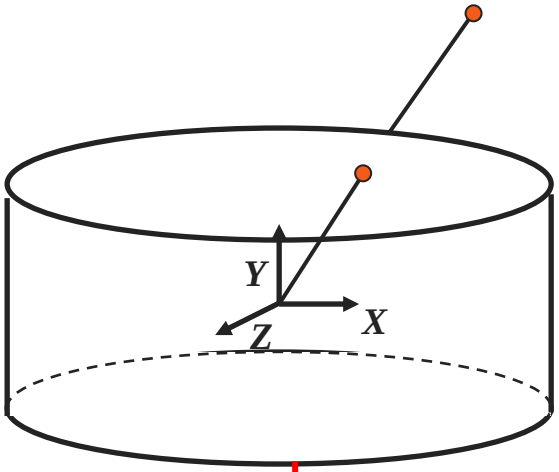
Determining the focal length

1. Initialize from homography ***H***
   (see text or [SzSh'97])
2. Use camera's EXIF tags (approx.)
3. Use a tape measure

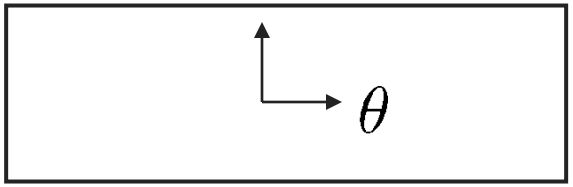


Szelis

Cylindrical projection



unit cylinder



unwrapped cylinder

Map 3D point (X,Y,Z) onto cylinder

- Convert to cylindrical coordinates

$$(sin\theta, h, cos\theta) = (\hat{x}, \hat{y}, \hat{z})$$

- Convert to cylindrical image coordinates

$$(\tilde{x}, \tilde{y}) = (s\theta, sh) + (\tilde{x}_c, \tilde{y}_c)$$

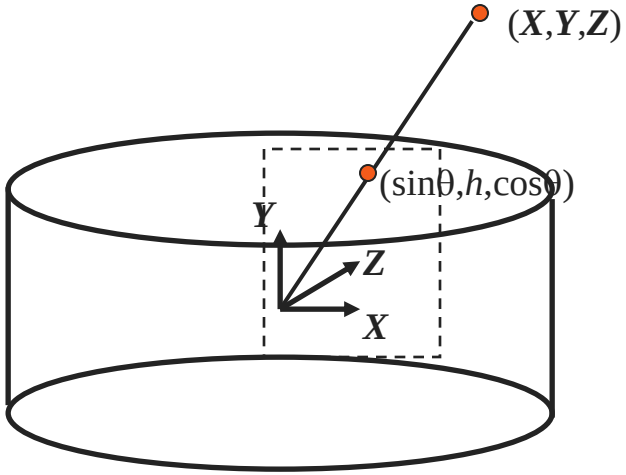
  - s defines size of the final image



cylindrical image

Cylindrical warping

Given focal length f and image center (x_c, y_c)



$$
\begin{aligned}
\theta &= (x_{cyl} - x_c)/f \\
h &= (y_{cyl} - y_c)/f \\
\hat{x} &= \sin\theta \\
\hat{y} &= h \\
\hat{z} &= \cos\theta \\
x &= f\hat{x}/\hat{z} + x_c \\
y &= f\hat{y}/\hat{z} + y_c
\end{aligned}
$$

Full-view Panorama



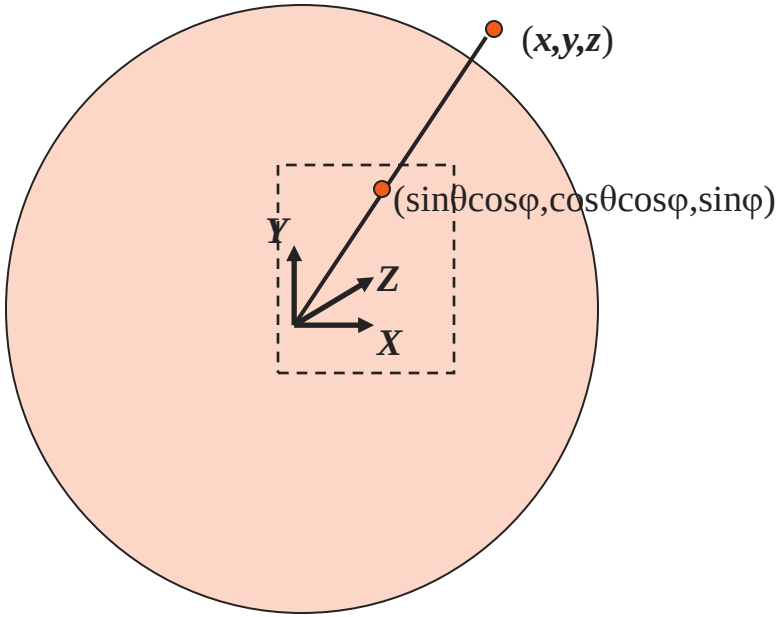





华中科技大学
Szelis

Spherical warping

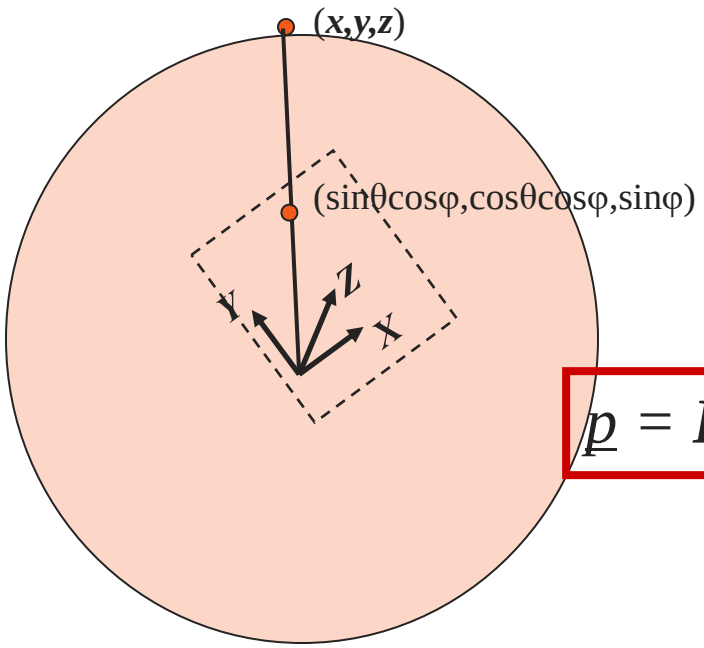
Given focal length f and image center (x_c, y_c)



$$
\begin{aligned}
\theta &= (x_{cyl} - x_c)/f \\
\varphi &= (y_{cyl} - y_c)/f \\
\hat{x} &= \sin\theta \cos\varphi \\
\hat{y} &= \sin\varphi \\
\hat{z} &= \cos\theta \cos\varphi \\
x &= f\hat{x}/\hat{z} + x_c \\
y &= f\hat{y}/\hat{z} + y_c
\end{aligned}
$$

3D rotation

Rotate image before placing on unrolled sphere

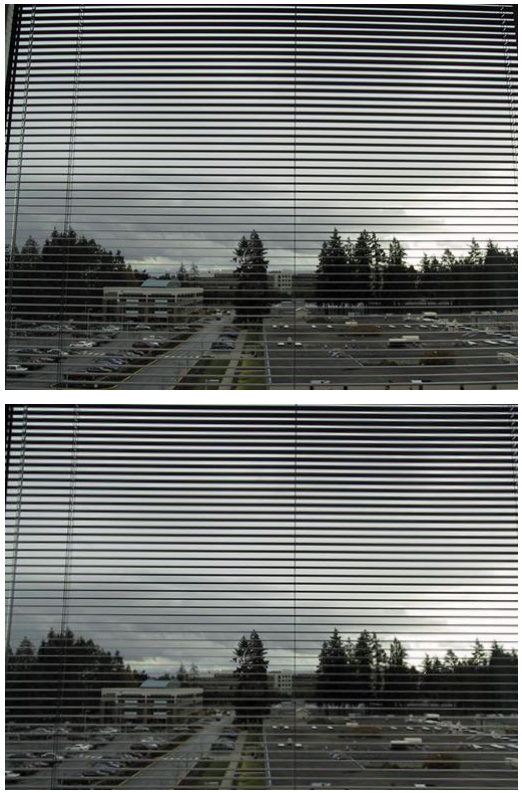


$$\underline{p} = \boldsymbol{R}\, p$$

$$\begin{aligned}
\theta &= (x_{cyl} - x_c)/f \\
\varphi &= (y_{cyl} - y_c)/f \\
\hat{x} &= \sin\theta \cos\varphi \\
\hat{y} &= \sin\varphi \\
\hat{z} &= \cos\theta \cos\varphi \\
x &= f\hat{x}/\hat{z} + x_c \\
y &= f\hat{y}/\hat{z} + y_c
\end{aligned}$$

Radial distortion

Correct for “bending” in wide field of view lenses



$$
\begin{aligned}
\hat{r}^2 &= \hat{x}^2 + \hat{y}^2 \\
\hat{x}' &= \hat{x}/(1 + \kappa_1 \hat{r}^2 + \kappa_2 \hat{r}^4) \\
\hat{y}' &= \hat{y}/(1 + \kappa_1 \hat{r}^2 + \kappa_2 \hat{r}^4) \\
x &= f\hat{x}'/\hat{z} + x_c \\
y &= f\hat{y}'/\hat{z} + y_c
\end{aligned}
$$

Szelis

Fisheye lens

Extreme “bending” in ultra-wide fields of view



$$\hat{r}^2 = \hat{x}^2 + \hat{y}^2$$

$$(\cos\theta \sin\phi, \sin\theta \sin\phi, \cos\phi) = s\,(x, y, z)$$

uations become

$$x' = s\phi\cos\theta = s\frac{x}{r}\tan^{-1}\frac{r}{z},$$

$$y' = s\phi\sin\theta = s\frac{y}{r}\tan^{-1}\frac{r}{z},$$

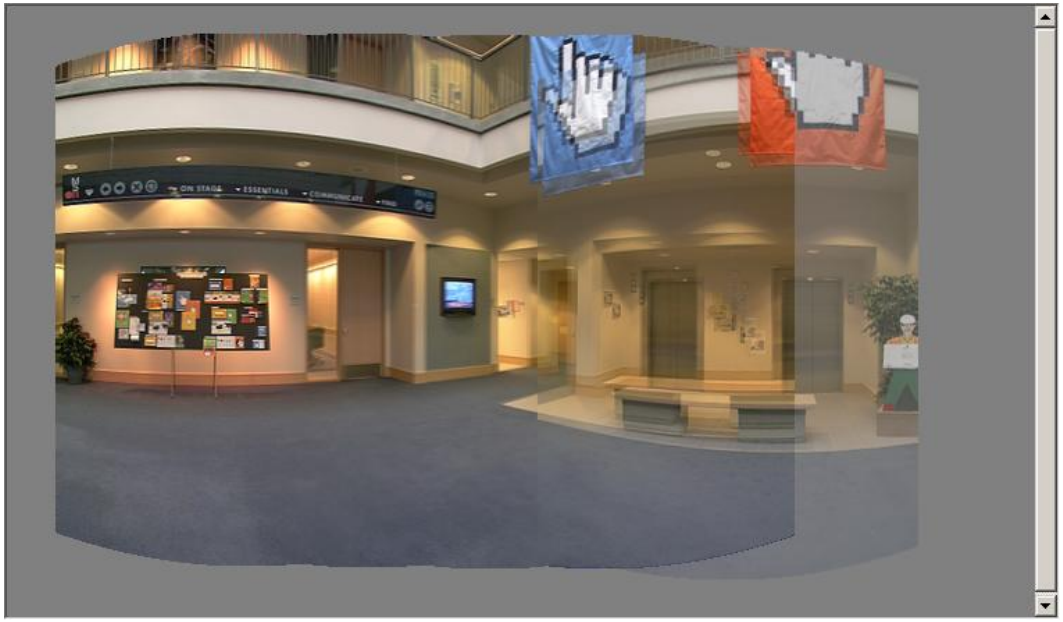
Szelis

Image Stitching

1. Align the images over each other

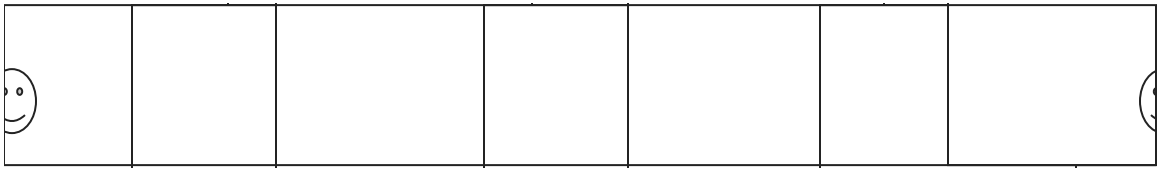
   camera pan ⇿ translation on cylinder

2. Blend the images together



Szelis

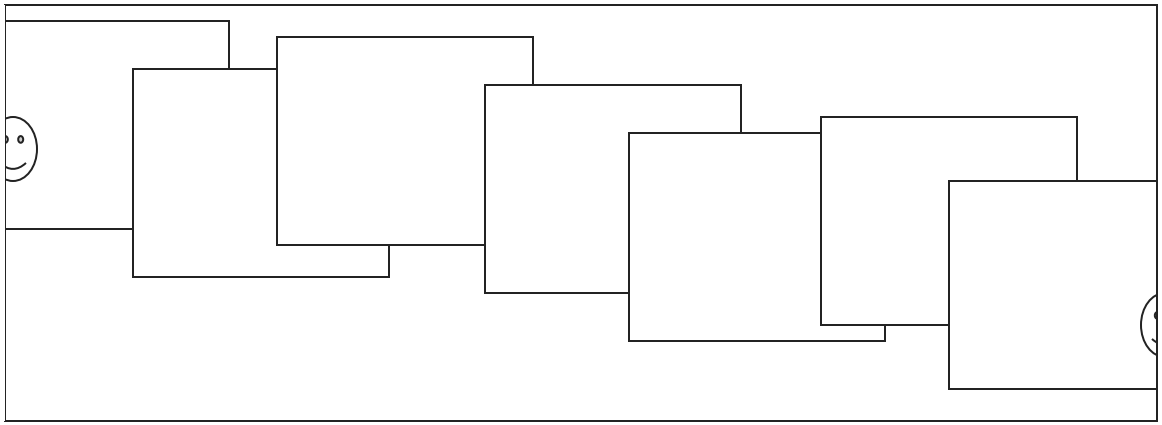
Assembling the panorama



Stitch pairs together, blend, then crop

Szelis

Problem: Drift



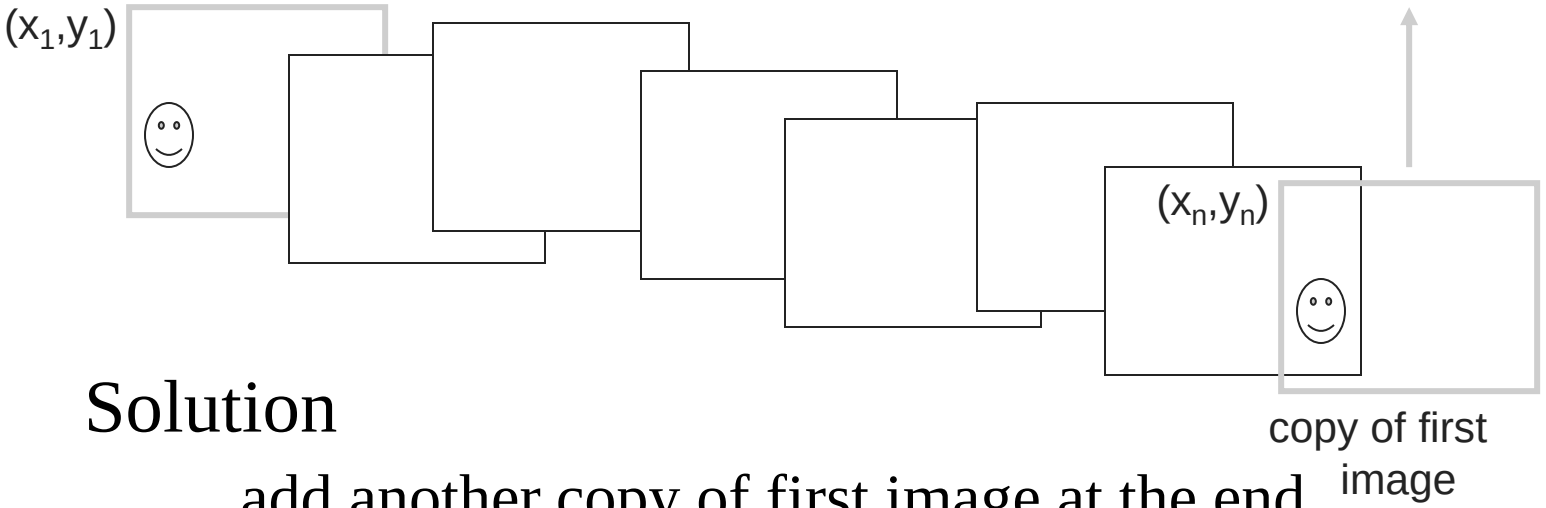
Error accumulation

small (vertical) errors accumulate over time

apply correction so that sum = 0 (for 360° pan.)

Szelis

Problem: Drift



Solution

add another copy of first image at the end

this gives a constraint: $y_n = y_1$

there are a bunch of ways to solve this problem

- add displacement of $(y_1 - y_n)/(n-1)$ to each image after the first
- compute a global warp: $y' = y + ax$
- run a big optimization problem, incorporating this constraint
  - best solution, but more complicated
  - known as "bundle adjustment"

Global alignment

- Register *all* pairwise overlapping images
- Use a 3D rotation model (one R per image)
- Use direct alignment (patch centers) or feature based
- *Infer* overlaps based on previous matches (incremental)
- Optionally *discover* which images overlap other images using feature selection (RANSAC)

Szelis

Homography for Rotation

Parameterise each camera by rotation and focal length

$$\mathbf{R}_i = e^{[\boldsymbol{\theta}_i]_\times}, \quad [\boldsymbol{\theta}_i]_\times = \begin{bmatrix} 0 & -\theta_{i3} & \theta_{i2} \\ \theta_{i3} & 0 & -\theta_{i1} \\ -\theta_{i2} & \theta_{i1} & 0 \end{bmatrix}$$

$$\mathbf{K}_i = \begin{bmatrix} f_i & 0 & 0 \\ 0 & f_i & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

This gives pairwise homographies

$$\tilde{\mathbf{u}}_i = \mathbf{H}_{ij}\tilde{\mathbf{u}}_j, \quad \mathbf{H}_{ij} = \mathbf{K}_i\mathbf{R}_i\mathbf{R}_j^T\mathbf{K}_j^{-1}$$

Error function

Sum of squared projection errors

$$e = \sum_{i=1}^{n} \sum_{j \epsilon \mathcal{I}(i)} \sum_{k \epsilon \mathcal{F}(i,j)} f(\mathbf{r}_{ij}^{k})^2$$

n = #images

I(i) = set of image matches to image i

F(i, j) = set of feature matches between images i,j

r_{ij}^{k} = residual of kth feature match between images i,j

Robust error function

$$f(\mathbf{x}) = \begin{cases} |\mathbf{x}|, & \text{if } |\mathbf{x}| < x_{max} \\ x_{max}, & \text{if } |\mathbf{x}| \geq x_{max} \end{cases}$$

Bundle adjustment formulations

Confidence / uncertainty of point i in image j

All pairs optimization:

$$E_{\text{all-pairs-2D}} = \sum_i \sum_{jk} c_{ij} c_{ik} \|\tilde{\boldsymbol{x}}_{ik}(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j, \boldsymbol{R}_k, f_k) - \hat{\boldsymbol{x}}_{ik}\|^2, \tag{9.29}$$

Map 2D point i in image j to 2D point in image k

Full bundle adjustment, using 3-D point positions $\{\boldsymbol{x}_i\}$

$$E_{\text{BA-2D}} = \sum_i \sum_j c_{ij} \|\tilde{\boldsymbol{x}}_{ij}(\boldsymbol{x}_i; \boldsymbol{R}_j, f_j) - \hat{\boldsymbol{x}}_{ij}\|^2, \tag{9.30}$$

Map 3D point i in to 2D point in image j

Bundle adjustment using 3-D ray:

$$E_{\text{BA-3D}} = \sum_i \sum_j c_{ij} \|\tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j) - \boldsymbol{x}_i\|^2, \tag{9.31}$$

3-D ray from point i

All-pairs 3-D ray formulation:

$$E_{\text{all-pairs-3D}} = \sum_i \sum_{jk} c_{ij} c_{ik} \|\tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j) - \tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ik}; \boldsymbol{R}_k, f_k)\|^2. \tag{9.32}$$

3-D ray from points i and j

Projected point → $\tilde{\boldsymbol{x}}_{ij} \sim \boldsymbol{K}_j \boldsymbol{R}_j \boldsymbol{x}_i$ and $\boldsymbol{x}_i \sim \boldsymbol{R}_j^{-1} \boldsymbol{K}_j^{-1} \tilde{\boldsymbol{x}}_{ij}$, ← *3-D ray from point*

Bundle Adjustment

New images initialised with rotation, focal length of best matching image



Bundle Adjustment

New images initialised with rotation, focal length of best matching image



Blending





Gain compensation

$$e = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} N_{ij}\left((g_i\bar{I}_{ij} - g_j\bar{I}_{ji})^2/\sigma_N^2 + (1-g_i)^2/\sigma_g^2\right)$$





How do we blend?

Linear blending



Multi-band blending

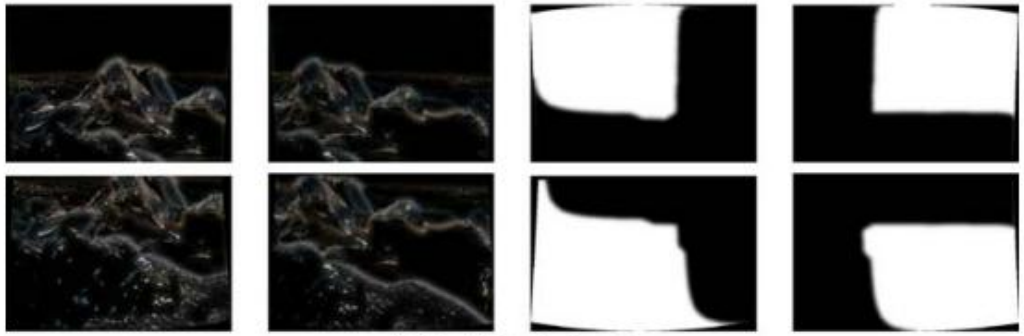


3-Band Blending

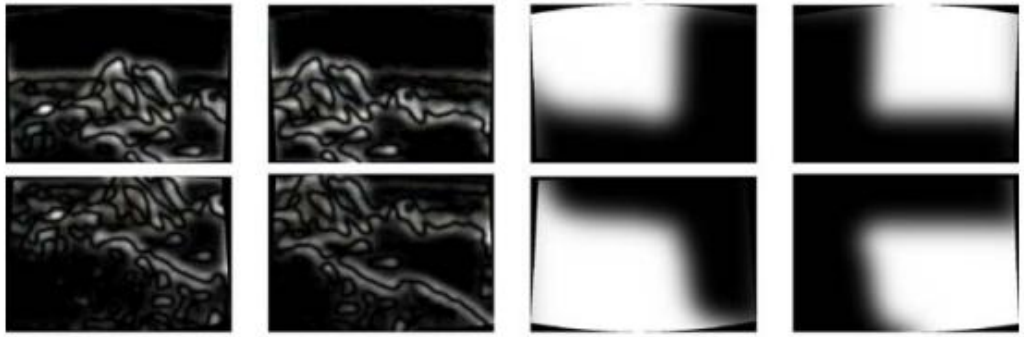


(a) Original images and blended result

3-Band Blending



(b) Band 1 (scale 0 to σ)



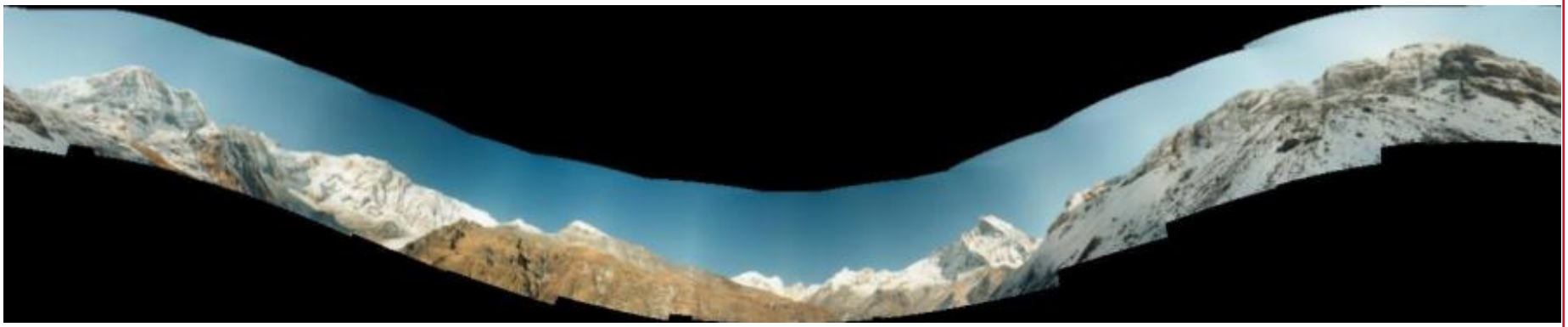
(c) Band 2 (scale σ to 2σ)



(d) Band 3 (scale lower than 2σ)

Panorama straightening

$$\left(\sum_{i=1}^{n} \mathbf{X}_i \mathbf{X}_i^T\right)\mathbf{u} = \mathbf{0}.$$





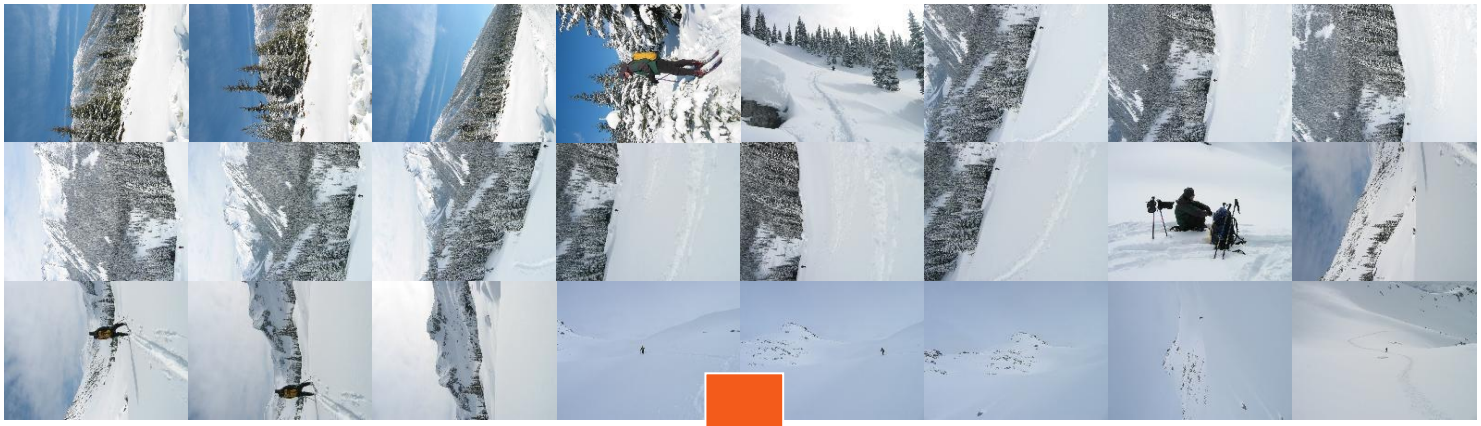
Recognizing Panoramas

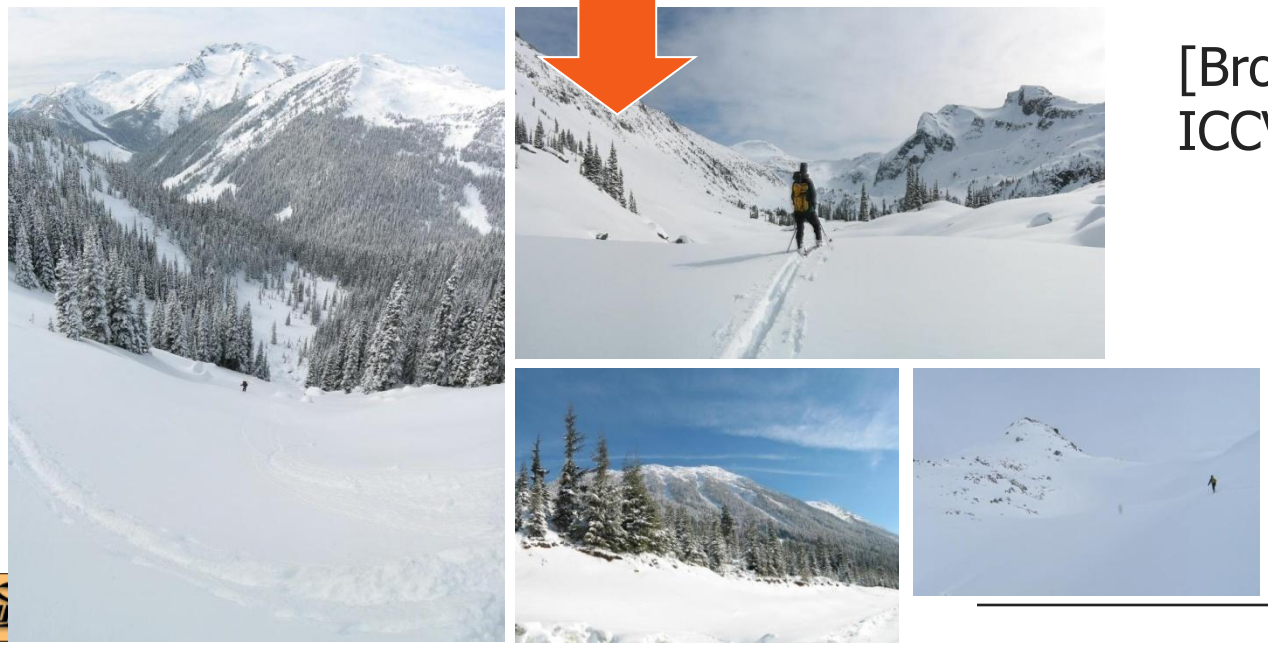
Matthew Brown & David Lowe

ICCV'2003

Szelis

Recognizing Panoramas

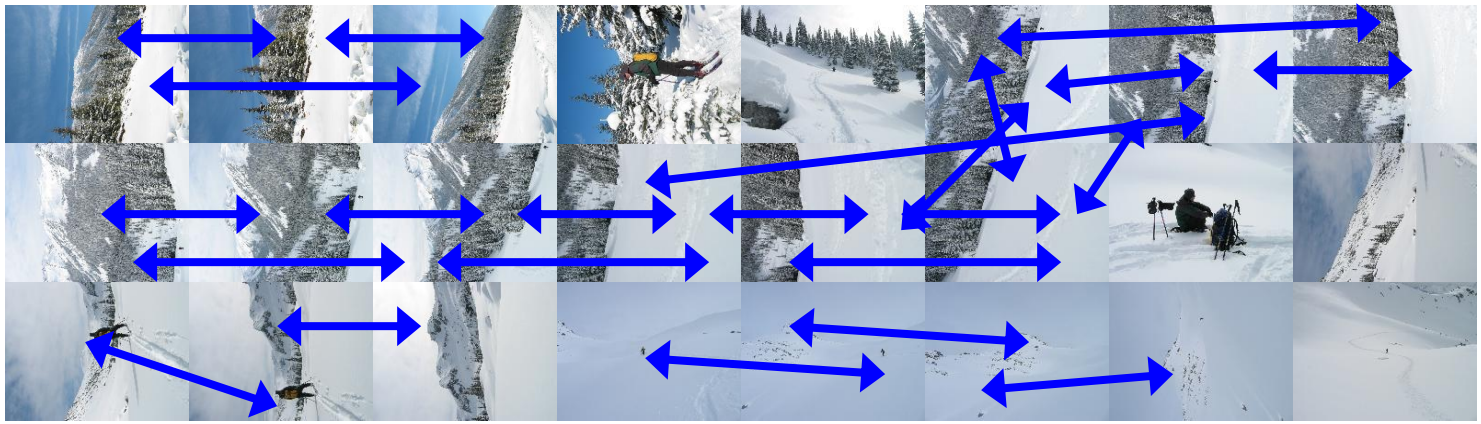




[Brown & Lowe, ICCV'03]

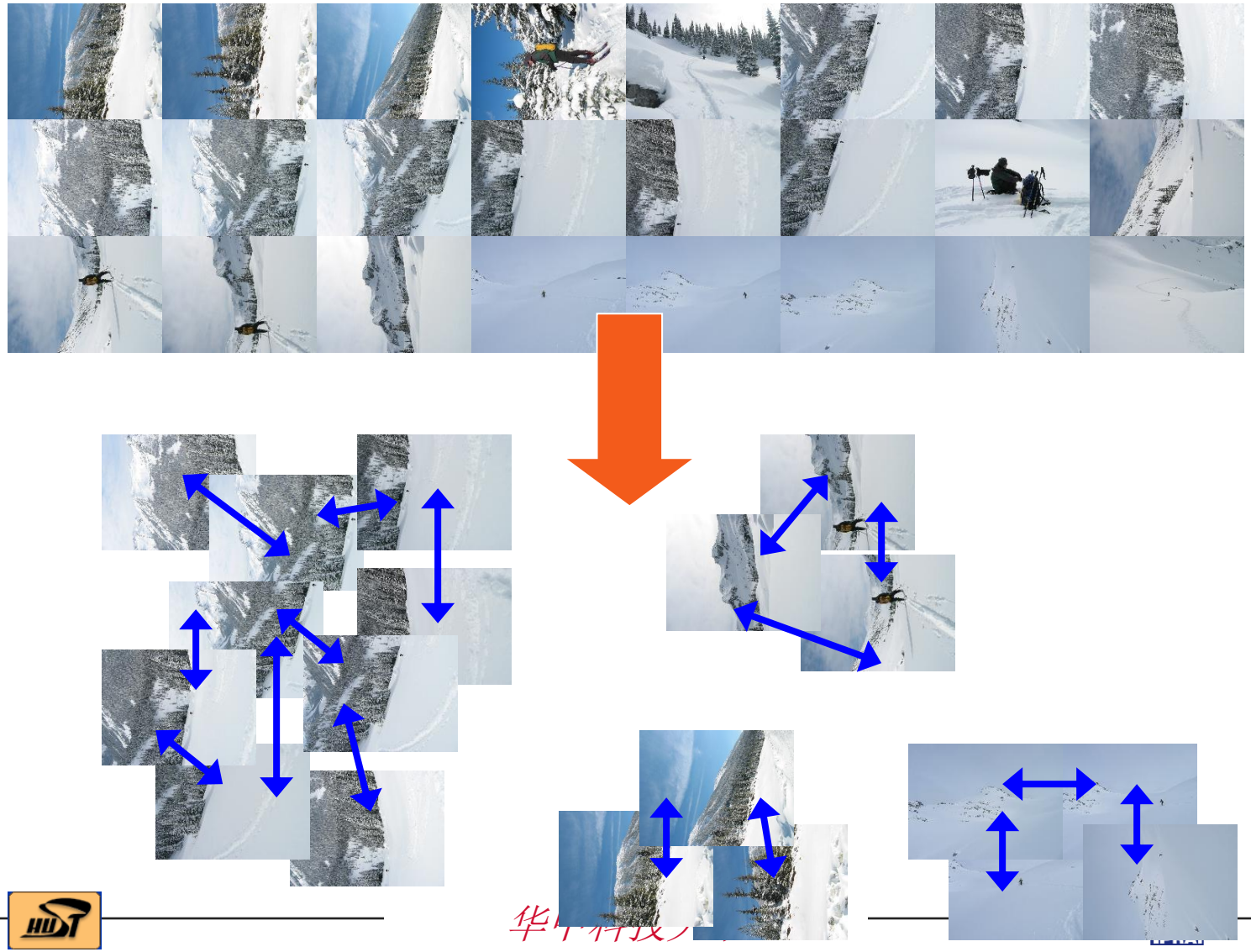
Szelis

Finding the panoramas



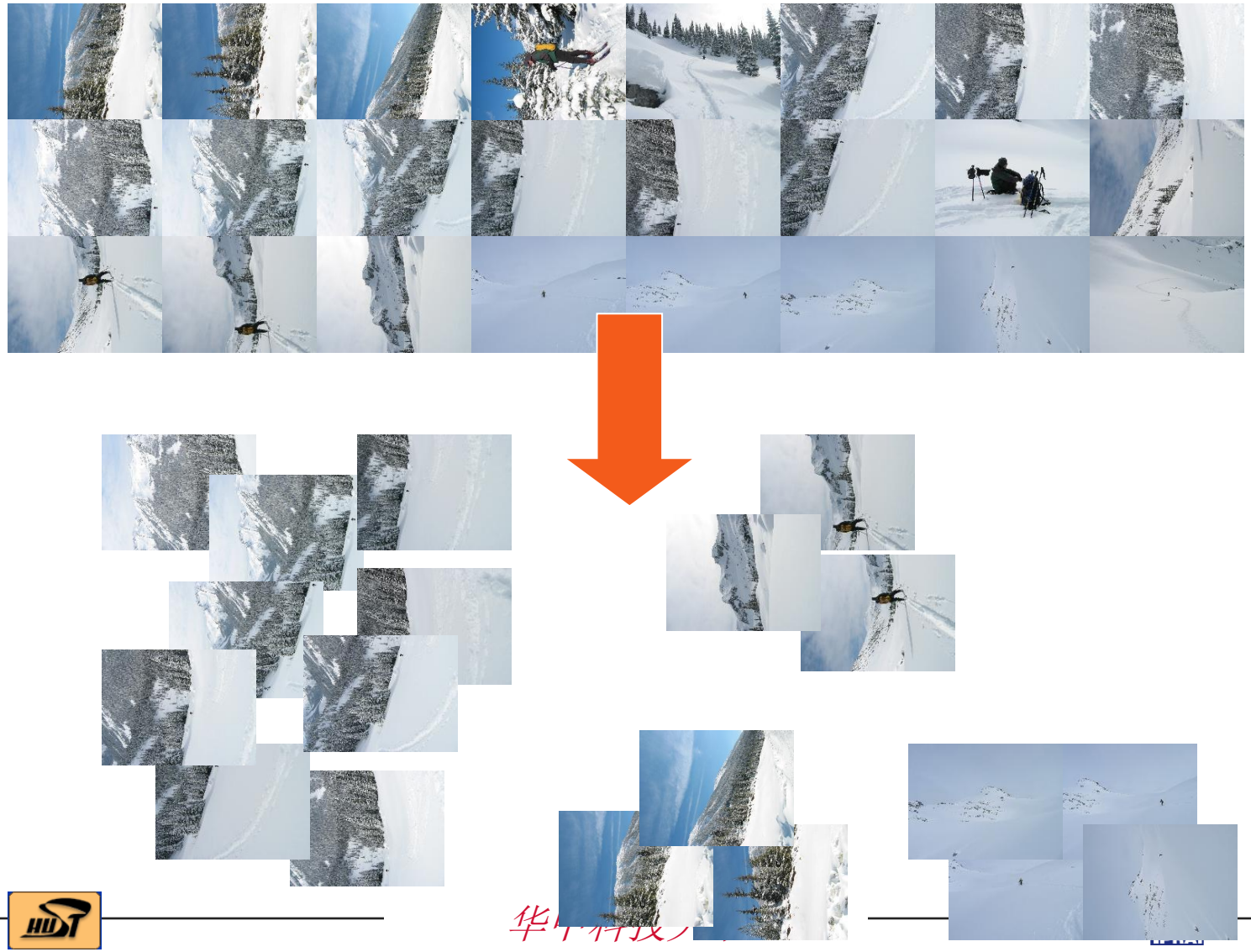
Szelis

Finding the panoramas



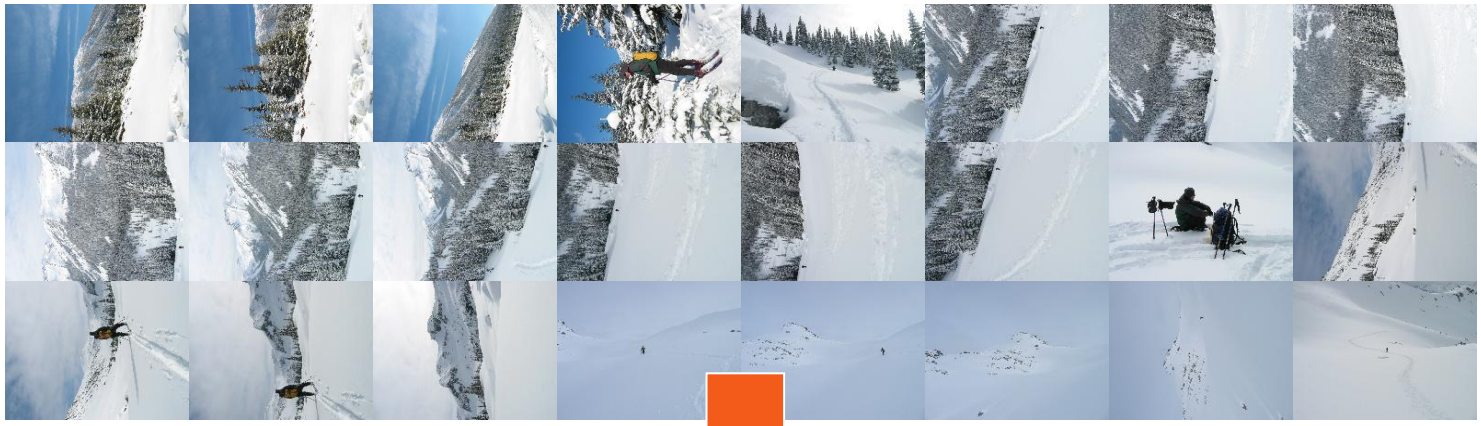
Szelis

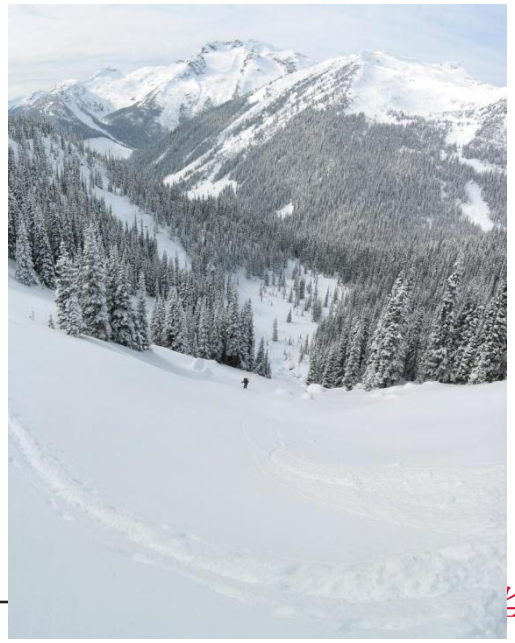
Finding the panoramas



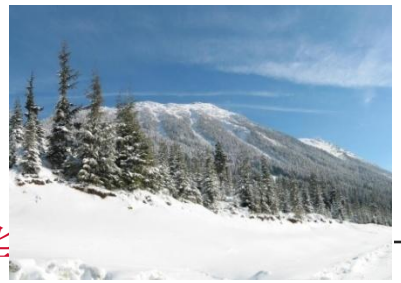
Szelis

Finding the panoramas





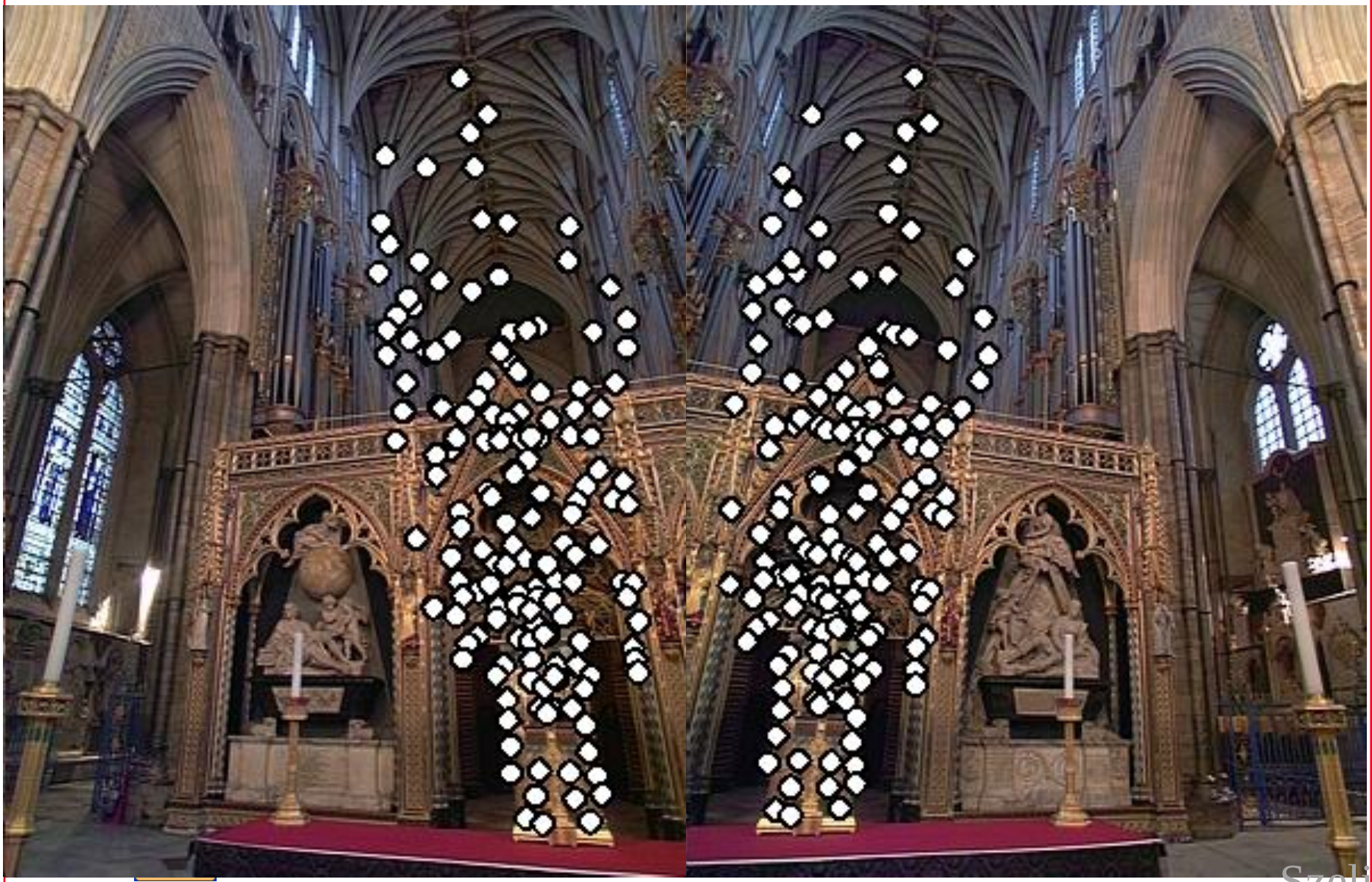






Szelis

Probabilistic Feature Matching



Matching Mistakes: False Positive





Szelis

Matching Mistakes: False Positive



Szelis

Matching Mistake: False Negative

Moving objects: large areas of disagreement



Szelis

Matching Mistakes

Accidental alignment

- repeated / similar regions

Failed alignments

- moving objects / parallax
- low overlap
- “feature-less” regions (more variety?)

No 100% reliable algorithm?

Szelis

How can we fix these?

Tune the feature detector

Tune the feature matcher (cost metric)

Tune the RANSAC stage (motion model)

Tune the verification stage

Use “higher-level” knowledge

e.g., typical camera motions

→ Sounds like a big “learning” problem

Need a large training/test data set (panoramas)

Szelis

Issues in alignment-based applications

- Choosing the geometric alignment model

  Tradeoff between "correctness" and robustness (also, efficiency)

- Choosing the descriptor

  "Rich" imagery (natural images): high-dimensional patch-based descriptors (e.g., SIFT)

  "Impoverished" imagery (e.g., star fields): need to create invariant geometric descriptors from k-tuples of point-based features

- Strategy for finding putative matches

  Small number of images, one-time computation (e.g., panorama stitching): brute force search

  Large database of model images, frequent queries: indexing or hashing

  Heuristics for feature-space pruning of putative matches

Issues in alignment-based applications

- Choosing the geometric alignment model
- Choosing the descriptor
- Strategy for finding putative matches
- Hypothesis generation strategy
  - Relatively large inlier ratio: RANSAC
  - Small inlier ratio: locality constraints, Hough transform
- Hypothesis verification strategy
  - Size of consensus set, residual tolerance depend on inlier ratio and expected accuracy of the model
  - Possible refinement of geometric model
  - Dense verification

Strategy 2: Incremental alignment

- Take advantage of strong locality constraints: only pick close-by matches to start with, and gradually add more matches in the same neighborhood

S. Lazebnik, C. Schmid and J. Ponce, **"Semi-local affine parts for object recognition,"** BMVC 2004.

Strategy 2: Incremental alignment

- Take advantage of strong locality constraints: only pick close-by matches to start with, and gradually add more matches in the same neighborhood





Strategy 2: Incremental alignment

- Take advantage of strong locality constraints: only pick close-by matches to start with, and gradually add more matches in the same neighborhood





Strategy 2: Incremental alignment

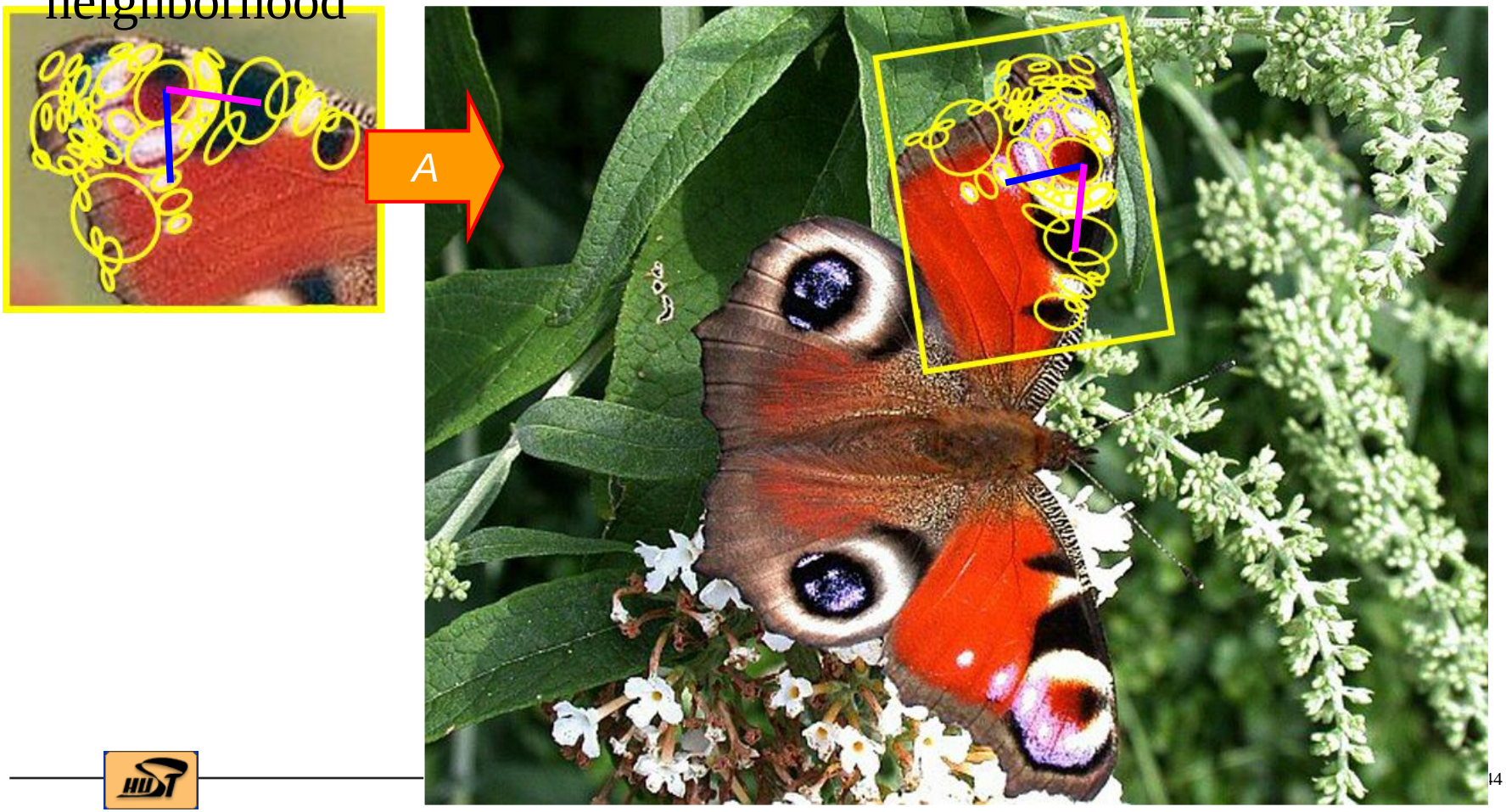
- Take advantage of strong locality constraints: only pick close-by matches to start with, and gradually add more matches in the same neighborhood





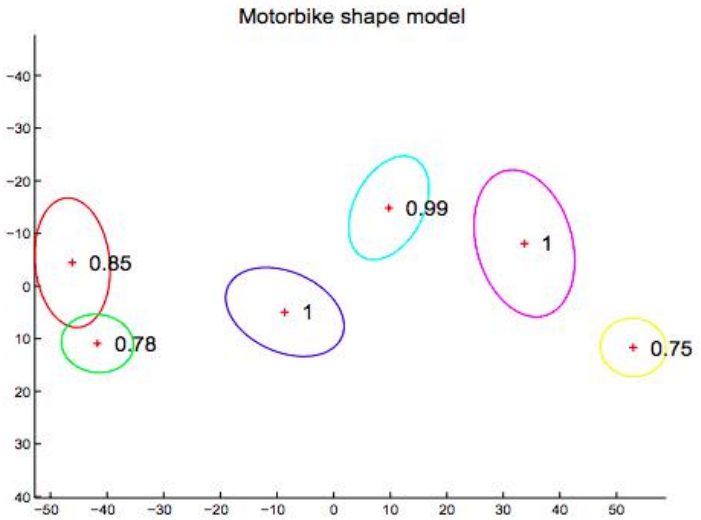
Strategy 2: Incremental alignment

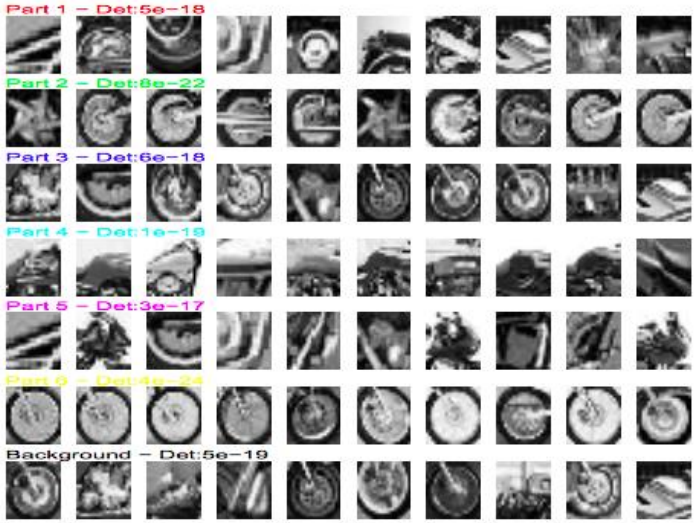
- Take advantage of strong locality constraints: only pick close-by matches to start with, and gradually add more matches in the same neighborhood

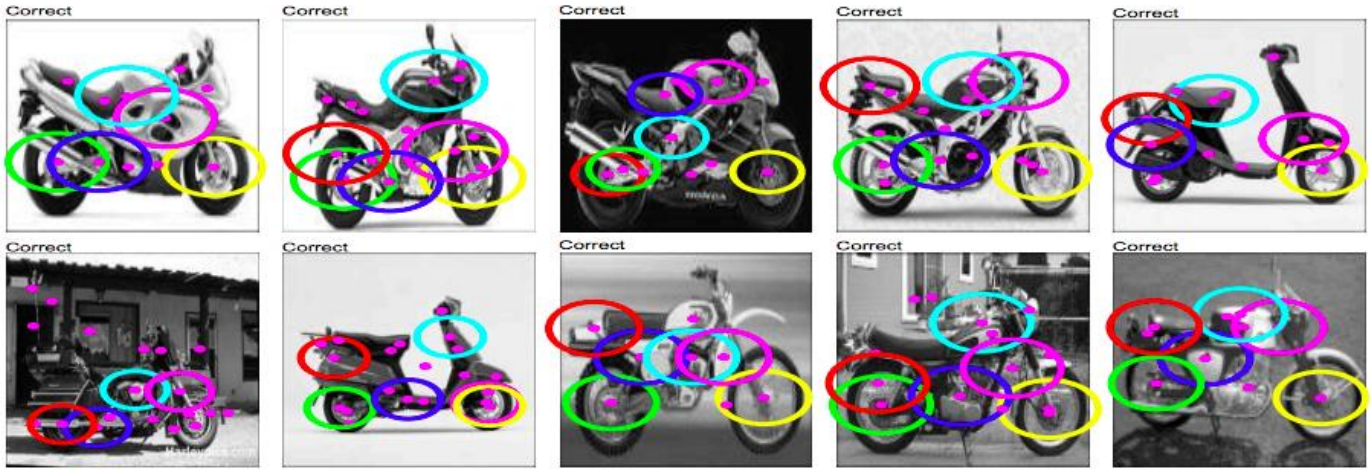


Strategy 3: Hough transform

Part-based models



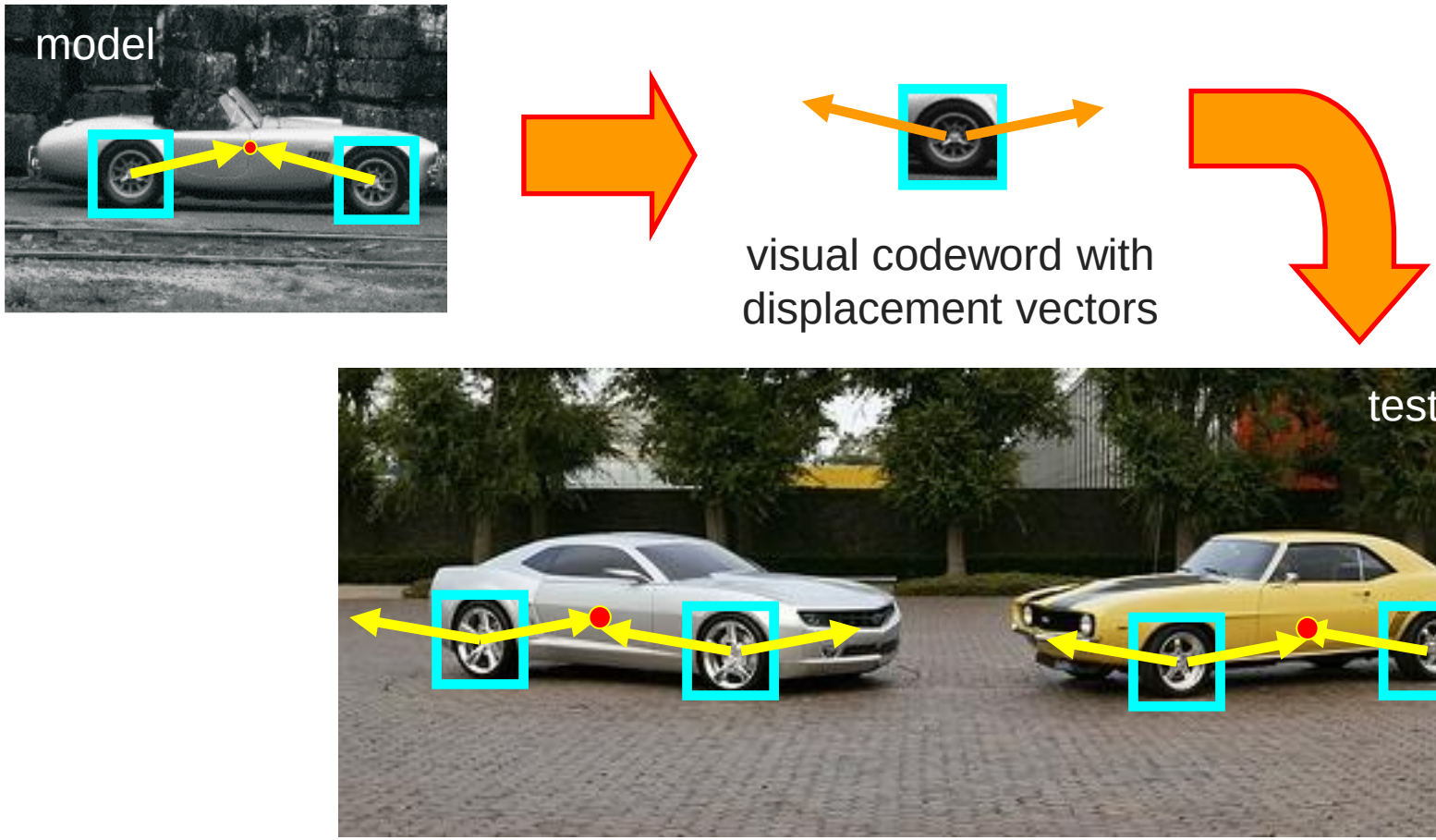




Weber, Welling & Perona (2000), Fergus, Perona & Zisserman (2003)

Strategy 3: Hough transform

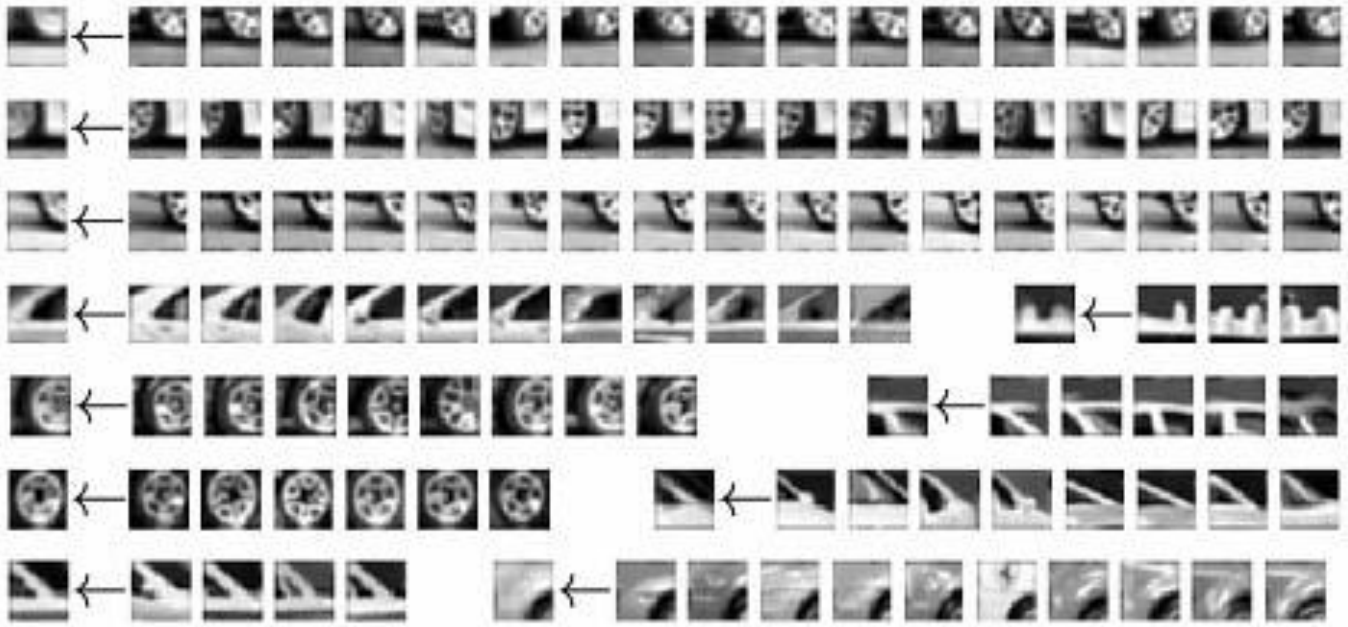
- Recall: Generalized Hough transform



B. Leibe, A. Leonardis, and B. Schiele, Combined Object Categorization and Segmentation with an Implicit Shape Model, ECCV Workshop on Statistical Learning in Computer Vision 2004

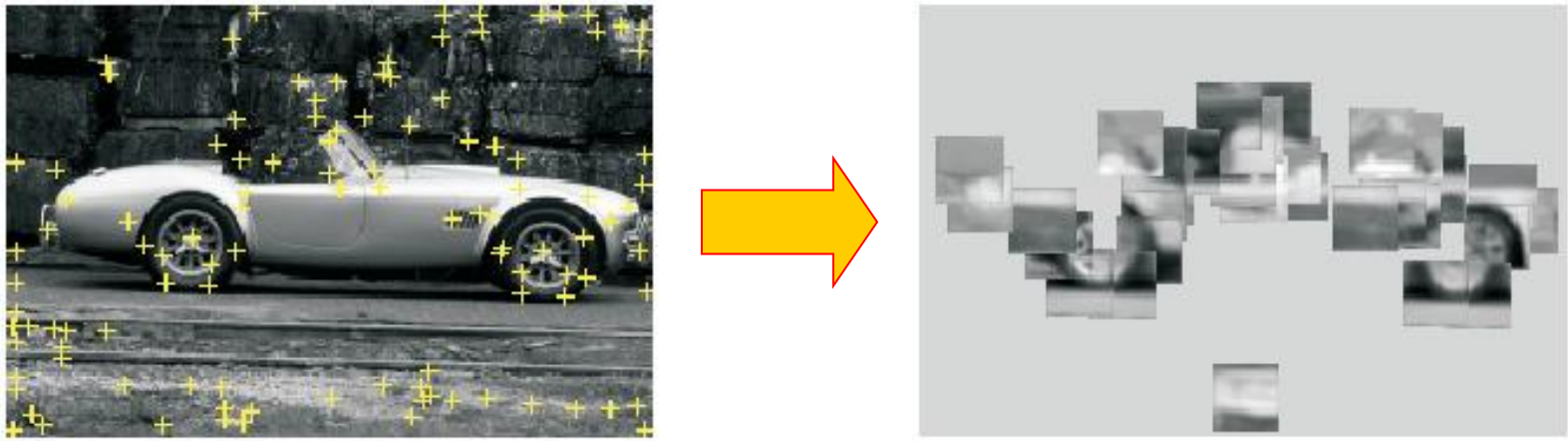
Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering



Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry



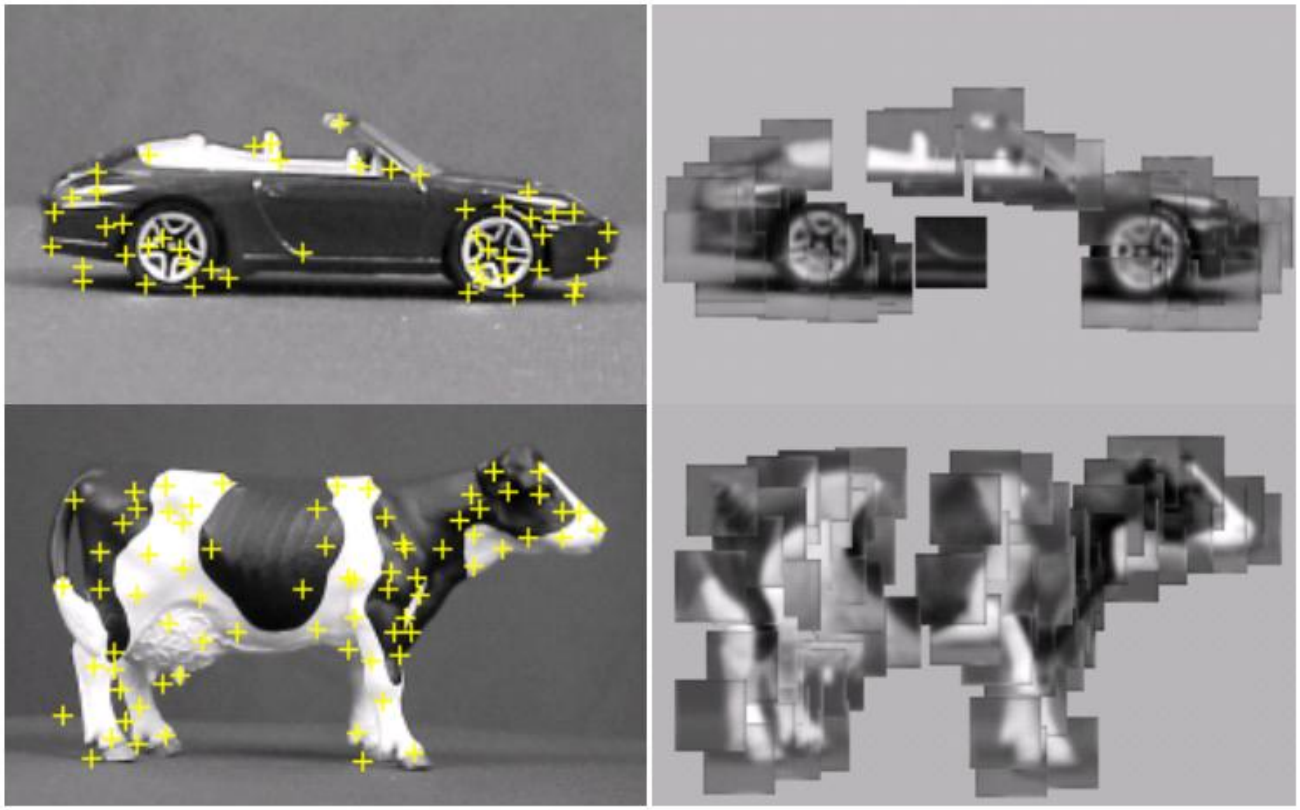
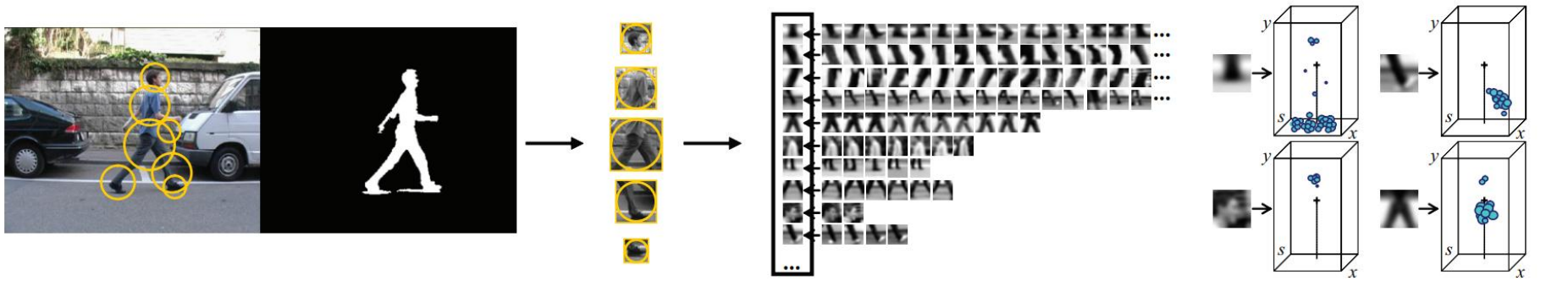


Fig. 1. *Local information used in the codebook generation process: (left) interest points; (right) features extracted around the interest points (visualized by the corresponding image patches). In most of our experiments, between 50 and 200 features are extracted per object.*

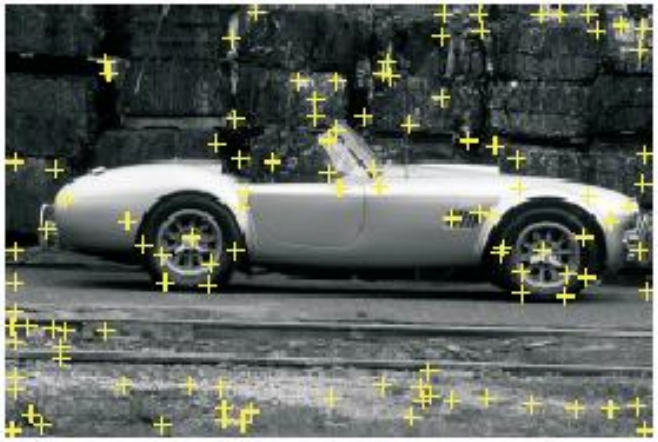
Implicit Shape Model $ISM(C) = (\mathcal{C}, P_{\mathcal{C}})$, which consists of a class-specific alphabet $\mathcal{C}$ (the *codebook*) of local appearances that are prototypical for the object category, and of a spatial probability distribution $P_{\mathcal{C}}$ which specifies where each codebook entry may be found on the object.

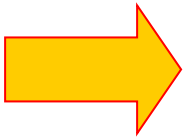


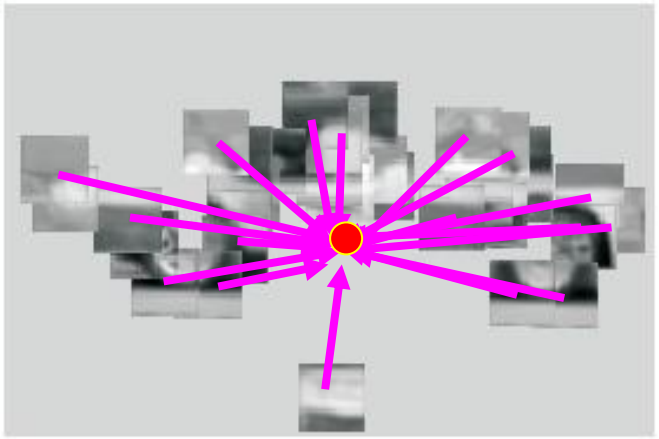
Training Images (+Reference Segmentations) | Local Features | Appearance Codebook (Cluster Centers) | Spatial Occurrence Distributions (non-parametric)

Implicit shape models: Training

1. Build *codebook* of patches around extracted interest points using clustering
2. Map the patch around each interest point to closest codebook entry
3. For each codebook entry, store all positions it was found, relative to object center







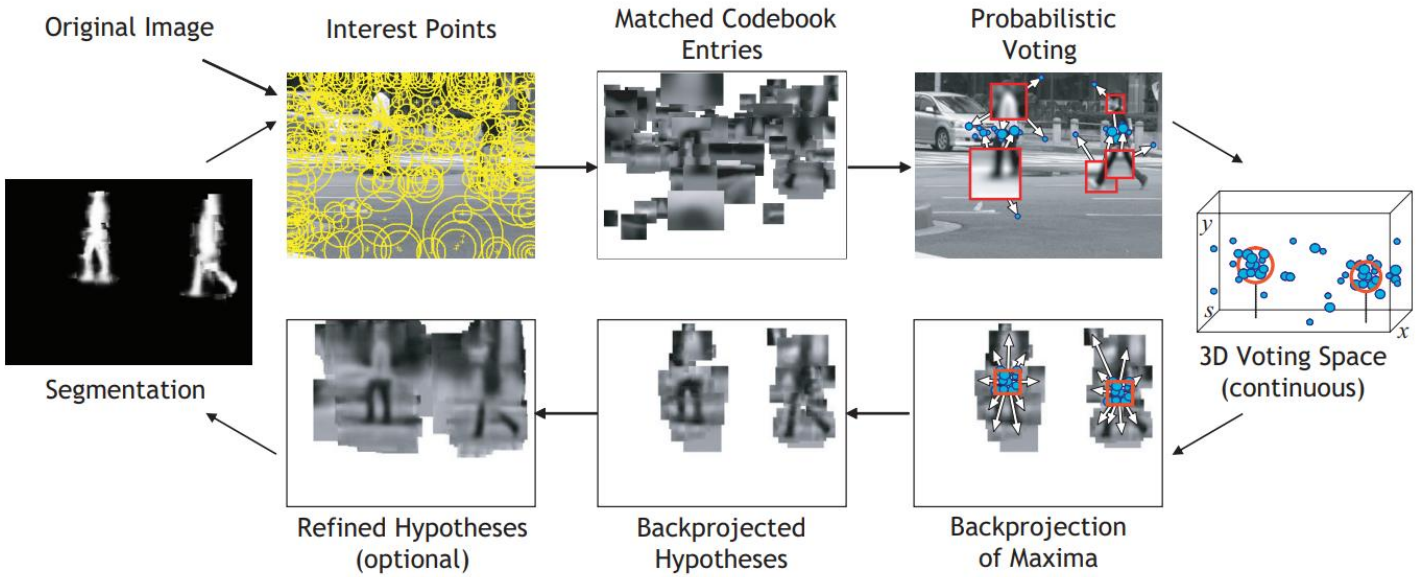
Algorithm 2 The training procedure.

// Create an appearance codebook $\mathcal{C}$.
$\mathcal{F} \leftarrow \emptyset$ *// Initialize the set of feature vectors $\mathcal{F}$*
for all training images **do**
  Apply the interest point detector.
  for all interest regions $\ell_k = (\ell_x, \ell_y, \ell_s)$ with descriptors f_k **do**
    $\mathcal{F} \leftarrow \mathcal{F} \cup f_k$
  end for
end for
Cluster $\mathcal{F}$ with cut-off threshold t and keep cluster centers $\mathcal{C}$.

// Compute occurrences Occ.
for all codebook entries $\mathcal{C}_i$ **do**
  $Occ[i] \leftarrow \emptyset$ *// Initialize occurrences for codebook entry $\mathcal{C}_i$*
end for
for all training images **do**
  Let (c_x, c_y) be the object center at a reference scale.
  Apply the interest point detector.
  for all interest regions $\ell_k = (\ell_x, \ell_y, \ell_s)$ with descriptors f_k **do**
    for all codebook entries $\mathcal{C}_i$ **do**
      if $sim(\mathcal{C}_i, f_k) \geq t$ **then**
        // Record an occurrence of codebook entry $\mathcal{C}_i$
        $Occ[i] \leftarrow Occ[i] \cup (c_x - \ell_x, c_y - \ell_y, \ell_s)$
      end if
    end for
  end for
end for

Recognition Approach

1) Probabilistic Hough Voting



$$p(o_n, x|f, \ell) = \sum_i p(o_n, x|\mathcal{C}_i, \ell)p(\mathcal{C}_i|f).$$
$$= \sum_i p(x|o_n, \mathcal{C}_i, \ell)p(o_n|\mathcal{C}_i, \ell)p(\mathcal{C}_i|f).$$

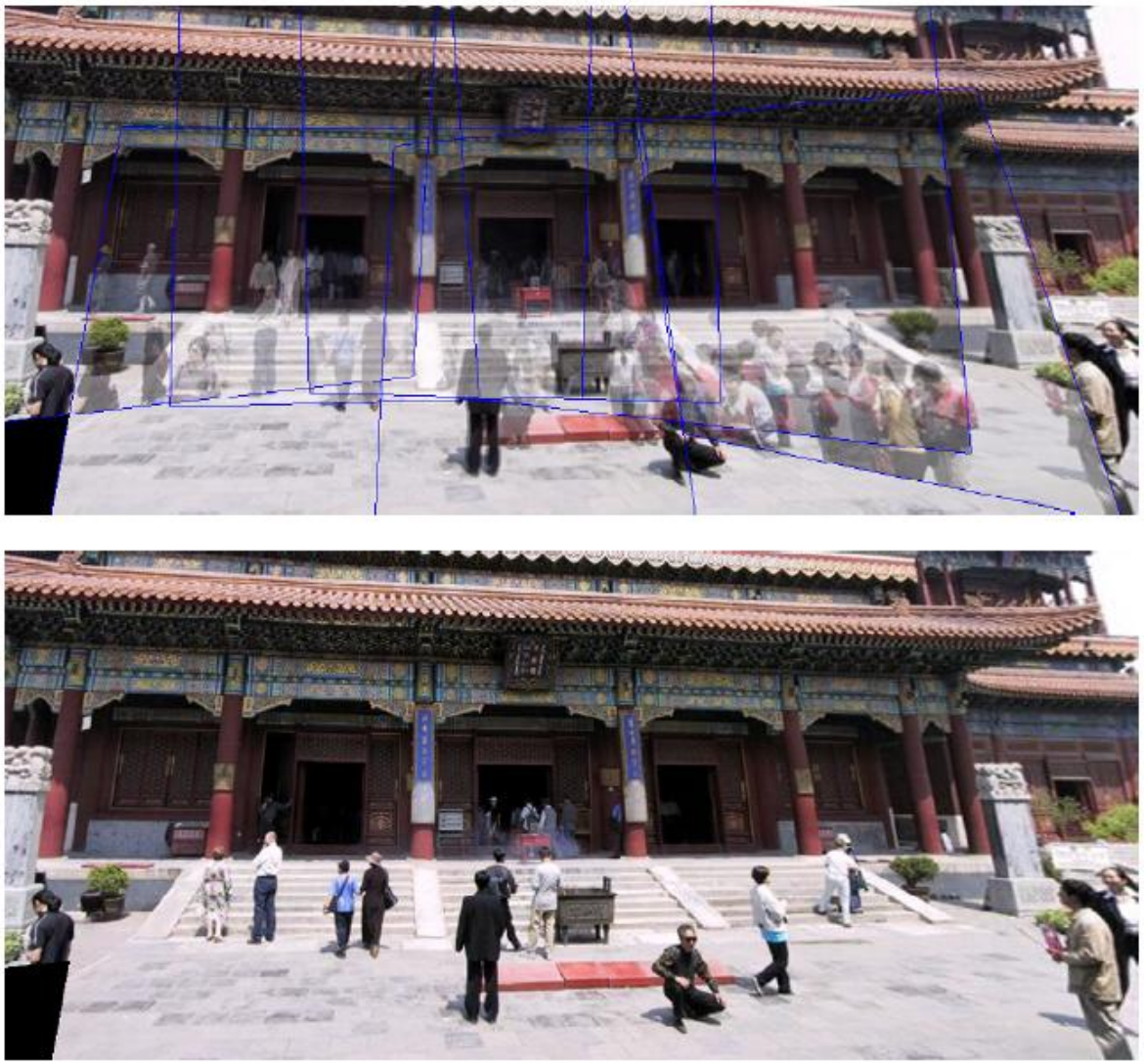
The first term is the probabilistic Hough vote for an object position given its class label and the feature interpretation. The second term specifies a confidence that the codebook cluster is really matched on the target category as opposed to the background. This can be used to include negative examples in the training process. Finally, the third term reflects the quality of the match between image feature and codebook cluster.

Algorithm 3 The ISM vote generation algorithm.

// Initialize the set of probabilistic votes $\mathcal{V}$.
$\mathcal{V} \leftarrow \emptyset$

Apply the interest point detector to the test image.
for all interest regions $\ell_k = (\ell_x, \ell_y, \ell_s)$ with descriptors f_k **do**
  // Initialize the set of matches $\mathcal{M}$
  $\mathcal{M} \leftarrow \emptyset$
  // Record all matches to the codebook
  for all codebook entries $\mathcal{C}_i$ **do**
    if $sim(f_k, \mathcal{C}_i) \geq t$ **then**
      $\mathcal{M} \leftarrow \mathcal{M} \cup (i, \ell_x, \ell_y, \ell_s)$ *// Record a match*
    end if
  end for
  for all matching codebook entries $\mathcal{C}_i^*$ **do**
    $p(\mathcal{C}_i^* | f_k) \leftarrow \frac{1}{|\mathcal{M}|}$ *//Set the match weight*
  end for
  // Cast the votes
  for all matches $(i, \ell_x, \ell_y, \ell_s) \in \mathcal{M}$ **do**
    for all occurrences $occ \in Occ[i]$ of codebook entry $\mathcal{C}_i$ **do**
      //Set the vote location
      $x \leftarrow (\ell_x - occ_x \frac{\ell_s}{occ_s}, \ell_y - occ_y \frac{\ell_s}{occ_s}, \frac{\ell_s}{occ_s})$
      //Set the occurrence weight
      $p(o_n, x | \mathcal{C}_i, \ell) \leftarrow \frac{1}{|Occ[i]|}$
      // Cast a vote (x, w, occ, ℓ) for position x with weight w
      $w \leftarrow p(o_n, x | \mathcal{C}_i, \ell) p(\mathcal{C}_i | f_k)$
      $\mathcal{V} \leftarrow \mathcal{V} \cup (x, w, occ, \ell)$
    end for
  end for
end for

De-Ghosting



Szelis

Local alignment (deghosting)

Use local optic flow to compensate for small motions [Shum & Szeliski, ICCV'98]



Figure 3: Deghosting a mosaic with motion parallax: (a) with parallax; (b) after single deghosting step (patch size 32); (c) multiple steps (sizes 32, 16 and 8).

Bundle adjustment formulations

Confidence / uncertainty of point i in image j

All pairs optimization:

$$E_{\text{all-pairs-2D}} = \sum_i \sum_{jk} c_{ij} c_{ik} \|\tilde{\boldsymbol{x}}_{ik}(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j, \boldsymbol{R}_k, f_k) - \hat{\boldsymbol{x}}_{ik}\|^2, \tag{9.29}$$

Map 2D point i in image j to 2D point in image k

Full bundle adjustment, using 3-D point positions $\{\boldsymbol{x}_i\}$

$$E_{\text{BA-2D}} = \sum_i \sum_j c_{ij} \|\tilde{\boldsymbol{x}}_{ij}(\boldsymbol{x}_i; \boldsymbol{R}_j, f_j) - \hat{\boldsymbol{x}}_{ij}\|^2, \tag{9.30}$$

Map 3D point i in to 2D point in image j

Bundle adjustment using 3-D ray:

$$E_{\text{BA-3D}} = \sum_i \sum_j c_{ij} \|\tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j) - \boldsymbol{x}_i\|^2, \tag{9.31}$$

3-D ray from point i

All-pairs 3-D ray formulation:

$$E_{\text{all-pairs-3D}} = \sum_i \sum_{jk} c_{ij} c_{ik} \|\tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ij}; \boldsymbol{R}_j, f_j) - \tilde{\boldsymbol{x}}_i(\hat{\boldsymbol{x}}_{ik}; \boldsymbol{R}_k, f_k)\|^2. \tag{9.32}$$

3-D ray from points i and j

Projected point → $\tilde{\boldsymbol{x}}_{ij} \sim \boldsymbol{K}_j \boldsymbol{R}_j \boldsymbol{x}_i$ and $\boldsymbol{x}_i \sim \boldsymbol{R}_j^{-1} \boldsymbol{K}_j^{-1} \tilde{\boldsymbol{x}}_{ij}$, ← *3-D ray from point*

Local alignment (deghosting)

Use local optic flow to compensate for small motions [Shum & Szeliski, ICCV'98， Construction and refinement of panoramic mosaics with global and local alignment]



Figure 3: Deghosting a mosaic with motion parallax: (a) with parallax; (b) after single deghosting step (patch size 32); (c) multiple steps (sizes 32, 16 and 8).

$$\mathbf{p}_j \sim \frac{1}{n_{jk}+1} \sum_{l \in \mathcal{N}_{jk} \cup k} \mathbf{R}_l^{-1} \mathbf{V}_l^{-1} \mathbf{x}_{jl}.$$

$$\bar{\mathbf{u}}_{jk} = \bar{\mathbf{x}}_{jk} - \mathbf{x}_{jk} = \frac{1}{n_{jk}+1} \sum_{l \in \mathcal{N}_{jk}} \mathbf{u}_{jl}.$$

$$\bar{\mathbf{x}}_{jk} \sim \mathbf{V}_k \mathbf{R}_k \frac{1}{n_{jk}+1} \sum_{l \in \mathcal{N}_{jk} \cup k} \mathbf{R}_l^{-1} \mathbf{V}_l^{-1} \mathbf{x}_{jl}$$

$$= \frac{1}{n_{jk}+1} \left(\mathbf{x}_{jk} + \sum_{l \in \mathcal{N}_{jk}} \tilde{\mathbf{x}}_{jl} \right).$$

Local alignment (deghosting)

Use local optic flow to compensate for radial distortion [Shum & Szeliski, ICCV'98]



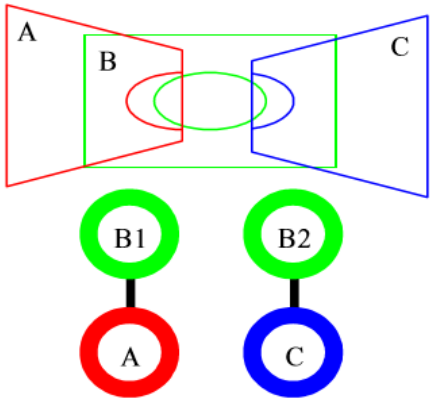
Figure 4: Deghosting a mosaic with optical distortion: (a) with distortion; (b) after multiple steps.

Szelis

Region-based de-ghosting

Select only one image in *regions-of-difference* using weighted vertex cover
[Uyttendaele *et al.*, CVPR'01]

RODs (vertices) are given a weight proportional to its size and proximity to the center of its image





(A)

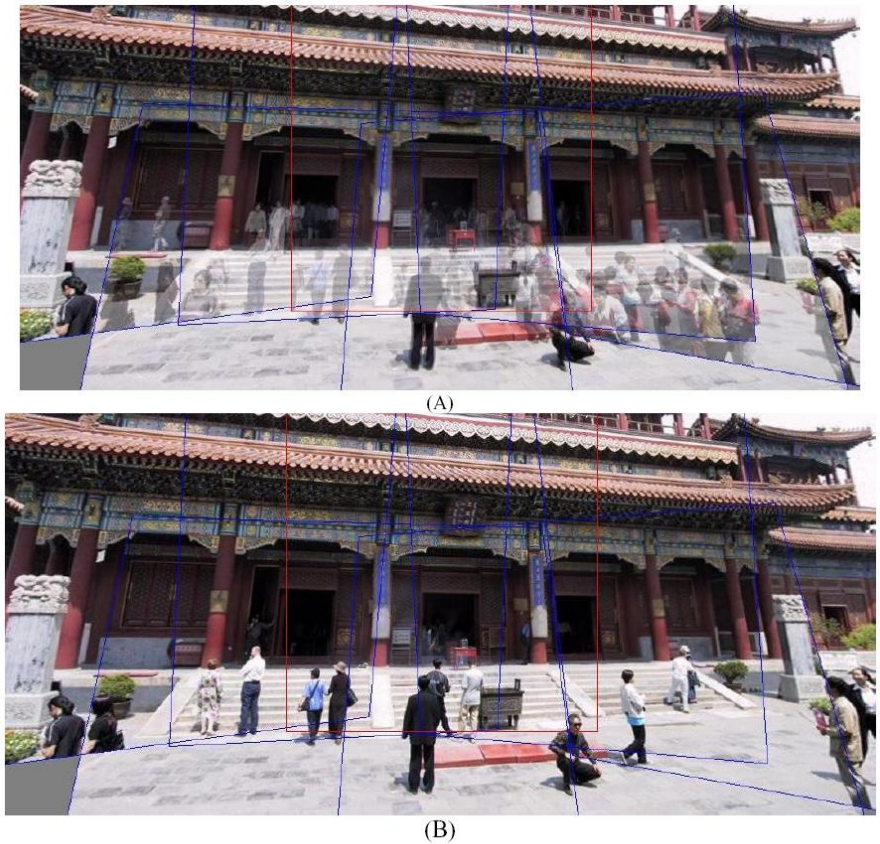


(B)

Figure 5 – (A) Ghosted mosaic. (B) Result of de-ghosting algorithm.

Region-based de-ghosting

Select only one image in *regions-of-difference* using weighted vertex cover [Uyttendaele *et al.*, CVPR'01]



(A)

(B)

Figure 6 – (A) Ghosted mosaic. (B) Result of de-ghosting algorithm.

Szelis

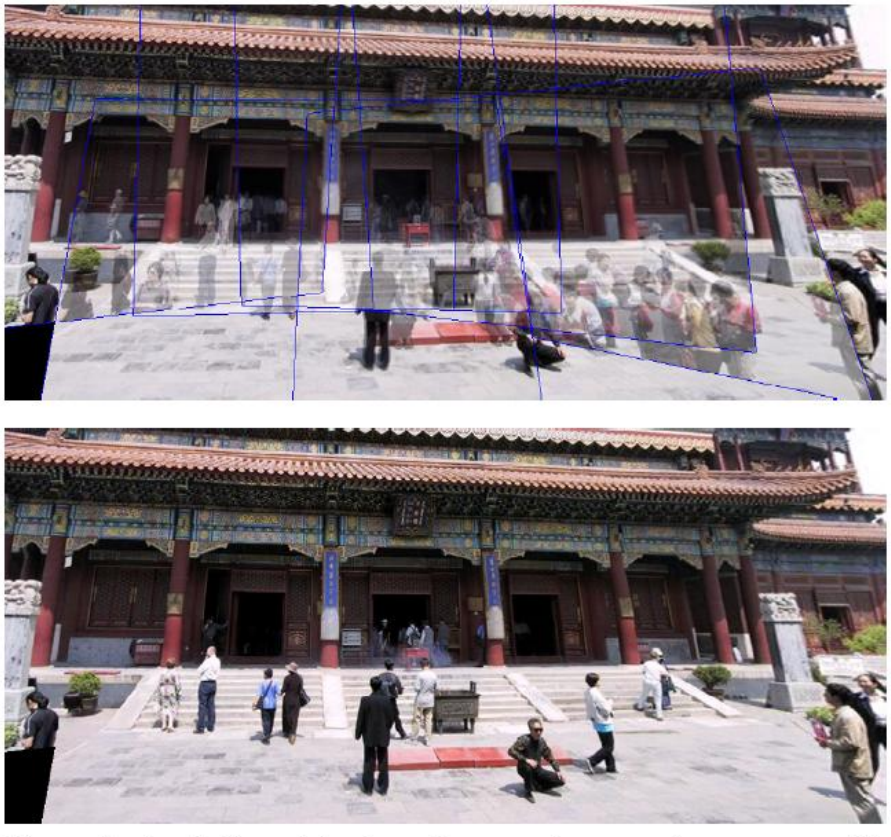
Cutout-based de-ghosting

- Select only one image per output pixel, using spatial continuity
- Blend across seams using gradient continuity ("Poisson blending")
[Agarwala *et al.*, SG'2004]

Poisson blending

$$\min_f \iint_\Omega |\nabla f - \mathbf{v}|^2 \text{ with } f|_{\partial\Omega} = f^*|_{\partial\Omega}$$

$$\Delta f = \text{div}\mathbf{v} \text{ over } \Omega, \text{ with } f|_{\partial\Omega} = f^*|_{\partial\Omega}$$



Szelis

CNN based de-ghosting



Fig. 1: HDR imaging pipeline.

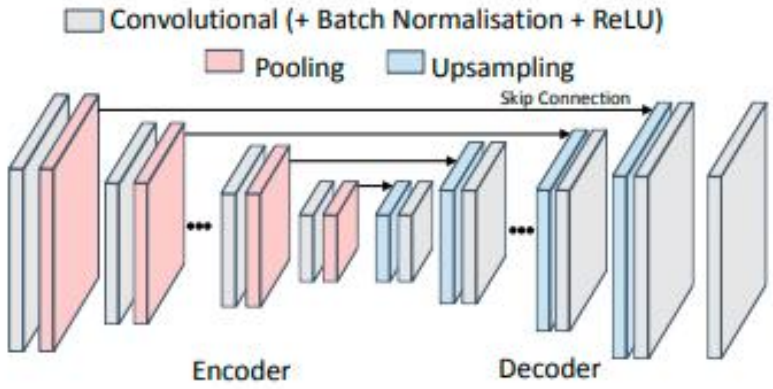
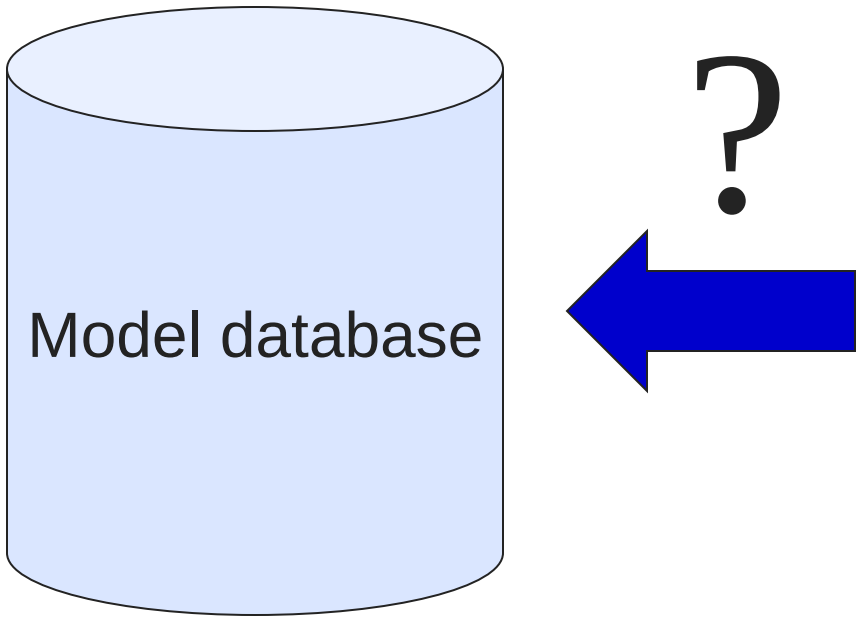


Fig. 2: An illustration of the CNN-based architecture.

Y Hu et.al ICIP2019

Scalability: Alignment to large databases

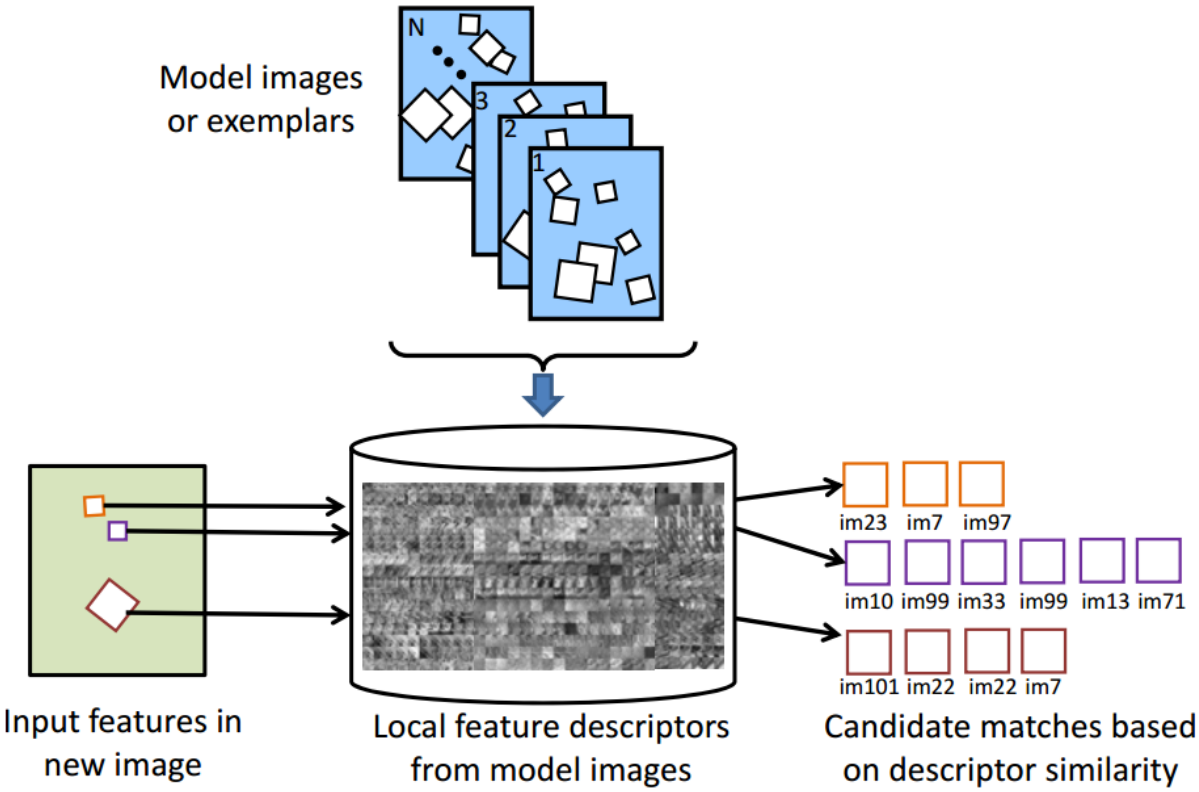
- What if we need to align a test image with thousands or millions of images in a model database?
  - Efficient putative match generation
    - Approximate descriptor similarity search, inverted indices



Test image



How to do the indexing?



- Cluster descriptors in the database to form codebook
- At query time, quantize descriptors in query image to nearest codevectors
- Problem solved?

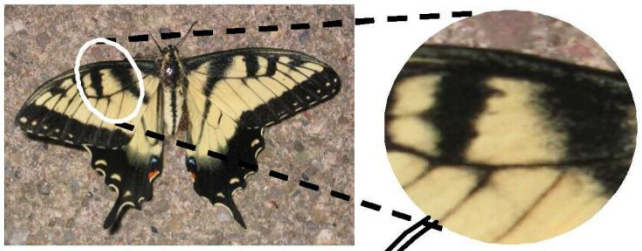
Scalability: Alignment to large databases

- What if we need to align a test image with thousands or millions of images in a model database?

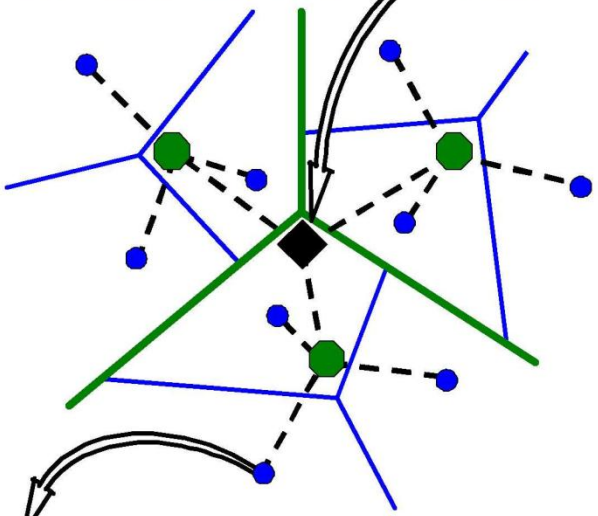
  Efficient putative match generation

  - Fast nearest neighbor search, inverted indexes

Test image



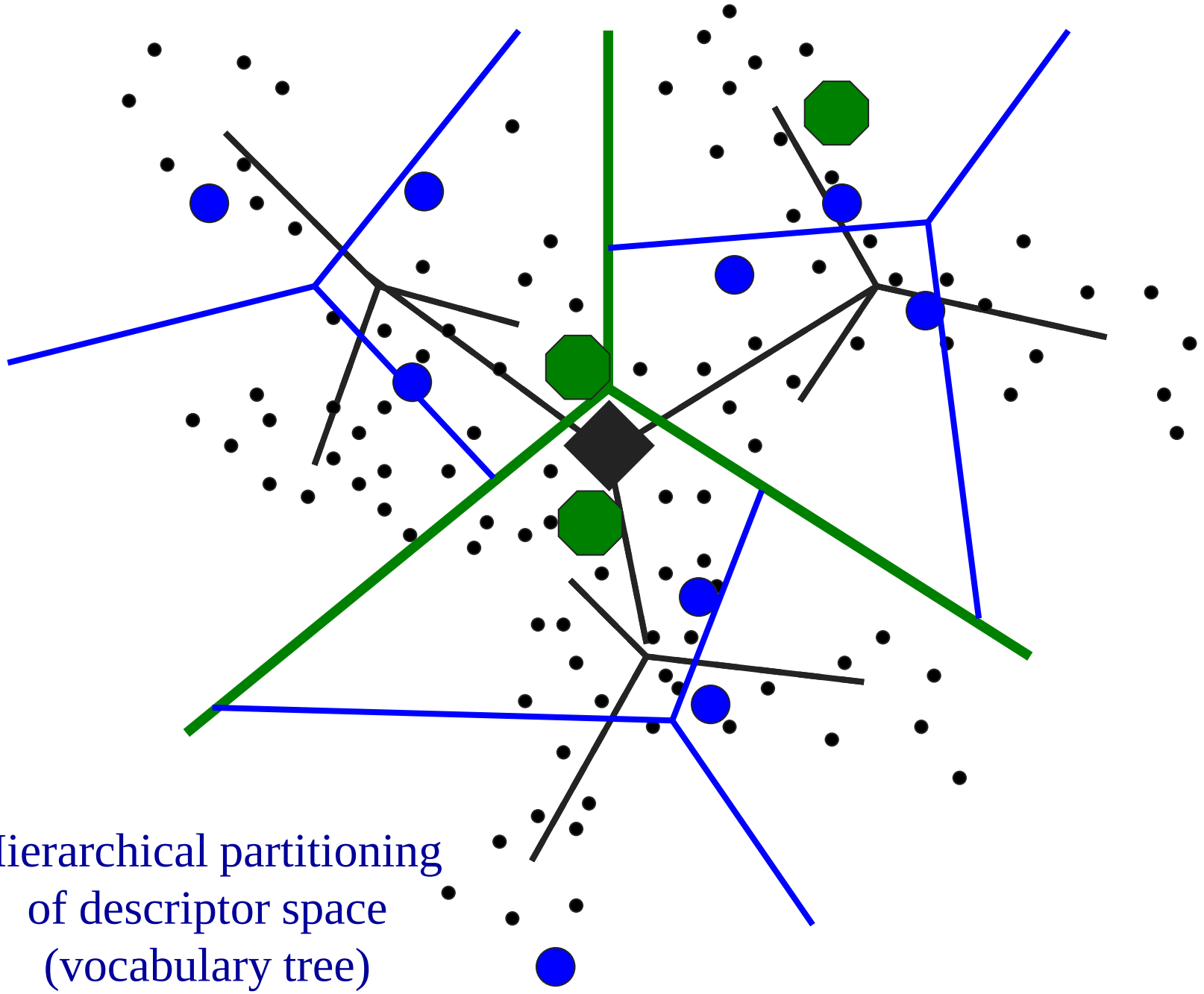
Vocabulary tree with inverted index



Database

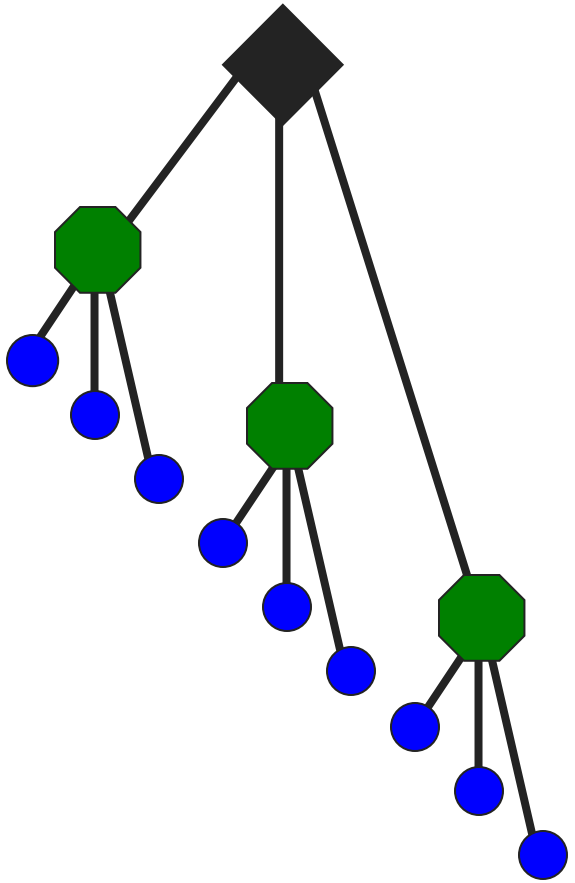


D. Nistér and H. Stewénius, Scalable Recognition with a Vocabulary Tree, CVPR 2006



Hierarchical partitioning
of descriptor space
(vocabulary tree)

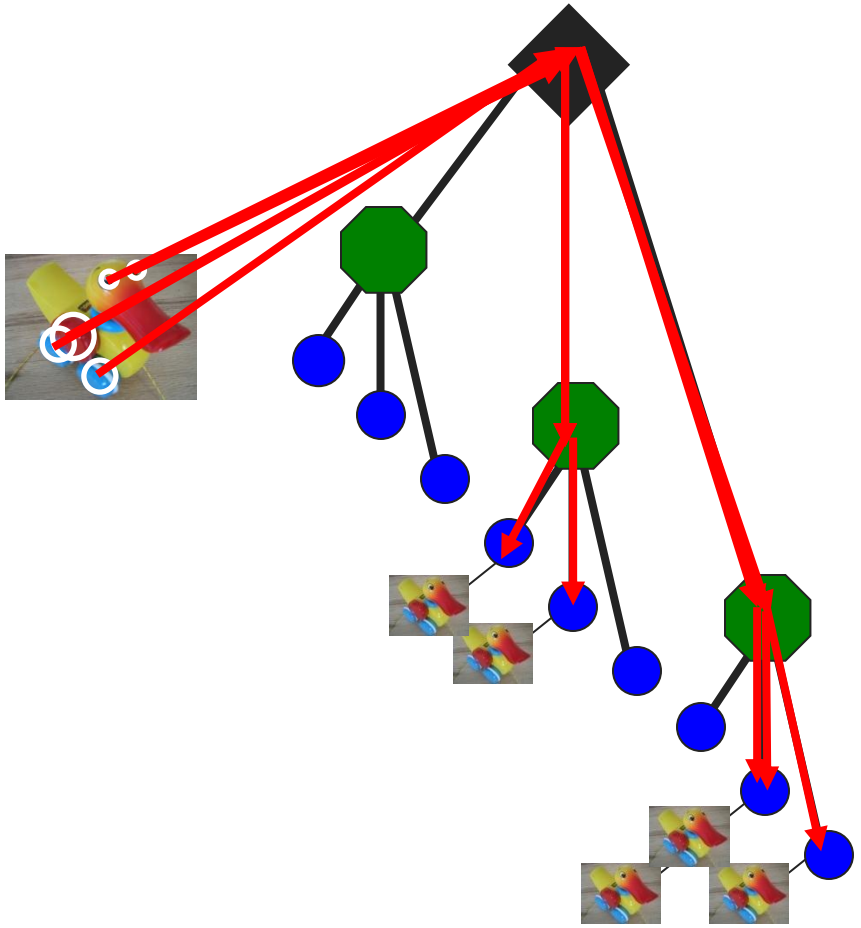
Slide credit: D. Nister



Vocabulary tree/inverted index

Slide credit: D. Nister

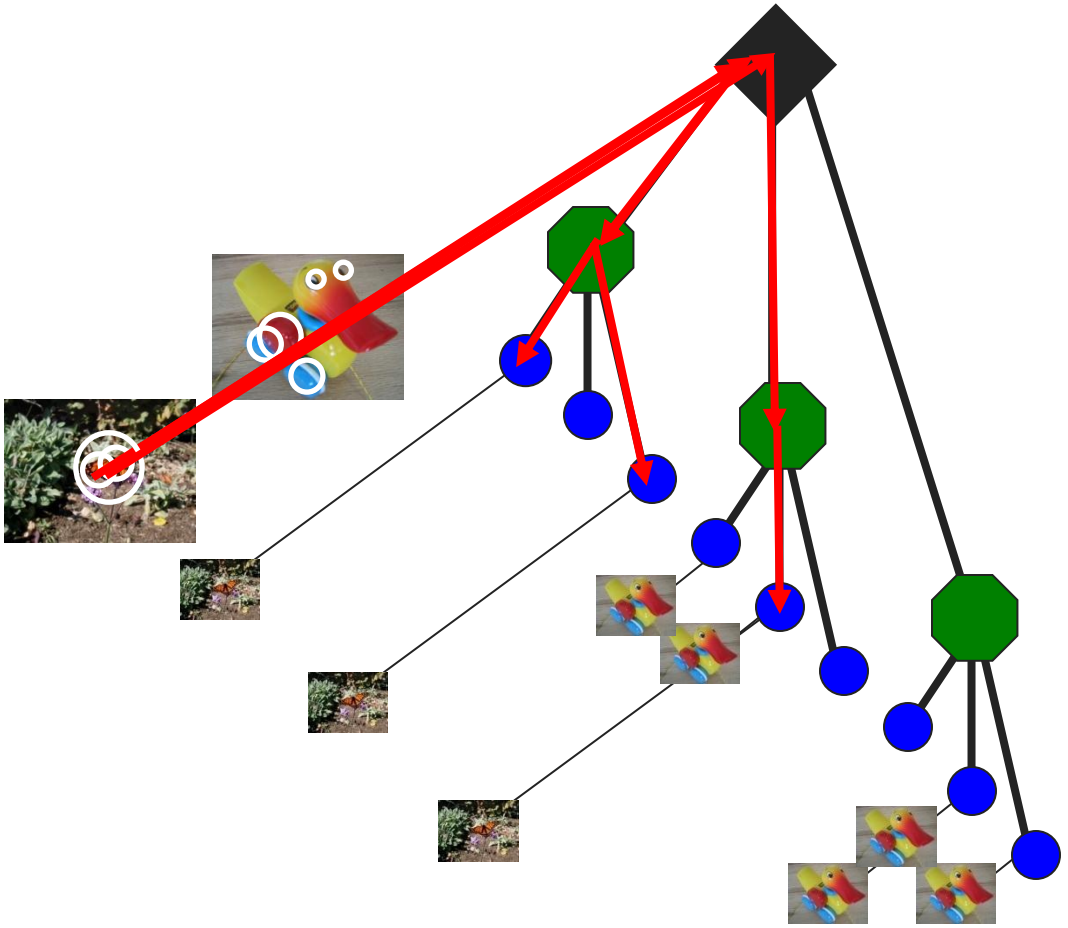
Model images



Populating the vocabulary tree/inverted index

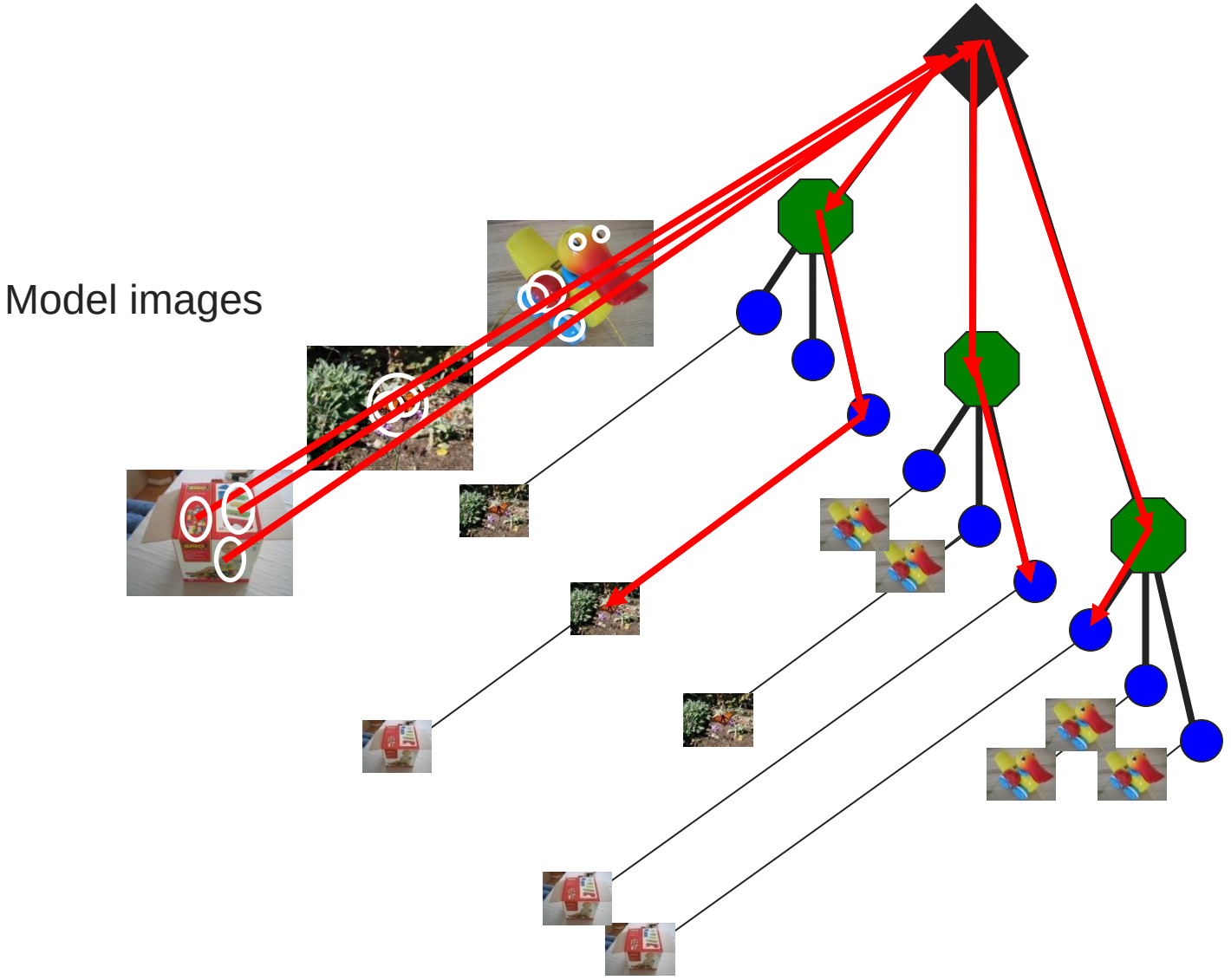
Slide credit: D. Nister

Model images



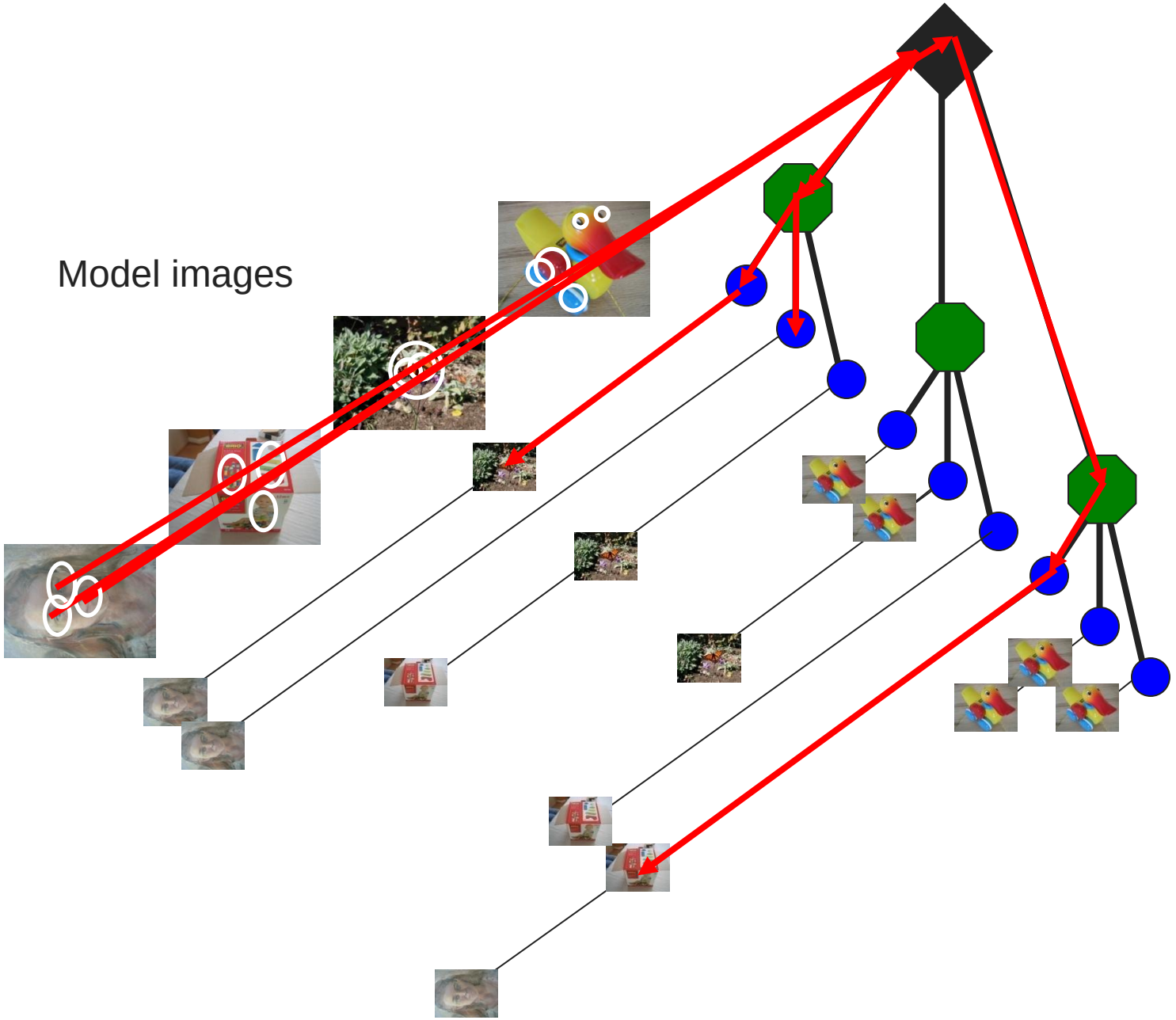
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



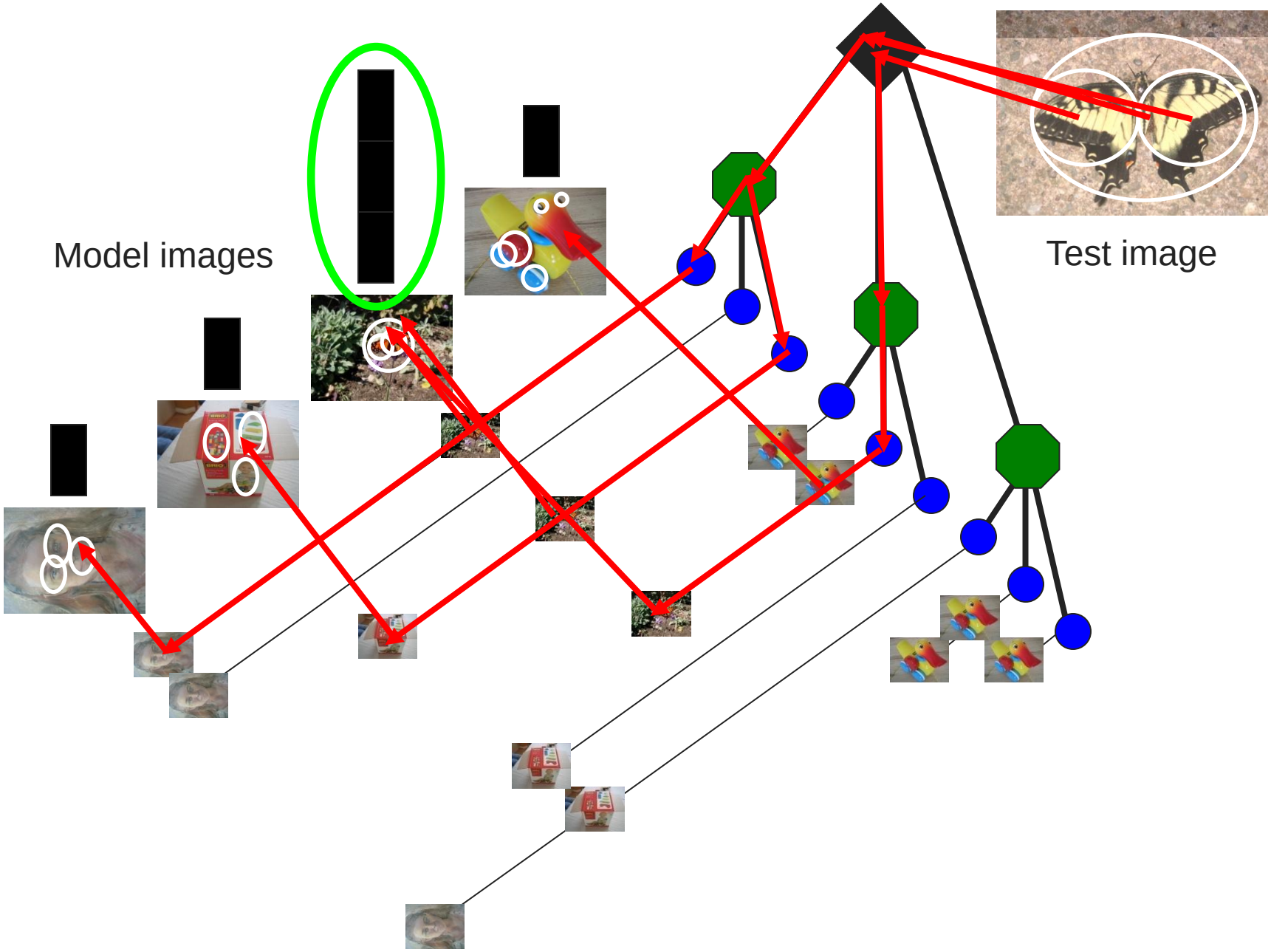
Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Populating the vocabulary tree/inverted index

Slide credit: D. Nister



Looking up a test image

Slide credit: D. Nister

Other types of mosaics



Can mosaic onto *any* surface if you know the geometry

See NASA's Visible Earth project for some stunning earth mosaics

http://earthobservatory.nasa.gov/Newsroom/BlueMarble/